

Common DP Patterns — Part 2

A comprehensive guide to advanced Dynamic Programming patterns for SDE placements, online assessments, and competitive programming.

1. Edit Distance

1. Overview

Edit Distance (also called Levenshtein Distance) is a DP problem that finds the minimum number of single-character operations (insert, delete, replace) required to convert one string into another. It's a classic string alignment problem that measures how dissimilar two strings are.

2. Intuition

Think of editing a document — you can type a new letter (insert), backspace (delete), or change a letter (replace). The edit distance is the minimum number of such keystrokes to turn word A into word B.

Core idea: If you're comparing two strings character by character, the characters either match (no operation needed) or don't match. If they don't match, you have three choices:

1. **Insert** a character into the first string to match the second
2. **Delete** a character from the first string
3. **Replace** a character in the first string with one from the second

The optimal choice is the one that costs the least. This naturally leads to DP: build a table where `dp[i][j]` is the edit distance between the first `i` characters of string A and first `j` characters of string B.

3. When to Use It

- Problems asking "minimum operations to convert string A to string B"
- String similarity / alignment problems
- Autocorrect / spell-checker systems
- Bioinformatics (DNA sequence alignment)
- Problems with insert/delete/replace operations with custom costs

Trigger phrases: "minimum number of operations", "convert string", "edit distance", "one edit away", "insert delete replace"

4. When Not to Use It

- When only one operation is allowed (e.g., only insert) — use simpler greedy/LCS
- When strings are very long and only approximate distance is needed — use heuristic algorithms
- When you need all possible alignments, not just the minimum cost — use backtracking + DP
- For very large strings (10^5+) — $O(n \times m)$ is too slow; use Myers' algorithm or similar

5. Core Concepts

5.1 Operations

- **Insert:** Add a character to string A. Cost = 1 (or custom cost). $dp[i][j] = dp[i][j-1] + 1$
- **Delete:** Remove a character from string A. Cost = 1. $dp[i][j] = dp[i-1][j] + 1$
- **Replace:** Change a character in string A. Cost = 1. $dp[i][j] = dp[i-1][j-1] + 1$

5.2 DP Table

A 2D table where $dp[i][j]$ = edit distance between $A[0..i-1]$ and $B[0..j-1]$.

5.3 Base Case

- $dp[0][j] = j$ (need to insert j characters into empty string)
- $dp[i][0] = i$ (need to delete i characters from A)

5.4 Recurrence

```
If A[i-1] == B[j-1]: dp[i][j] = dp[i-1][j-1] (match, no cost)
Else: dp[i][j] = 1 + min(dp[i-1][j],      // delete
                        dp[i][j-1],      // insert
                        dp[i-1][j-1])    // replace
```

6. Step-by-Step Algorithm

1. Let $n = \text{len}(A)$, $m = \text{len}(B)$
2. Create DP table of size $(n+1) \times (m+1)$
3. Initialize first row: $dp[0][j] = j$ for all j
4. Initialize first column: $dp[i][0] = i$ for all i
5. For i from 1 to n:
 - For j from 1 to m:
 - If $A[i-1] == B[j-1]$: $dp[i][j] = dp[i-1][j-1]$
 - Else: $dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$
6. Answer = $dp[n][m]$

7. Dry Run

Convert "CAT" → "DOG"

		D	O	G
C	1	1	2	3
A	2	2	2	3
T	3	3	3	3

Step-by-step:

- $dp[1][1]$: C vs D $\rightarrow \min(1,1,0)+1 = 1$ (replace)
- $dp[1][2]$: C vs DO $\rightarrow \min(2,1,1)+1 = 2$ (delete C then insert O)
- $dp[1][3]$: C vs DOG $\rightarrow \min(3,2,2)+1 = 3$
- $dp[2][1]$: CA vs D $\rightarrow \min(2,2,1)+1 = 2$
- $dp[2][2]$: CA vs DO $\rightarrow \min(2,1,1)+1 = 2$
- $dp[2][3]$: CA vs DOG $\rightarrow \min(3,2,2)+1 = 3$
- $dp[3][1]$: CAT vs D $\rightarrow \min(3,3,2)+1 = 3$
- $dp[3][2]$: CAT vs DO $\rightarrow \min(3,2,2)+1 = 3$
- $dp[3][3]$: CAT vs DOG $\rightarrow \min(3,3,3)+1 = 3$

Answer: 3 (replace C $\rightarrow$ D, A $\rightarrow$ O, T $\rightarrow$ G)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int editDistance(const string& a, const string& b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1));

    // Base cases
    for (int i = 0; i <= n; i++) dp[i][0] = i;
    for (int j = 0; j <= m; j++) dp[0][j] = j;

    // Fill DP table
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (a[i - 1] == b[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1]; // match
            } else {
                dp[i][j] = 1 + min({dp[i - 1][j], // delete
                                   dp[i][j - 1], // insert
                                   dp[i - 1][j - 1]}); // replace
            }
        }
    }
    return dp[n][m];
}

// Space-optimized version using two rows
int editDistanceOptimized(const string& a, const string& b) {
    int n = a.size(), m = b.size();
    vector<int> prev(m + 1), curr(m + 1);

    for (int j = 0; j <= m; j++) prev[j] = j;

    for (int i = 1; i <= n; i++) {
        curr[0] = i;
```

```

        for (int j = 1; j <= m; j++) {
            if (a[i - 1] == b[j - 1]) {
                curr[j] = prev[j - 1];
            } else {
                curr[j] = 1 + min({prev[j], curr[j - 1], prev[j - 1]});
            }
        }
        swap(prev, curr);
    }
    return prev[m];
}

int main() {
    string a = "CAT", b = "DOG";
    cout << editDistance(a, b) << endl; // 3
    cout << editDistanceOptimized(a, b) << endl; // 3
    return 0;
}

```

9. Python Implementation

```

def edit_distance(a: str, b: str) -> int:
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]

    for i in range(n + 1):
        dp[i][0] = i
    for j in range(m + 1):
        dp[0][j] = j

    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(
                    dp[i - 1][j],    # delete
                    dp[i][j - 1],    # insert
                    dp[i - 1][j - 1] # replace
                )
    return dp[n][m]

def edit_distance_optimized(a: str, b: str) -> int:
    n, m = len(a), len(b)
    prev = list(range(m + 1))

    for i in range(1, n + 1):
        curr = [0] * (m + 1)
        curr[0] = i
        for j in range(1, m + 1):

```

```

        if a[i - 1] == b[j - 1]:
            curr[j] = prev[j - 1]
        else:
            curr[j] = 1 + min(prev[j], curr[j - 1], prev[j - 1])
        prev = curr
    return prev[m]

if __name__ == "__main__":
    print(edit_distance("CAT", "DOG")) # 3
    print(edit_distance_optimized("CAT", "DOG")) # 3

```

10. Code Explanation

- **DP table initialization:** $dp[0][j] = j$ means to convert empty string to first j chars of B , insert j characters. $dp[i][0] = i$ means to delete i characters from A to get empty string.
- **Match case:** If characters are equal, cost is same as cost for strings without these characters — no operation needed.
- **Mismatch case:** Take minimum of three operations. Delete removes from A (go up), insert adds to B (go left), replace changes both (go diagonal).
- **Space optimization:** Only need previous row (`prev`) and current row (`curr`) since recurrence only looks at $dp[i-1][j]$, $dp[i][j-1]$, $dp[i-1][j-1]$ — i.e., up, left, and diagonal.

11. Complexity Analysis

Aspect	Complexity
Time	$O(n \times m)$
Space (2D)	$O(n \times m)$
Space (optimized)	$O(\min(n, m))$
Best case	$O(n)$ when strings are identical
Worst case	$O(n \times m)$ when strings are completely different

12. Common Patterns

Pattern 1: Standard Edit Distance

- **How to identify:** "Minimum operations to convert string A to B with insert/delete/replace"
- **Approach:** Standard 2D DP as above
- **Example:** LeetCode 72 — Edit Distance

Pattern 2: Custom Operation Costs

- **How to identify:** "Each operation has a different cost"
- **Approach:** Modify recurrence to use given costs instead of 1
- **Example:** GFG — Edit Distance with custom costs

Pattern 3: One Edit Distance

- **How to identify:** "Can you convert in exactly one operation?"
- **Approach:** $O(n)$ two-pointer scan, check if strings differ by at most 1 character
- **Example:** LeetCode 161 — One Edit Distance

Pattern 4: Longest Common Subsequence (LCS) variation

- **How to identify:** Edit distance with only insert/delete (no replace)
- **Approach:** $n + m - 2 * \text{LCS}(a, b)$ — cost is $n + m - 2 * \text{LCS}$
- **Example:** LeetCode 583 — Delete Operation for Two Strings

13. Common Mistakes

- Forgetting to initialize first row and column
- Confusing $\text{dp}[i-1][j]$ (delete from A) with $\text{dp}[i][j-1]$ (insert into A)
- Using 0-indexing vs 1-indexing incorrectly when accessing characters
- Off-by-one in DP table dimensions
- Not handling empty strings as input
- Using `min` with wrong arguments (forgetting $\text{min}(\{a, b, c\})$ with braces in C++)

14. Edge Cases

- Both strings empty $\rightarrow 0$
- One string empty $\rightarrow$ length of the other string
- Strings already equal $\rightarrow 0$
- Single character strings
- Strings with all different characters
- Very long strings (performance matters, use space optimization)
- Unicode characters (size may vary)

15. Variations

Damerau–Levenshtein Distance

Adds **transposition** (swap adjacent characters) as an operation. Used in spell-checkers. More complex recurrence.

Longest Common Subsequence

Same DP structure but different recurrence: $\text{dp}[i][j] = \max(\text{dp}[i-1][j], \text{dp}[i][j-1])$ when mismatch, $+1$ when match. No replace operation.

Needleman-Wunsch (Global Alignment)

Used in bioinformatics. Adds scoring for matches, mismatches, and gaps. Same DP structure.

16. Related Algorithms/Data Structures

Algorithm	Connection
LCS (Longest Common Subsequence)	Same DP table, different recurrence
LCS (Longest Common Substring)	Uses $dp[i][j] = dp[i-1][j-1] + 1$ only on match, reset on mismatch
Regular Expression Matching	More complex string matching with patterns
Sequence Alignment (Bioinformatics)	Generalized edit distance with scoring matrices

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Edit Distance	LeetCode 72	Standard	Easy
One Edit Distance	LeetCode 161	Check if one edit away	Easy

Medium

Problem	Platform	Pattern	Difficulty
Delete Operation for Two Strings	LeetCode 583	LCS-based	Medium
Minimum ASCII Delete Sum for Two Strings	LeetCode 712	Weighted edit distance	Medium
Shortest Common Supersequence	LeetCode 1092	Print LCS + edit distance	Medium

Hard

Problem	Platform	Pattern	Difficulty
Edit Distance with Custom Costs	GFG	Weighted operations	Hard
Regular Expression Matching	LeetCode 10	Pattern matching DP	Hard
Wildcard Matching	LeetCode 44	Pattern matching	Hard

18. Interview Explanation

"Edit Distance finds the minimum number of operations — insert, delete, replace — to convert one string to another. I build a 2D DP table where $dp[i][j]$ represents the cost to convert the first i characters of string A to the first j characters of string B. If characters match, I take the diagonal value. If they don't, I take 1 plus the minimum of three choices: delete from A (go up), insert into A (go left), or replace (go diagonal). The base cases handle empty strings. Time is $O(n \times m)$ and space can be optimized to $O(\min(n, m))$ using two rows."

19. Revision Notes

- **Key idea:** $dp[i][j]$ = edit distance between $A[0..i-1]$ and $B[0..j-1]$
- **Recurrence (mismatch):** $dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$
- **Recurrence (match):** $dp[i][j] = dp[i-1][j-1]$
- **Base:** $dp[i][0] = i, dp[0][j] = j$
- **Complexity:** $O(n \times m)$ time, $O(\min(n, m))$ space with optimization
- **Common trap:** Confusing delete (up) and insert (left)
- **LCS connection:** When only insert/delete allowed, $cost = n + m - 2 \times LCS$

20. Final Cheat Sheet

Edit Distance (Levenshtein Distance)

When: Convert $A \rightarrow B$ with ins/del/replace

DP: $dp[i][j]$ = cost for $A[0..i-1]$, $B[0..j-1]$

Base: $dp[i][0] = i, dp[0][j] = j$

Recurrence:

match: $dp[i][j] = dp[i-1][j-1]$

mismatch: $dp[i][j] = 1 + \min(\text{up}, \text{left}, \text{diag})$

Time: $O(n \times m)$ Space: $O(\min(n, m))$

Edge: empty strings, identical strings

2. Palindrome DP

1. Overview

Palindrome DP refers to a family of DP problems involving palindromic substrings or subsequences in a string. The core idea is to check whether substrings are palindromes and build solutions for larger substrings from smaller ones. Common problems include "Longest Palindromic Substring", "Palindromic Substrings count", and "Minimum cuts for palindrome partitioning".

2. Intuition

A palindrome reads the same forwards and backwards. For example, "racecar" or "aba".

Key insight: If you know that $s[i+1..j-1]$ is a palindrome AND $s[i] == s[j]$, then $s[i..j]$ is also a palindrome. This is a textbook DP optimal substructure — a larger palindrome is built from a smaller palindrome.

Think of it like expanding from the center. Every palindrome has a center (either a single character for odd-length palindromes or two characters for even-length ones). The DP approach builds a table of all substrings, starting from length 1 and 2, then expanding outward.

Analogy: Imagine a ripple effect — drop a stone (palindrome center) and the ripples (palindromic expansions) grow outward symmetrically.

3. When to Use It

- Counting palindromic substrings in a string
- Finding the longest palindromic substring
- Minimum cuts to partition a string into palindromes
- Longest palindromic subsequence (LPS)
- Checking if a string can be rearranged into a palindrome
- Problems involving "palindromic partitioning" of strings

Trigger phrases: "palindromic substring", "palindromic subsequence", "palindrome partitioning", "minimum cuts", "longest palindrome"

4. When Not to Use It

- For checking if the whole string is a palindrome — use $O(n)$ two-pointer
- For finding longest palindromic subsequence, LCS-based approach is simpler
- When string length is very large (10^5+) — use Manacher's algorithm for $O(n)$ substring palindrome queries
- For counting palindromic subsequences (not substrings) — different DP approach

5. Core Concepts

5.1 Palindrome Table

A 2D boolean DP table where $dp[i][j] = \text{true}$ if $s[i..j]$ is a palindrome.

5.2 Substring vs Subsequence

- **Substring:** Contiguous characters. Used in standard palindrome DP.
- **Subsequence:** Characters can be non-contiguous. Different DP (LPS).

5.3 Centers (Odd/Even)

- **Odd-length palindrome:** Center is a single character (e.g., "aba" center is 'b')
- **Even-length palindrome:** Center is between two characters (e.g., "abba" center between 'b' and 'b')

5.4 Recurrence

```
dp[i][j] = (s[i] == s[j]) AND dp[i+1][j-1]
```

Base cases:

- $dp[i][i] = \text{true}$ (single character, length 1)
- $dp[i][i+1] = (s[i] == s[i+1])$ (two characters, length 2)

6. Step-by-Step Algorithm

Longest Palindromic Substring

1. Let $n = \text{len}(s)$
2. Create boolean DP table $dp[n][n]$ initialized to false

3. Set `dp[i][i] = true` for all `i` (single char palindromes)
4. Check two-character substrings: `dp[i][i+1] = (s[i] == s[i+1])`
5. For length from 3 to `n`:
 - For `i` from 0 to `n-length`:
 - `j = i + length - 1`
 - If `s[i] == s[j]` and `dp[i+1][j-1]`: `dp[i][j] = true`
6. Track the longest substring where `dp[i][j]` is true
7. Return that substring

Count Palindromic Substrings

Same DP table, just count all `true` entries.

7. Dry Run

Find longest palindromic substring in "babad".

(i,j)	0(b)	1(a)	2(b)	3(a)	4(d)
0(b)	T	F	T	F	F
1(a)	-	T	F	T	F
2(b)	-	-	T	F	F
3(a)	-	-	-	T	F
4(d)	-	-	-	-	T

Step-by-step:

- **Length 1:** All true (diagonal)
- **Length 2:** "ba"=F, "ab"=F, "ba"=F, "ad"=F
- **Length 3:**
 - `dp[0][2]`: `s[0]==s2` and `dp[1][1]=T` → T ✓ ("bab")
 - `dp[1][3]`: `s[1]==s3` and `dp[2][2]=T` → T ✓ ("aba")
 - `dp[2][4]`: `s[2]==s4` → F
- **Length 4:**
 - `dp[0][3]`: `s[0]!='d'` → F
 - `dp[1][4]`: `s[1]!='d'` → F
- **Length 5:** `dp[0][4]`: `s[0]!='d'` → F

Answer: "bab" or "aba" (both length 3)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Longest Palindromic Substring
string longestPalindrome(const string& s) {
```

```

int n = s.size();
if (n == 0) return "";

vector<vector<bool>> dp(n, vector<bool>(n, false));
int start = 0, maxLen = 1;

// Length 1
for (int i = 0; i < n; i++) dp[i][i] = true;

// Length 2
for (int i = 0; i < n - 1; i++) {
    if (s[i] == s[i + 1]) {
        dp[i][i + 1] = true;
        start = i;
        maxLen = 2;
    }
}

// Length >= 3
for (int len = 3; len <= n; len++) {
    for (int i = 0; i + len - 1 < n; i++) {
        int j = i + len - 1;
        if (s[i] == s[j] && dp[i + 1][j - 1]) {
            dp[i][j] = true;
            if (len > maxLen) {
                start = i;
                maxLen = len;
            }
        }
    }
}
return s.substr(start, maxLen);
}

// Count Palindromic Substrings
int countPalindromicSubstrings(const string& s) {
    int n = s.size(), count = 0;
    vector<vector<bool>> dp(n, vector<bool>(n, false));

    for (int len = 1; len <= n; len++) {
        for (int i = 0; i + len - 1 < n; i++) {
            int j = i + len - 1;
            if (len == 1) {
                dp[i][j] = true;
            } else if (len == 2) {
                dp[i][j] = (s[i] == s[j]);
            } else {
                dp[i][j] = (s[i] == s[j] && dp[i + 1][j - 1]);
            }
            if (dp[i][j]) count++;
        }
    }
    return count;
}

```

```
// Minimum cuts for palindrome partitioning
int minCut(const string& s) {
    int n = s.size();
    vector<vector<bool>> isPal(n, vector<bool>(n, false));
    vector<int> dp(n, INT_MAX);

    // Build palindrome table
    for (int len = 1; len <= n; len++) {
        for (int i = 0; i + len - 1 < n; i++) {
            int j = i + len - 1;
            if (len == 1) isPal[i][j] = true;
            else if (len == 2) isPal[i][j] = (s[i] == s[j]);
            else isPal[i][j] = (s[i] == s[j] && isPal[i + 1][j - 1]);
        }
    }

    // DP for min cuts
    for (int i = 0; i < n; i++) {
        if (isPal[0][i]) {
            dp[i] = 0;
        } else {
            for (int j = 0; j < i; j++) {
                if (isPal[j + 1][i]) {
                    dp[i] = min(dp[i], dp[j] + 1);
                }
            }
        }
    }
    return dp[n - 1];
}

int main() {
    string s = "babad";
    cout << longestPalindrome(s) << endl; // "bab" or "aba"
    cout << countPalindromicSubstrings(s) << endl; // 5

    s = "aab";
    cout << minCut(s) << endl; // 1 ("aa" | "b")
    return 0;
}
```

9. Python Implementation

```
def longest_palindrome(s: str) -> str:
    n = len(s)
    if n == 0:
        return ""

    dp = [[False] * n for _ in range(n)]
    start, max_len = 0, 1
```

```

    for i in range(n):
        dp[i][i] = True

    for i in range(n - 1):
        if s[i] == s[i + 1]:
            dp[i][i + 1] = True
            start, max_len = i, 2

    for length in range(3, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            if s[i] == s[j] and dp[i + 1][j - 1]:
                dp[i][j] = True
                if length > max_len:
                    start, max_len = i, length

    return s[start:start + max_len]

def count_palindromic_substrings(s: str) -> int:
    n = len(s)
    dp = [[False] * n for _ in range(n)]
    count = 0

    for length in range(1, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            if length == 1:
                dp[i][j] = True
            elif length == 2:
                dp[i][j] = (s[i] == s[j])
            else:
                dp[i][j] = (s[i] == s[j] and dp[i + 1][j - 1])
            if dp[i][j]:
                count += 1

    return count

def min_cut(s: str) -> int:
    n = len(s)
    is_pal = [[False] * n for _ in range(n)]

    for length in range(1, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            if length == 1:
                is_pal[i][j] = True
            elif length == 2:
                is_pal[i][j] = (s[i] == s[j])
            else:
                is_pal[i][j] = (s[i] == s[j] and is_pal[i + 1][j - 1])

    dp = [float('inf')] * n

```

```

for i in range(n):
    if is_pal[0][i]:
        dp[i] = 0
    else:
        for j in range(i):
            if is_pal[j + 1][i]:
                dp[i] = min(dp[i], dp[j] + 1)
return dp[n - 1]

if __name__ == "__main__":
    print(longest_palindrome("babad")) # bab or aba
    print(count_palindromic_substrings("babad")) # 5
    print(min_cut("aab")) # 1

```

10. Code Explanation

- **DP table (`dp[i][j]`):** Stores whether substring `s[i..j]` is a palindrome
- **Length loop:** Process substrings by increasing length — ensures smaller substrings are computed before larger ones
- **Base cases:** Length 1 (always palindrome) and length 2 (check if characters equal)
- **Recurrence:** `s[i] == s[j]` AND inner substring `s[i+1..j-1]` is a palindrome
- **Min cut DP:** `dp[i] = min cuts for prefix s[0..i]`. If `s[0..i]` is palindrome, 0 cuts. Otherwise, try all partition points `j` where `s[j+1..i]` is palindrome.

11. Complexity Analysis

Aspect	Complexity
Time (palindrome table)	$O(n^2)$
Space (DP table)	$O(n^2)$
Time (min cut)	$O(n^2)$
Space (min cut)	$O(n^2)$
Alternative (Manacher)	$O(n)$ time, $O(n)$ space

12. Common Patterns

Pattern 1: Longest Palindromic Substring

- **How to identify:** "Find the longest palindrome in a string"
- **Approach:** DP table or expand around centers
- **Example:** LeetCode 5 — Longest Palindromic Substring

Pattern 2: Count Palindromic Substrings

- **How to identify:** "Count how many palindromes exist in a string"
- **Approach:** Same DP table, count true entries

- **Example:** LeetCode 647 — Palindromic Substrings

Pattern 3: Palindrome Partitioning (Min Cuts)

- **How to identify:** "Minimum cuts to make all parts palindrome"
- **Approach:** Build palindrome table, then DP for min cuts
- **Example:** LeetCode 132 — Palindrome Partitioning II

Pattern 4: Longest Palindromic Subsequence

- **How to identify:** "Remove characters to make a palindrome"
- **Approach:** LCS between string and its reverse
- **Example:** LeetCode 516 — Longest Palindromic Subsequence

13. Common Mistakes

- Building DP table in wrong order (must go by increasing length)
- Off-by-one in substring indices
- Forgetting to handle length 2 separately
- Using `dp[i+1][j-1]` before it's computed (solved by length-based iteration)
- Confusing substring and subsequence
- $O(n^3)$ min cut implementation (should be $O(n^2)$ with precomputed palindrome table)

14. Edge Cases

- Empty string $\rightarrow 0$ or ""
- Single character $\rightarrow$ palindrome itself
- All same characters $\rightarrow$ entire string is palindrome
- No palindrome longer than 1 $\rightarrow$ "ab" $\rightarrow$ "a" or "b"
- Even-length palindrome $\rightarrow$ "abba"
- Odd-length palindrome $\rightarrow$ "aba"
- String of length 2 $\rightarrow$ handles separately

15. Variations

Expand Around Center

Instead of $O(n^2)$ DP table, use $O(n^2)$ expansion from each center ($2n-1$ centers). Simpler, less memory.

Manacher's Algorithm

$O(n)$ algorithm for longest palindromic substring. Faster but more complex. Important for CP with large strings.

Palindrome Partitioning (All Partitions)

Return all possible palindrome partitions. Use backtracking + palindrome check.

16. Related Algorithms/Data Structures

Algorithm	Connection
Manacher's Algorithm	$O(n)$ alternative for longest palindrome
LCS (Longest Common Subsequence)	$LPS = LCS(s, reverse(s))$
Backtracking	Generate all palindrome partitions
String Hashing (Rolling Hash)	Can check palindrome in $O(1)$ after preprocessing

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Valid Palindrome	LeetCode 125	Check if whole string is palindrome	Easy
Palindrome Linked List	LeetCode 234	Check palindrome in linked list	Easy

Medium

Problem	Platform	Pattern	Difficulty
Longest Palindromic Substring	LeetCode 5	Classic DP	Medium
Palindromic Substrings	LeetCode 647	Count palindromes	Medium
Longest Palindromic Subsequence	LeetCode 516	LPS using LCS	Medium

Hard

Problem	Platform	Pattern	Difficulty
Palindrome Partitioning II	LeetCode 132	Min cuts DP	Hard
Minimum Insertion Steps to Make Palindrome	LeetCode 1312	LPS variation	Hard
Count Different Palindromic Subsequences	LeetCode 730	Complex DP	Hard

18. Interview Explanation

"For palindrome DP problems, I use a 2D table where $dp[i][j]$ indicates if substring $s[i..j]$ is a palindrome. The key recurrence is: $dp[i][j] = (s[i] == s[j] \ \&\& \ dp[i+1][j-1])$. I fill the table by increasing substring length so that smaller substrings are computed first. Base cases are length 1 (always palindrome) and length 2 (check if both chars equal). For the longest palindrome, I track the maximum length substring. For min cuts, I build the palindrome table first, then use a separate 1D DP. Time is $O(n^2)$ and space can be $O(n^2)$ or $O(1)$ with expand-around-center."

19. Revision Notes

- **Key idea:** $dp[i][j] = (s[i]==s[j]) \ \text{AND} \ dp[i+1][j-1]$
- **Fill order:** by increasing substring length (not by i then j)

- **Base:** $dp[i][i] = \text{true}$, $dp[i][i+1] = (s[i] == s[i+1])$
- **Longest palindrome:** track max length while filling
- **Count palindromes:** count all **true** entries
- **Min cuts:** $cut[i] = \min(cut[j] + 1)$ where $s[j+1..i]$ is palindrome
- **Complexity:** $O(n^2)$ time, $O(n^2)$ or $O(1)$ space
- **Alternative:** Expand around center ($O(n^2)$ time, $O(1)$ space)
- **Manacher:** $O(n)$ time, $O(n)$ space for large inputs

20. Final Cheat Sheet

Palindrome DP

When: Palindromic substring/subsequence problems

DP: $dp[i][j] = \text{is } s[i..j] \text{ a palindrome?}$

Recurrence: $dp[i][j] = (s[i] == s[j]) \ \&\& \ dp[i+1][j-1]$

Base: $\text{len}=1 \rightarrow \text{true}$, $\text{len}=2 \rightarrow s[i] == s[j]$

Fill: by increasing length

Time: $O(n^2)$ Space: $O(n^2)$ or $O(1)$

Edge: empty string, single char, all same

3. Matrix Chain Multiplication

1. Overview

Matrix Chain Multiplication (MCM) is a classic DP problem that finds the most efficient way to multiply a sequence of matrices. The goal is to minimize the total number of scalar multiplications by choosing the optimal parenthesization (order of multiplication). Matrix multiplication is associative, so the order doesn't change the result, but it drastically affects the cost.

2. Intuition

When multiplying matrices A ($p \times q$) and B ($q \times r$), the result is a $p \times r$ matrix, and the cost is $p \times q \times r$ scalar multiplications.

Now consider multiplying three matrices: $A(10 \times 30)$, $B(30 \times 5)$, $C(5 \times 60)$.

- **$(A \times B) \times C$:** Cost = $(10 \times 30 \times 5) + (10 \times 5 \times 60) = 1500 + 3000 = 4500$
- **$A \times (B \times C)$:** Cost = $(30 \times 5 \times 60) + (10 \times 30 \times 60) = 9000 + 18000 = 27000$

Same matrices, but one order is $6 \times$ faster! The difference grows exponentially with more matrices.

Analogy: Think of it like a delivery route. You have to visit multiple stops. The order you visit them affects the total distance. MCM finds the shortest route (parenthesization) for multiplying matrices.

Core insight: The problem has optimal substructure. The optimal way to multiply matrices $i..j$ involves splitting at some k between i and $j-1$, multiplying $i..k$ and $(k+1)..j$ optimally, then multiplying the two results.

3. When to Use It

- Minimizing cost of matrix chain multiplication
- Problems that ask "minimum cost to perform operations" where the operation cost depends on the sides (like merging stones, polygon triangulation, etc.)
- Problems where you combine adjacent elements with a cost function
- Burst balloons problem (similar structure)
- Optimal binary search tree (OBST) — same DP pattern

Trigger phrases: "parenthesize", "minimize cost of multiplication", "optimal order", "minimum cost to merge", "burst balloons"

4. When Not to Use It

- When the matrices are all the same size (all $M \times N$) — order doesn't matter, any order gives same cost
- For multiplying just 2 matrices — only one way, no decision needed
- When the chain is short ($n \leq 3$) — can manually check
- When approximate solution is acceptable — use greedy algorithms

5. Core Concepts

5.1 Dimensions Array

If matrices are $A_1(p_0 \times p_1)$, $A_2(p_1 \times p_2)$, ..., $A_n(p_{n-1} \times p_n)$, store dimensions in array `arr[]` of size $n+1$: `arr[i]` = rows of A_i , `arr[i+1]` = cols of A_i .

5.2 DP Table

`dp[i][j]` = minimum cost to multiply matrices A_i through A_j (1-indexed).

5.3 Split Point

For each k between i and $j-1$, we split:

- Multiply $A_i..A_k \rightarrow \text{cost} = \text{dp}[i][k]$
- Multiply $A_{k+1}..A_j \rightarrow \text{cost} = \text{dp}[k+1][j]$
- Multiply the two results $\rightarrow \text{cost} = \text{arr}[i-1] \times \text{arr}[k] \times \text{arr}[j]$

5.4 Recurrence

```
dp[i][j] = min over k in [i, j-1] of (dp[i][k] + dp[k+1][j] + arr[i-1] * arr[k] * arr[j])
```

6. Step-by-Step Algorithm

1. Let n = number of matrices
2. Let `arr[]` of size $n+1$ store dimensions
3. Create DP table `dp[n+1][n+1]` initialized to 0

4. For length from 2 to n (number of matrices in chain):
 - For i from 1 to n-length+1:
 - $j = i + \text{length} - 1$
 - Initialize $\text{dp}[i][j] = \text{INT_MAX}$
 - For k from i to j-1:
 - $\text{cost} = \text{dp}[i][k] + \text{dp}[k+1][j] + \text{arr}[i-1] * \text{arr}[k] * \text{arr}[j]$
 - Update $\text{dp}[i][j] = \min(\text{dp}[i][j], \text{cost})$
5. Answer = $\text{dp}[1][n]$

7. Dry Run

Matrices: $A_1(10 \times 30)$, $A_2(30 \times 5)$, $A_3(5 \times 60)$ arr = [10, 30, 5, 60]

(i,j)	1	2	3
1	0	1500	4500
2	-	0	9000
3	-	-	0

Step-by-step:

Length 2 (k=1):

- $\text{dp}[1][2]$: $\text{arr}[0] \times \text{arr}[1] \times \text{arr}[2] = 10 \times 30 \times 5 = 1500$
- $\text{dp}[2][3]$: $\text{arr}[1] \times \text{arr}[2] \times \text{arr}[3] = 30 \times 5 \times 60 = 9000$

Length 3 (k=1,2):

- $\text{dp}[1][3]$:
 - k=1: $\text{dp}[1][1] + \text{dp}[2][3] + 10 \times 30 \times 60 = 0 + 9000 + 18000 = 27000$
 - k=2: $\text{dp}[1][2] + \text{dp}[3][3] + 10 \times 5 \times 60 = 1500 + 0 + 3000 = 4500$
 - Min = 4500

Answer: 4500 (parenthesize as $(A_1 \times A_2) \times A_3$)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Returns minimum cost and also stores the split points for reconstruction
int matrixChainOrder(const vector<int>& arr, vector<vector<int>>& split) {
    int n = arr.size() - 1; // number of matrices
    vector<vector<int>> dp(n + 1, vector<int>(n + 1, 0));
    split.assign(n + 1, vector<int>(n + 1, 0));

    // len = chain length
    for (int len = 2; len <= n; len++) {
        for (int i = 1; i + len - 1 <= n; i++) {
```

```

        int j = i + len - 1;
        dp[i][j] = INT_MAX;
        for (int k = i; k < j; k++) {
            int cost = dp[i][k] + dp[k + 1][j] + arr[i - 1] * arr[k] * arr[j];
            if (cost < dp[i][j]) {
                dp[i][j] = cost;
                split[i][j] = k;
            }
        }
    }
}

return dp[1][n];
}

// Reconstruct the parenthesization
string buildParenthesis(int i, int j, const vector<vector<int>>& split) {
    if (i == j) return "A" + to_string(i);
    int k = split[i][j];
    return "(" + buildParenthesis(i, k, split) + "x" + buildParenthesis(k + 1, j,
split) + ")";
}

int main() {
    vector<int> arr = {10, 30, 5, 60}; // A1(10x30), A2(30x5), A3(5x60)
    vector<vector<int>> split;
    int minCost = matrixChainOrder(arr, split);
    cout << "Minimum cost: " << minCost << endl; // 4500
    cout << "Order: " << buildParenthesis(1, arr.size() - 1, split) << endl;
    // (A1xA2)xA3
    return 0;
}

```

9. Python Implementation

```

def matrix_chain_order(arr):
    n = len(arr) - 1 # number of matrices
    dp = [[0] * (n + 1) for _ in range(n + 1)]
    split = [[0] * (n + 1) for _ in range(n + 1)]

    for length in range(2, n + 1):
        for i in range(1, n - length + 2):
            j = i + length - 1
            dp[i][j] = float('inf')
            for k in range(i, j):
                cost = dp[i][k] + dp[k + 1][j] + arr[i - 1] * arr[k] * arr[j]
                if cost < dp[i][j]:
                    dp[i][j] = cost
                    split[i][j] = k

    return dp[1][n], split

```

```
def build_parenthesis(i, j, split):
    if i == j:
        return f"A{i}"
    k = split[i][j]
    return f"({build_parenthesis(i, k, split)}x{build_parenthesis(k + 1, j,
split)})"

if __name__ == "__main__":
    arr = [10, 30, 5, 60]
    min_cost, split = matrix_chain_order(arr)
    print(f"Minimum cost: {min_cost}") # 4500
    print(f"Order: {build_parenthesis(1, len(arr) - 1, split)}") # (A1x A2)x A3
```

10. Code Explanation

- **`arr[i-1] * arr[k] * arr[j]`**: Cost of multiplying the two resulting matrices. After multiplying $A_i..A_k$, we get a matrix of size $arr[i-1] \times arr[k]$. After multiplying $A_{k+1}..A_j$, we get $arr[k] \times arr[j]$. The multiplication cost is $arr[i-1] \times arr[k] \times arr[j]$.
- **Length loop**: Processes chains of increasing length so smaller subproblems are computed first.
- **Split array**: Stores the optimal split point k for each (i, j) to reconstruct the parenthesization.
- **`dp[i][k]` and `dp[k+1][j]`**: Minimum costs of the left and right subchains.

11. Complexity Analysis

Aspect	Complexity
Time	$O(n^3)$ — three nested loops
Space	$O(n^2)$ — DP table
Reconstruct	$O(n)$ recursion depth

12. Common Patterns

Pattern 1: Standard MCM

- **How to identify**: "Minimum cost to multiply a chain of matrices"
- **Approach**: $O(n^3)$ DP with split points
- **Example**: GFG — Matrix Chain Multiplication

Pattern 2: Burst Balloons

- **How to identify**: "Burst balloons to maximize coins, $\text{coin} = \text{nums}[i-1] \times \text{nums}[i] \times \text{nums}[i+1]$ "
- **Approach**: Reverse MCM — think of last balloon to burst as the split point
- **Example**: LeetCode 312 — Burst Balloons

Pattern 3: Optimal Binary Search Tree (OBST)

- **How to identify**: "Minimum search cost for a binary search tree with given frequencies"

- **Approach:** Similar recurrence, cost = sum of frequencies in range + left + right
- **Example:** GFG — Optimal Binary Search Tree

Pattern 4: Polygon Triangulation

- **How to identify:** "Minimum cost to triangulate a polygon"
- **Approach:** MCM on vertices, cost = triangle area/weight
- **Example:** LeetCode 1039 — Minimum Score Triangulation of Polygon

Pattern 5: Minimum Cost to Merge Stones

- **How to identify:** "Merge adjacent piles/stones with cost = sum of pile sizes"
- **Approach:** MCM-like with prefix sums
- **Example:** LeetCode 1000 — Minimum Cost to Merge Stones

13. Common Mistakes

- Off-by-one in dimension array (size $n+1$, indices $0..n$)
- Confusing `arr[i-1]*arr[k]*arr[j]` with wrong indices
- Not initializing `dp[i][j] = INT_MAX` before min computation
- Using 0-indexed matrices instead of 1-indexed
- Forgetting that `dp[i][i] = 0` (no cost for single matrix)
- Wrong loop order — must iterate by chain length, not by i
- Integer overflow with large dimensions (use `long long`)

14. Edge Cases

- Single matrix: cost = 0
- Two matrices: only one way
- All dimensions equal: $O(n^3)$ calculates correctly but any order is same
- $n = 0$: no matrices
- Large dimensions may overflow int, use long long
- Square matrices: cost still depends on order

15. Variations

Optimal Binary Search Tree (OBST)

Same recurrence but with probability weights. `dp[i][j] = min(dp[i][k-1] + dp[k+1][j] + sum(weight[i..j]))`

Burst Balloons

Reverse MCM — think of the last balloon burst as the "root". Different dimension handling.

Minimum Cost to Merge Stones

MCM with prefix sums. Can be restricted to merging exactly K stones at a time.

16. Related Algorithms/Data Structures

Algorithm	Connection
Dynamic Programming	MCM is the classic interval DP example
Divide and Conquer	MCM finds optimal split points, similar to D&C
Floyd-Warshall	All-pairs shortest path uses similar $O(n^3)$ structure
Triangulation	Polygon triangulation is MCM on vertices

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Matrix Chain Multiplication	GFG	Standard	Easy
Minimum Cost to Multiply	SPOJ MCM	Standard	Easy

Medium

Problem	Platform	Pattern	Difficulty
Burst Balloons	LeetCode 312	Reverse MCM	Medium
Minimum Score Triangulation	LeetCode 1039	Polygon triangulation	Medium
Optimal Binary Search Tree	GFG	MCM with weights	Medium

Hard

Problem	Platform	Pattern	Difficulty
Minimum Cost to Merge Stones	LeetCode 1000	MCM with K-merges	Hard
Remove Boxes	LeetCode 546	Complex MCM-like DP	Hard
Strange Printer	LeetCode 664	MCM-like interval DP	Hard

18. Interview Explanation

"Matrix Chain Multiplication finds the optimal order to multiply a sequence of matrices. I use interval DP where $dp[i][j]$ is the minimum cost to multiply matrices from i to j . The recurrence tries every split point k between i and $j-1$, computing $dp[i][k] + dp[k+1][j] + arr[i-1] \times arr[k] \times arr[j]$, and takes the minimum. I fill the table by increasing chain length. The time is $O(n^3)$ and space is $O(n^2)$. I can also track split points to reconstruct the optimal parenthesization."

19. Revision Notes

- **Key idea:** $dp[i][j] = \min(dp[i][k] + dp[k+1][j] + arr[i-1] \times arr[k] \times arr[j])$
- **Fill order:** by increasing chain length
- **Base:** $dp[i][i] = 0$

- **Dimensions array:** size $n+1$, `arr[i]` = dimension of matrix A_i
- **Complexity:** $O(n^3)$ time, $O(n^2)$ space
- **Common trap:** off-by-one in dimension indices
- **Variations:** Burst Balloons, OBST, Triangulation, Merge Stones

20. Final Cheat Sheet

Matrix Chain Multiplication

When: Minimize cost of chained operations

DP: `dp[i][j] = min cost for matrices i..j`

Recurrence: `dp[i][j] = min(dp[i][k] + dp[k+1][j] + arr[i-1]*arr[k]*arr[j])`

Base: `dp[i][i] = 0`

Fill: by increasing length

Time: $O(n^3)$ Space: $O(n^2)$

Edge: single matrix, two matrices, large dimensions overflow

4. DP on Trees

1. Overview

DP on Trees (Tree DP) is a technique where we run dynamic programming on a tree data structure. The tree's natural hierarchical structure makes it ideal for DP — we solve subproblems for children, combine results for the parent. Typically implemented using DFS (post-order traversal), where each node's DP state depends on its children's states.

2. Intuition

A tree is a recursive structure. A tree is a node connected to subtrees. So to solve a problem at the root, you can:

1. Solve the same problem for each child subtree
2. Combine the children's results to get the root's answer

Analogy: Think of a company with a CEO (root), managers (internal nodes), and employees (leaves). To calculate total salary budget, each manager asks their direct reports, sums up the numbers, adds their own salary, and passes it up. This is bottom-up DP on a tree.

Core insight: The DP on a tree always has two phases:

1. **DFS down:** Explore the tree structure
2. **DP up:** Compute values while returning from recursion (post-order)

3. When to Use It

- Problems on tree structures where answer at a node depends on its children
- Maximum independent set on a tree (no two adjacent nodes selected)

- Tree diameter
- Tree distances / sum of distances
- Maximum path sum in a tree
- Tree coloring problems
- Rooted tree queries (subtree properties)
- Rerooting problems (calculate DP for all nodes as root)

Trigger phrases: "tree", "maximum sum in a tree", "select nodes such that no two adjacent", "distance between all pairs", "subtree", "tree diameter"

4. When Not to Use It

- When the graph is not a tree (has cycles) — use DP on graphs or general graph algorithms
- For simple tree traversals (just finding height, counting nodes) — no DP needed
- When the tree is extremely large and DP state is heavy — may overflow stack
- For problems where the tree structure is not relevant to the computation

5. Core Concepts

5.1 Rooted Tree

DP on trees typically requires rooting the tree at an arbitrary node (usually 1). Parent-child relationships are established.

5.2 Post-order Traversal

Children are processed before the parent. This ensures that when computing DP for a node, all its children's DP values are already computed.

5.3 DP State

The DP state at a node usually represents some property of the subtree rooted at that node. For example:

- $dp[u][0]$ = max value when node u is NOT selected
- $dp[u][1]$ = max value when node u IS selected

5.4 Adjacency List

Tree is stored as an adjacency list. A parent array or visited set is used to avoid going back to the parent during DFS.

6. Step-by-Step Algorithm (Maximum Independent Set — "House Robber III")

1. Root the tree at any node (say 1)
2. Perform DFS from root:
 - For each child, recursively compute DP
 - $dp[u][0]$ = sum of $\max(dp[child][0], dp[child][1])$ over all children (u not selected, children can be either)
 - $dp[u][1]$ = $value[u] + \sum(dp[child][0])$ (u selected, children cannot be selected)

3. Answer = $\max(\text{dp}[\text{root}][0], \text{dp}[\text{root}][1])$

7. Dry Run

Tree: 1-2-3 (path, rooted at 1) Values: [5, 10, 15]

```

1(5)
 |
2(10)
 |
3(15)

```

DFS:

- Node 3 (leaf):
 - $\text{dp}[3][0] = 0$ (not selected)
 - $\text{dp}[3][1] = 15$ (selected)
- Node 2:
 - $\text{dp}[2][0] = \max(\text{dp}[3][0], \text{dp}[3][1]) = \max(0, 15) = 15$
 - $\text{dp}[2][1] = 10 + \text{dp}[3][0] = 10 + 0 = 10$
- Node 1:
 - $\text{dp}[1][0] = \max(\text{dp}[2][0], \text{dp}[2][1]) = \max(15, 10) = 15$
 - $\text{dp}[1][1] = 5 + \text{dp}[2][0] = 5 + 15 = 20$

Answer: $\max(15, 20) = 20$ (select nodes 1 and 3)

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

// Maximum Independent Set (House Robber III style)
// dp[u][0] = max sum when u is NOT selected
// dp[u][1] = max sum when u IS selected
pair<int, int> dfs(int u, int parent, const vector<int>& values,
                  const vector<vector<int>>& adj) {
    int take = values[u];    // select this node
    int skip = 0;           // don't select this node

    for (int v : adj[u]) {
        if (v == parent) continue;
        auto [childSkip, childTake] = dfs(v, u, values, adj);
        // If u is taken, children must be skipped
        take += childSkip;
        // If u is skipped, children can be either
        skip += max(childSkip, childTake);
    }
    return {skip, take};
}

```

```

int maxIndependentSet(const vector<int>& values, const vector<vector<int>>& adj) {
    auto [skip, take] = dfs(0, -1, values, adj);
    return max(skip, take);
}

// Tree Diameter (returns height and diameter)
pair<int, int> dfsDiameter(int u, int parent, const vector<vector<int>>& adj) {
    int max1 = 0, max2 = 0; // top two heights from children
    int diameter = 0;

    for (int v : adj[u]) {
        if (v == parent) continue;
        auto [h, d] = dfsDiameter(v, u, adj);
        diameter = max(diameter, d);

        int height = h + 1;
        if (height > max1) {
            max2 = max1;
            max1 = height;
        } else if (height > max2) {
            max2 = height;
        }
    }
    // Diameter through this node = max1 + max2
    diameter = max(diameter, max1 + max2);
    return {max1, diameter};
}

int treeDiameter(const vector<vector<int>>& adj) {
    return dfsDiameter(0, -1, adj).second;
}

int main() {
    // Tree: 0-1-2 (path), values = [5, 10, 15]
    vector<int> values = {5, 10, 15};
    vector<vector<int>> adj = {{1}, {0, 2}, {1}};

    cout << "Max Independent Set: " << maxIndependentSet(values, adj) << endl; //
20
    cout << "Tree Diameter: " << treeDiameter(adj) << endl; // 2 (edges: 0-1-2)
    return 0;
}

```

9. Python Implementation

```

def max_independent_set(values, adj):
    def dfs(u, parent):
        take = values[u]
        skip = 0
        for v in adj[u]:

```

```

        if v == parent:
            continue
        child_skip, child_take = dfs(v, u)
        take += child_skip
        skip += max(child_skip, child_take)
    return skip, take

skip, take = dfs(0, -1)
return max(skip, take)

def tree_diameter(adj):
    def dfs(u, parent):
        max1 = max2 = 0
        diameter = 0
        for v in adj[u]:
            if v == parent:
                continue
            h, d = dfs(v, u)
            diameter = max(diameter, d)
            height = h + 1
            if height > max1:
                max2 = max1
                max1 = height
            elif height > max2:
                max2 = height
            diameter = max(diameter, max1 + max2)
        return max1, diameter

    return dfs(0, -1)[1]

if __name__ == "__main__":
    values = [5, 10, 15]
    adj = [[1], [0, 2], [1]]
    print(f"Max Independent Set: {max_independent_set(values, adj)}") # 20
    print(f"Tree Diameter: {tree_diameter(adj)}") # 2

```

10. Code Explanation

- **DFS structure:** The function returns a pair (or tuple) representing DP values for the subtree.
- **take:** When node is selected, add its value, and children must be skipped.
- **skip:** When node is not selected, children can be either selected or skipped — take the max.
- **Tree diameter:** For each node, track the top two longest child heights. The diameter through this node is the sum of these two heights. The overall diameter is the maximum across all nodes.
- **Parent parameter:** Prevents going back to the parent in an undirected tree.

11. Complexity Analysis

Aspect	Complexity
Time	$O(N)$ — each node visited once
Space (DFS stack)	$O(H)$ where H = height of tree
Space (adjacency)	$O(N)$
Worst-case stack	$O(N)$ for skewed tree

12. Common Patterns

Pattern 1: Maximum Independent Set (House Robber III)

- **How to identify:** "Select nodes, no two adjacent, maximize sum"
- **Approach:** `dp[u][0] = sum(max(child)), dp[u][1] = val[u] + sum(dp[child][0])`
- **Example:** LeetCode 337 — House Robber III

Pattern 2: Tree Diameter

- **How to identify:** "Longest path between any two nodes"
- **Approach:** Track top two heights from children, take max of sum
- **Example:** LeetCode 543 — Diameter of Binary Tree

Pattern 3: Sum of Distances

- **How to identify:** "Sum of distances from each node to all others"
- **Approach:** Two-pass DFS: first compute subtree sizes and distances, then reroot
- **Example:** LeetCode 834 — Sum of Distances in Tree

Pattern 4: Tree Coloring

- **How to identify:** "Color nodes with constraints, count ways"
- **Approach:** DP with color states, multiply children's DP values
- **Example:** Codeforces — Tree Coloring

13. Common Mistakes

- Forgetting to pass parent parameter, causing infinite loops
- Using recursion on skewed trees causing stack overflow
- Not handling the case where a node has no children (leaf)
- Confusing the two states (selected vs not selected)
- Forgetting to add the node's own value when selected
- Trying to use iterative DP instead of recursion (possible but harder)

14. Edge Cases

- Single node (leaf) — both DP states are simple
- Skewed tree (like a linked list) — deep recursion
- All nodes have same value
- Negative values (can skip nodes to avoid negative)

- Tree with only 2 nodes
- Complete binary tree

15. Variations

Rerooting DP

Compute DP for all nodes as root. Use two DFS passes: first compute subtree values, second propagate parent values down.

Centroid Decomposition

Decompose tree into centroids, useful for path queries.

Heavy-Light Decomposition

Decompose tree into heavy paths, useful for path queries with updates.

16. Related Algorithms/Data Structures

Algorithm	Connection
Tree Traversal (DFS)	Foundation of tree DP
Tree DP (Rerooting)	Extension for all-roots queries
Centroid Decomposition	Advanced tree DP for path queries
Binary Tree DP	Special case of tree DP
Segment Tree on Trees	HLD + segment tree for path queries

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
House Robber III	LeetCode 337	Max independent set	Easy
Diameter of Binary Tree	LeetCode 543	Tree diameter	Easy

Medium

Problem	Platform	Pattern	Difficulty
Sum of Distances in Tree	LeetCode 834	Rerooting DP	Medium
Binary Tree Cameras	LeetCode 968	Tree DP with states	Medium
Maximum Path Sum	LeetCode 124	Path sum in tree	Medium

Hard

Problem	Platform	Pattern	Difficulty
Tree Distances II	CSES	Rerooting DP	Hard
Minimum Time to Collect All Apples	LeetCode 1443	Tree DP	Hard
Count Subtrees With Max Distance	LeetCode 1617	Tree DP + diameter	Hard

18. Interview Explanation

"DP on Trees uses the recursive nature of trees. I root the tree, then perform a post-order DFS. For each node, I compute DP values based on its children's DP values. Common patterns include maximum independent set (where $dp[node][0/1]$ represents whether the node is selected) and tree diameter (where I track the two longest branch heights through each node). The time is $O(N)$ since each node is visited once. The key is to define the correct DP state and recurrence that combines children's results."

19. Revision Notes

- **Key idea:** Post-order DFS, children's DP $\rightarrow$ parent's DP
- **State definition:** $dp[u][0/1]$ or any property of subtree rooted at u
- **Two-pass pattern:** For rerooting, first pass computes subtree DP, second pass propagates parent info
- **Complexity:** $O(N)$ time, $O(H)$ stack space
- **Common trap:** Forgetting parent check, stack overflow on skewed trees
- **Diameter trick:** Top two heights + sum

20. Final Cheat Sheet

DP on Trees

When: Problems on trees where answer depends on subtrees
 DP: Post-order DFS, compute child DP first
 State: Depends on problem (take/skip, height, distance, etc.)
 Time: $O(N)$ Space: $O(H)$ recursion stack
 Edge: single node, skewed tree, negative values
 Common: House Robber III, Tree Diameter, Sum of Distances

5. DP on Graphs / DAG

1. Overview

DP on Graphs (especially DAGs — Directed Acyclic Graphs) is a technique where we solve dynamic programming problems on graph structures. The key requirement is that the graph must be acyclic (or we need to handle cycles carefully). DAGs are particularly well-suited for DP because we can topologically order the vertices, ensuring that when we compute DP for a node, all its predecessors have already been processed.

2. Intuition

Think of a DAG as a dependency graph. If you have tasks with prerequisites, you can't do a task until all its prerequisites are done. This is exactly what topological sorting gives you — an order where every edge goes from earlier to later.

Analogy: Imagine a project with tasks $A \rightarrow B \rightarrow C$ (A must be done before B, B before C). To find the shortest time to complete the project, you process tasks in order: A, then B, then C. Each task's DP depends on the previous task's DP.

Core insight: On a DAG, we can process vertices in topological order. For each vertex, we consider all incoming edges from predecessors, compute DP using those predecessors' values, and move forward. This guarantees that when we arrive at a vertex, all its dependencies are already computed.

3. When to Use It

- Finding longest/shortest path in a DAG
- Counting number of paths from source to destination
- Maximum sum path in a DAG
- Critical path method (CPM) / PERT charts
- Problems with dependencies (e.g., course prerequisites)
- Game theory on DAGs (winning/losing states)
- DP on graphs with cycles converted to DAG via SCC condensation

Trigger phrases: "directed acyclic graph", "prerequisites", "dependency", "topological order", "longest path in DAG", "number of ways to reach"

4. When Not to Use It

- When the graph has cycles and you can't condense them — use Bellman-Ford, Floyd-Warshall, or other general graph algorithms
- For shortest path on a general graph with non-negative weights — use Dijkstra
- When the graph is undirected and unweighted — use BFS for shortest path
- For very dense graphs — $O(V^2)$ may be too slow
- When you need all-pairs shortest paths — use Floyd-Warshall

5. Core Concepts

5.1 Topological Ordering

A linear ordering of vertices such that for every edge $u \rightarrow v$, u comes before v . Only exists for DAGs.

5.2 DP State

$dp[v]$ = some property of paths ending at vertex v (e.g., longest distance, number of paths, max sum, etc.)

5.3 Recurrence (Longest Path)

$$dp[v] = \max \text{ over } u \text{ such that } u \rightarrow v \text{ exists of } (dp[u] + \text{weight}(u, v))$$

For shortest path: $dp[v] = \min(\dots)$ instead of $\max(\dots)$.

5.4 SCC Condensation

For graphs with cycles, condense each SCC into a super-node (making a DAG), then run DP on the resulting DAG.

6. Step-by-Step Algorithm (Longest Path in DAG)

1. Build adjacency list of the graph
2. Compute topological order using Kahn's algorithm (BFS) or DFS
3. Initialize $dp[v] = 0$ for all vertices (or $-\infty$ for unweighted)
4. Process vertices in topological order:
 - For each outgoing edge $u \rightarrow v$:
 - $dp[v] = \max(dp[v], dp[u] + \text{weight}(u, v))$
5. Answer = max over all $dp[v]$

7. Dry Run

DAG: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$, $0 \rightarrow 2$, $1 \rightarrow 3$ Edge weights: $0 \rightarrow 1(2)$, $0 \rightarrow 2(3)$, $1 \rightarrow 2(1)$, $1 \rightarrow 3(4)$, $2 \rightarrow 3(2)$

Topological order: 0, 1, 2, 3

Vertex	dp (longest path from source)
0	0
1	2 (from 0)
2	$\max(3 \text{ from } 0, 2+1=3 \text{ from } 1) = 3$
3	$\max(2+4=6 \text{ from } 1, 3+2=5 \text{ from } 2) = 6$

Answer: 6 (path $0 \rightarrow 1 \rightarrow 3$)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Longest path in DAG
vector<int> topologicalSort(int n, const vector<vector<pair<int,int>>>& adj) {
    vector<int> inDegree(n, 0);
    for (int u = 0; u < n; u++) {
        for (auto [v, w] : adj[u]) {
            inDegree[v]++;
        }
    }

    queue<int> q;
    for (int i = 0; i < n; i++) {
        if (inDegree[i] == 0) q.push(i);
    }
}
```

```

    }

    vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (auto [v, w] : adj[u]) {
            if (--inDegree[v] == 0) q.push(v);
        }
    }
    return order;
}

int longestPathDAG(int n, const vector<vector<pair<int,int>>>& adj, int source) {
    vector<int> order = topologicalSort(n, adj);
    vector<int> dp(n, INT_MIN);
    dp[source] = 0;

    for (int u : order) {
        if (dp[u] == INT_MIN) continue; // not reachable
        for (auto [v, w] : adj[u]) {
            dp[v] = max(dp[v], dp[u] + w);
        }
    }

    return *max_element(dp.begin(), dp.end());
}

// Count paths in DAG (mod MOD)
int countPathsDAG(int n, const vector<vector<int>>& adj, int source, int target) {
    vector<int> order = topologicalSort(n, adj);
    const int MOD = 1e9 + 7;
    vector<int> dp(n, 0);
    dp[source] = 1;

    for (int u : order) {
        if (dp[u] == 0) continue;
        for (int v : adj[u]) {
            dp[v] = (dp[v] + dp[u]) % MOD;
        }
    }
    return dp[target];
}

int main() {
    int n = 4;
    vector<vector<pair<int,int>>> adj(n);
    adj[0] = {{1, 2}, {2, 3}};
    adj[1] = {{2, 1}, {3, 4}};
    adj[2] = {{3, 2}};

    cout << "Longest path from 0: " << longestPathDAG(n, adj, 0) << endl; // 6

    vector<vector<int>> adj2(n);

```

```

adj2[0] = {1, 2};
adj2[1] = {2, 3};
adj2[2] = {3};
cout << "Paths from 0 to 3: " << countPathsDAG(n, adj2, 0, 3) << endl; // 3
return 0;
}

```

9. Python Implementation

```

from collections import deque

def topological_sort(n, adj):
    in_degree = [0] * n
    for u in range(n):
        for v, _ in adj[u]:
            in_degree[v] += 1

    q = deque([i for i in range(n) if in_degree[i] == 0])
    order = []

    while q:
        u = q.popleft()
        order.append(u)
        for v, _ in adj[u]:
            in_degree[v] -= 1
            if in_degree[v] == 0:
                q.append(v)
    return order

def longest_path_dag(n, adj, source):
    order = topological_sort(n, adj)
    dp = [float('-inf')] * n
    dp[source] = 0

    for u in order:
        if dp[u] == float('-inf'):
            continue
        for v, w in adj[u]:
            dp[v] = max(dp[v], dp[u] + w)

    return max(dp)

def count_paths_dag(n, adj, source, target):
    order = topological_sort(n, adj)
    MOD = 10**9 + 7
    dp = [0] * n
    dp[source] = 1

```

```

for u in order:
    if dp[u] == 0:
        continue
    for v in adj[u]:
        dp[v] = (dp[v] + dp[u]) % MOD

return dp[target]

if __name__ == "__main__":
    # With weights
    adj = [[] for _ in range(4)]
    adj[0] = [(1, 2), (2, 3)]
    adj[1] = [(2, 1), (3, 4)]
    adj[2] = [(3, 2)]
    print(f"Longest path from 0: {longest_path_dag(4, adj, 0)}") # 6

    # Without weights
    adj2 = [[1, 2], [2, 3], [3], []]
    print(f"Paths from 0 to 3: {count_paths_dag(4, adj2, 0, 3)}") # 3

```

10. Code Explanation

- **Topological sort:** Kahn's algorithm uses in-degree. Vertices with 0 in-degree have no dependencies and are processed first.
- **DP initialization:** `dp[source] = 0` (or 1 for counting). Others are `-inf` (or 0 for counting).
- **Processing order:** For each vertex in topological order, relax its outgoing edges. This is identical to how Bellman-Ford works, but on a DAG we only need one pass.
- **Counting paths:** `dp[v] += dp[u]` means every path ending at `u` can be extended to `v`. The number of paths to `v` is the sum of paths to all its predecessors.

11. Complexity Analysis

Aspect	Complexity
Topological sort	$O(V + E)$
DP computation	$O(V + E)$
Total	$O(V + E)$
Space	$O(V + E)$

12. Common Patterns

Pattern 1: Longest Path in DAG

- **How to identify:** "Longest path in a directed graph with no cycles"
- **Approach:** Topological sort + DP relaxation
- **Example:** GFG — Longest Path in DAG

Pattern 2: Number of Paths / Ways to Reach

- **How to identify:** "Count number of ways to reach destination"
- **Approach:** $dp[v] = \sum(dp[u])$ in topological order
- **Example:** LeetCode 1976 — Number of Ways to Arrive at Destination

Pattern 3: Minimum/Maximum Time to Complete Tasks

- **How to identify:** "Minimum time to complete all tasks with dependencies"
- **Approach:** DP on topological order, track earliest/latest start times
- **Example:** LeetCode 2050 — Parallel Courses III

Pattern 4: Game Theory (Winning/Losing States)

- **How to identify:** "Two players, optimal play on a DAG"
- **Approach:** DP with win/lose states, if any move leads to losing state, current is winning
- **Example:** CSES — Game Routes

13. Common Mistakes

- Running DP on a graph with cycles (will give wrong results or infinite loop)
- Not handling disconnected components / unreachable nodes
- Forgetting to initialize dp with appropriate sentinel values (-inf for max, +inf for min)
- Using int for path counts without modulo (can overflow)
- Confusing topological order direction (incoming vs outgoing edges)
- Not checking if node is reachable before processing

14. Edge Cases

- Single node (no edges)
- Disconnected DAG (multiple components)
- Source = target (distance 0, path count 1)
- Graph with no topological order (has cycles)
- Node with no incoming edges (source)
- Node with no outgoing edges (sink)
- Multiple sources or sinks
- Zero-weight edges

15. Variations

Shortest Path in DAG

Same as longest path but use `min` instead of `max`, initialize with `+inf`.

DP on Condensed Graph (SCC + DAG)

For graphs with cycles, first find SCCs, condense them into a DAG, then run DP.

DP with State on Graph

When you need more than just distance — e.g., DP with mask for Hamiltonian path.

16. Related Algorithms/Data Structures

Algorithm	Connection
Bellman-Ford	Relaxes edges $V-1$ times, DP on DAG does it once
Dijkstra	Shortest path on general graphs, not needed for DAG
Floyd-Warshall	All-pairs shortest paths, overkill for DAG
Topological Sort	Prerequisite for DAG DP
SCC (Kosaraju/Tarjan)	Condense cycles to make DAG

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Longest Path in DAG	GFG	Standard	Easy
Find if Path Exists	LeetCode 1971	Path existence	Easy

Medium

Problem	Platform	Pattern	Difficulty
Parallel Courses III	LeetCode 2050	Min time with dependencies	Medium
Course Schedule II	LeetCode 210	Topological sort + DP	Medium
Number of Ways to Arrive	LeetCode 1976	Count paths	Medium

Hard

Problem	Platform	Pattern	Difficulty
Minimum Difficulty of Job Schedule	LeetCode 1335	DAG-like DP	Hard
Minimum Cost to Make at Least One Valid Path	LeetCode 1368	Grid as DAG DP	Hard
Collect Coins in a Tree	LeetCode 2603	Tree $\rightarrow$ DAG DP	Hard

18. Interview Explanation

"DP on DAGs works by first computing a topological ordering. Since every edge goes from earlier to later in this order, I can process vertices in order and for each vertex, relax its outgoing edges. This guarantees that when I reach a vertex, all its incoming dependencies are already computed. The recurrence is simple: $dp[v] = \max/\min/\text{sum over all predecessors } u \text{ of } (dp[u] + \text{weight})$. Time is $O(V+E)$ which is optimal. If the graph has cycles, I first condense SCCs into a DAG."

19. Revision Notes

- **Key idea:** Topological order → process in order → relax outgoing edges
- **Recurrence:** $dp[v] = \max/\min/\text{sum}(dp[u] + \text{weight}(u,v))$ over all $u \rightarrow v$
- **Longest path:** $dp[v] = \max(dp[u] + w)$, initialize with $-\text{inf}$
- **Shortest path:** $dp[v] = \min(dp[u] + w)$, initialize with $+\text{inf}$
- **Count paths:** $dp[v] = \text{sum}(dp[u])$, initialize source = 1
- **Complexity:** $O(V+E)$ time, $O(V+E)$ space
- **Common trap:** Forgetting to check if node is reachable
- **For cycles:** Condense SCCs first

20. Final Cheat Sheet

DP on Graphs / DAG

When: Path problems on DAGs, dependency chains

Process: Topological sort → DP relaxation

Recurrence: $dp[v] = \text{aggregate}(dp[u] + \text{weight}(u,v))$

Time: $O(V+E)$ Space: $O(V+E)$

Edge: cycles (need SCC condensation), disconnected components

6. Digit DP

1. Overview

Digit DP is a technique for counting numbers in a range $[L, R]$ that satisfy certain conditions based on their digits. The key idea is to process the number digit by digit, maintaining state about the prefix (tightness, leading zeros, and other problem-specific conditions). The DP state is typically $dp[pos][tight][sum]$ or similar, where pos is the current digit position being processed.

2. Intuition

Suppose you want to count numbers from 0 to 123 that have sum of digits = 5. Instead of iterating through all numbers, you process each digit position from left to right.

Analogy: Think of rolling a lock with digits. At each position, you can choose a digit 0-9, but with constraints. The "tight" flag represents whether you've already chosen a smaller digit at a higher position (so you're free to choose 0-9) or you're still matching the upper bound (so you can only choose up to that digit).

Core insight: By processing digits left to right, we maintain:

- **Tight flag:** Whether the prefix so far matches the upper bound exactly
- **Leading zero flag:** Whether we've started placing non-zero digits yet
- The DP memoizes on (position, tight, leadingZero, otherState) to avoid recomputation

3. When to Use It

- Counting numbers in $[L, R]$ with digit sum constraints
- Counting numbers divisible by K
- Counting numbers with no consecutive same digits
- Counting numbers with specific digit patterns
- Palindromic numbers in a range
- Numbers with non-decreasing digits
- Any problem where the answer depends on digits and range is large (10^9 to 10^{18})

Trigger phrases: "count numbers from L to R ", "digit sum", "divisible by", "no consecutive", "numbers with property", "in range"

4. When Not to Use It

- When the range is small ($R \leq 10^6$) — brute force is simpler
- When the condition is very simple (e.g., even numbers) — formula works
- When the problem doesn't involve digit-based constraints
- When the DP state space is too large (e.g., K is 10^9 for divisible DP)

5. Core Concepts

5.1 State Representation

```
dp[pos][tight][leadingZero][extraState]
```

- **pos**: Current digit position (0 = most significant, $n-1$ = least significant)
- **tight**: 0 = can choose any digit 0-9, 1 = must stay $\leq$ bound digit
- **leadingZero**: 0 = have started placing non-zero digits, 1 = still at leading zeros
- **extraState**: Problem-specific state (sum, mod value, previous digit, etc.)

5.2 Tight Flag

- If **tight** == 1: we've matched the bound exactly so far. Current digit can only go up to **bound[pos]**.
- If **tight** == 0: we've already chosen a smaller digit. Current digit can be 0-9.

5.3 Leading Zero

- Crucial for problems where numbers can't start with zero (e.g., counting numbers with digit sum = K)
- Leading zeros shouldn't be counted in the digit sum
- Once we place a non-zero digit, **leadingZero** becomes 0

5.4 Range $[L, R]$ Trick

To count numbers in $[L, R]$, compute **count(R)** - **count($L-1$)**.

6. Step-by-Step Algorithm

1. Convert the upper bound to a string of digits
2. Define recursive function **solve(pos, tight, leadingZero, ...)**:

- If `pos == n`: return 1 if conditions satisfied, else 0
- If memoized, return stored value
- Determine `limit = tight ? bound[pos] : 9`
- For each digit from 0 to limit:
 - `newTight = tight && (digit == limit)`
 - `newLeadingZero = leadingZero && (digit == 0)`
 - Add `solve(pos+1, newTight, newLeadingZero, ...)` to result

3. Return `count(R) - count(L-1)`

7. Dry Run

Count numbers from 0 to 123 with sum of digits = 5.

Bound = "123", n = 3

Let's trace `solve(0, tight=1, leadingZero=1, sum=0)`:

At pos=0 (digit '1'):

- limit = 1 (tight)
- digit=0: newTight=0, newLeadingZero=1, sum=0 → recurse
- digit=1: newTight=1, newLeadingZero=0, sum=1 → recurse

At pos=1 (digit '2'): If tight=1: limit=2

- digit=0: newTight=0, sum=0 or 1...
- ...

(Full trace shows numbers: 5, 14, 23, 32, 41, 50, 104, 113, 122 — total 9 numbers)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Count numbers 0..R with sum of digits = K
class DigitDP {
    string bound;
    int K;
    vector<vector<vector<vector<int>>>> memo;
    // memo[pos][tight][leadingZero][sum]

public:
    int solve(const string& num, int K) {
        this->bound = num;
        this->K = K;
        int n = num.size();
        memo.assign(n, vector<vector<vector<int>>>(2,
            vector<vector<int>>(2, vector<int>(K + 1, -1))));
        return dfs(0, 1, 1, 0);
    }
}
```

```

int countInRange(int L, int R, int K) {
    string rStr = to_string(R);
    string lStr = to_string(L - 1);
    int right = solve(rStr, K);
    if (L == 0) return right;
    int left = solve(lStr, K);
    return right - left;
}

private:
int dfs(int pos, int tight, int leadingZero, int sum) {
    if (pos == bound.size()) {
        return (sum == K) ? 1 : 0;
    }
    if (sum > K) return 0;
    if (memo[pos][tight][leadingZero][sum] != -1) {
        return memo[pos][tight][leadingZero][sum];
    }

    int limit = tight ? (bound[pos] - '0') : 9;
    int ans = 0;

    for (int digit = 0; digit <= limit; digit++) {
        int newTight = tight && (digit == limit);
        int newLeadingZero = leadingZero && (digit == 0);
        int newSum = sum;
        if (!(leadingZero && digit == 0)) {
            newSum += digit; // don't add leading zeros
        }
        ans += dfs(pos + 1, newTight, newLeadingZero, newSum);
    }
    return memo[pos][tight][leadingZero][sum] = ans;
}

};

// Count numbers divisible by K
class DigitDPDivisible {
    string bound;
    int K;
    vector<vector<vector<vector<int>>>> memo;

public:
    int countDivisible(int R, int K) {
        this->bound = to_string(R);
        this->K = K;
        int n = bound.size();
        memo.assign(n, vector<vector<vector<int>>>>(2,
            vector<vector<int>>>(2, vector<int>(K, -1))));
        return dfs(0, 1, 1, 0);
    }

private:
    int dfs(int pos, int tight, int leadingZero, int mod) {

```

```

        if (pos == bound.size()) {
            return (mod == 0) ? 1 : 0;
        }
        if (memo[pos][tight][leadingZero][mod] != -1) {
            return memo[pos][tight][leadingZero][mod];
        }

        int limit = tight ? (bound[pos] - '0') : 9;
        int ans = 0;

        for (int digit = 0; digit <= limit; digit++) {
            int newTight = tight && (digit == limit);
            int newLeadingZero = leadingZero && (digit == 0);
            int newMod = (mod * 10 + digit) % K;
            ans += dfs(pos + 1, newTight, newLeadingZero, newMod);
        }
        return memo[pos][tight][leadingZero][mod] = ans;
    }
};

int main() {
    DigitDP d1;
    cout << "Numbers 0..123 with sum=5: " << d1.solve("123", 5) << endl; // 9
    cout << "Numbers 10..123 with sum=5: " << d1.countInRange(10, 123, 5) << endl;

    DigitDPDivisible d2;
    cout << "Numbers 0..20 divisible by 3: " << d2.countDivisible(20, 3) << endl;
// 7
    return 0;
}

```

9. Python Implementation

```

class DigitDP:
    def count_in_range(self, L: int, R: int, K: int) -> int:
        def solve(num_str: str, K: int) -> int:
            n = len(num_str)
            from functools import lru_cache

            @lru_cache(maxsize=None)
            def dfs(pos: int, tight: bool, leading_zero: bool, sum_val: int) ->
int:
                if pos == n:
                    return 1 if sum_val == K else 0
                if sum_val > K:
                    return 0

                limit = int(num_str[pos]) if tight else 9
                ans = 0

                for digit in range(limit + 1):

```

```

        new_tight = tight and (digit == limit)
        new_leading_zero = leading_zero and (digit == 0)
        new_sum = sum_val
        if not (leading_zero and digit == 0):
            new_sum += digit
        ans += dfs(pos + 1, new_tight, new_leading_zero, new_sum)
    return ans

    return dfs(0, True, True, 0)

right = solve(str(R), K)
left = solve(str(L - 1), K) if L > 0 else 0
return right - left

class DigitDPDivisible:
    def count_divisible(self, R: int, K: int) -> int:
        num_str = str(R)
        n = len(num_str)
        from functools import lru_cache

        @lru_cache(maxsize=None)
        def dfs(pos: int, tight: bool, leading_zero: bool, mod: int) -> int:
            if pos == n:
                return 1 if mod == 0 else 0

            limit = int(num_str[pos]) if tight else 9
            ans = 0

            for digit in range(limit + 1):
                new_tight = tight and (digit == limit)
                new_leading_zero = leading_zero and (digit == 0)
                new_mod = (mod * 10 + digit) % K
                ans += dfs(pos + 1, new_tight, new_leading_zero, new_mod)
            return ans

        return dfs(0, True, True, 0)

if __name__ == "__main__":
    d1 = DigitDP()
    print(f"Numbers 0..123 with sum=5: {d1.count_in_range(0, 123, 5)}") # 9
    print(f"Numbers 10..123 with sum=5: {d1.count_in_range(10, 123, 5)}")

    d2 = DigitDPDivisible()
    print(f"Numbers 0..20 divisible by 3: {d2.count_divisible(20, 3)}") # 7

```

10. Code Explanation

- **dfs(pos, tight, leadingZero, state):** Recursive function that counts valid numbers from position **pos** to the end, given the current tight and leading-zero status.

- **Tight logic:** `limit = tight ? bound[pos] : 9`. If we're tight, we can only go up to the bound digit. If we're not tight, we can use 0-9.
- **New tight:** `newTight = tight && (digit == limit)`. We stay tight only if we were tight before AND we chose the maximum allowed digit.
- **Leading zero:** `newLeadingZero = leadingZero && (digit == 0)`. We stay in leading-zero mode only if we haven't placed any non-zero digit yet.
- **Sum update:** `if (!(leadingZero && digit == 0)) newSum += digit`. Don't count leading zeros in the digit sum.
- **Count(R) - Count(L-1):** Standard trick for range queries.

11. Complexity Analysis

Aspect	Complexity
Number of states	$O(n \times 2 \times 2 \times S)$ where S = extra state space
Transitions per state	$O(10)$ = digits 0-9
Time	$O(n \times 2 \times 2 \times S \times 10)$
Space	$O(n \times 2 \times 2 \times S)$
n (digits)	Up to 18 for 64-bit, up to 1000 for big integers

12. Common Patterns

Pattern 1: Digit Sum

- **How to identify:** "Count numbers with digit sum = K"
- **Approach:** Extra state = current sum. Don't count leading zeros in sum.
- **Example:** GFG — Digit Sum

Pattern 2: Divisible by K

- **How to identify:** "Count numbers divisible by K"
- **Approach:** Extra state = current modulo. `newMod = (mod*10 + digit) % K`.
- **Example:** CSES — Counting Numbers

Pattern 3: Adjacent Digit Constraints

- **How to identify:** "No two consecutive digits are same"
- **Approach:** Extra state = previous digit. Compare before placing.
- **Example:** LeetCode — Numbers with Same Consecutive Differences

Pattern 4: Palindrome Numbers

- **How to identify:** "Count palindromic numbers in range"
- **Approach:** Generate first half of digits, mirror to second half, check range.
- **Example:** LeetCode — 906. Super Palindromes

13. Common Mistakes

- Not handling leading zeros correctly (they affect digit sum and length)
- Forgetting to convert `L-1` to string (what if `L=0`?)
- Not resetting memo for each new query (if `K` changes)
- Memoization without tight flag (tight state is crucial for correctness)
- Integer overflow in mod calculation (use `(mod * 10 + digit) % K`)
- Too many states making memo too large (be careful with extra state dimensions)

14. Edge Cases

- Range `[0, R]` — handle `L=0` case
- `R = 0` — only number 0
- Large upper bound (10^{18}) — 19 digits is fine
- `K = 0` for digit sum (no numbers except 0)
- `K = 1` for divisible (every number is divisible by 1)
- Very large `K` in modulo DP (state space too large)
- Numbers with leading zeros shouldn't be double-counted

15. Variations

Big Integer Digit DP

For numbers with hundreds of digits, use string representation and recursion. State space same but digits more.

Sum of Digit Values (Not Count)

Instead of counting, sum the numbers themselves. `dp[pos][tight][leadingZero]` returns sum of all valid numbers from this state.

Digit DP with Multiple Constraints

Combine multiple conditions: e.g., sum of digits = `K` AND divisible by `M`.

16. Related Algorithms/Data Structures

Algorithm	Connection
Memoization (Recursive DP)	Foundation of digit DP
String Processing	Converting number to string
Inclusion-Exclusion	$\text{Count}(R) - \text{Count}(L-1)$
Combinatorics	Some digit DP problems can be solved with combinatorial formulas

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Digit Sum	GFG	Sum of digits	Easy
Count Numbers with Unique Digits	LeetCode 357	Digit DP with set	Easy

Medium

Problem	Platform	Pattern	Difficulty
Numbers With Repeated Digits	LeetCode 1012	Complement of unique	Medium
Count Special Numbers	GFG	Digit DP with conditions	Medium
Number of Digit One	LeetCode 233	Count occurrences of digit 1	Medium

Hard

Problem	Platform	Pattern	Difficulty
Count of Integers	LeetCode 2719	Digit DP with two bounds	Hard
Numbers at Most N Given Digit Set	LeetCode 902	Restricted digit set	Hard
Count Stepping Numbers	LeetCode	Adjacent difference 1	Hard

18. Interview Explanation

"Digit DP is used to count numbers in a range that satisfy digit-based conditions. I process the number digit by digit from left to right, maintaining a tight flag (whether the prefix matches the upper bound) and a leading-zero flag. The DP state is `dp[pos][tight][leadingZero][extraState]` where `extraState` depends on the problem — like digit sum or modulo value. The recurrence tries each digit 0-9 (or up to the bound digit if tight), computes new flags, and recurses. The answer is `count(R) - count(L-1)`. Time is $O(n \times 2 \times 2 \times S \times 10)$ where n is the number of digits."

19. Revision Notes

- **Key idea:** Process digits left to right with tight/leadingZero flags
- **State:** `dp[pos][tight][leadingZero][extra]`
- **Range trick:** `count(R) - count(L-1)`
- **Tight:** `newTight = tight && (digit == limit)`
- **Leading zero:** `newLeadingZero = leadingZero && (digit == 0)`
- **Complexity:** $O(n \times \text{states} \times 10)$
- **Common trap:** Forgetting to exclude leading zeros from sum
- **Memo reset:** Different K or bound needs new memo

20. Final Cheat Sheet

Digit DP

When: Count numbers in $[L, R]$ with digit constraints

```

State: dp[pos][tight][leadingZero][sum/mod/prev]
Range: count(R) - count(L-1)
Tight: newTight = tight && (digit == limit)
Time:  $O(n \times 2 \times 2 \times S \times 10)$ 
Space:  $O(n \times 2 \times 2 \times S)$ 
Edge: empty range, large K, leading zeros

```

7. Bitmask DP

1. Overview

Bitmask DP (also called DP with bitmask or subset DP) is a technique where we use a bitmask (an integer whose bits represent a set) to represent the state of a DP. It's typically used for problems with small N (≤ 20 -25) where we need to consider subsets of items. The most famous example is the Traveling Salesman Problem (TSP).

2. Intuition

Imagine you have N items and you need to choose a subset of them. There are 2^N possible subsets. A bitmask represents each subset as a binary number: bit $i = 1$ means item i is included, bit $i = 0$ means it's excluded.

Analogy: Think of a light switch panel with N switches. Each switch can be ON (1) or OFF (0). The state of all switches can be represented as an N -bit number. Bitmask DP iterates through all possible states (2^N of them) and transitions between them.

Core insight: For problems involving subsets, permutations, or assignments, bitmask DP allows us to:

- Represent any subset as an integer (0 to 2^N-1)
- Test membership: `mask & (1 << i)`
- Add/remove items: `mask | (1 << i)`, `mask & ~(1 << i)`
- Iterate over subsets in $O(2^N)$ or $O(3^N)$ time

3. When to Use It

- Traveling Salesman Problem (TSP) with small N
- Assignment problems (assign N items to N positions)
- Partition problems (divide set into subsets with constraints)
- Hamiltonian path/cycle problems
- Exact cover problems
- Matching problems with small sets
- State compression when $N \leq 20$ -25

Trigger phrases: "small N ", "subset", "assignment", "permutation", "assignment", "traveling salesman", " $N \leq 20$ ", "cover all"

4. When Not to Use It

- When N is large ($N > 25$) — 2^N is too large
- When the problem doesn't involve subsets/permutations
- When a greedy or simpler DP works
- When the problem can be solved with min-cost max-flow or Hungarian algorithm
- When the graph is dense and N is moderate — still $O(N^2 \times 2^N)$ may be too slow

5. Core Concepts

5.1 Bitmask Operations

- `mask & (1<<i)`: Check if item i is in the set
- `mask | (1<<i)`: Add item i to the set
- `mask & ~(1<<i)`: Remove item i from the set
- `__builtin_popcount(mask)`: Count bits set (GCC)
- `(1<<N) - 1`: Full set mask

5.2 DP State

`dp[mask]` = optimal value for the subset represented by mask. Sometimes `dp[mask][last]` where `last` is the last item added (for TSP).

5.3 Subset Iteration

- Forward: For each mask, try adding a new item
- Reverse: For each mask, try removing an item

5.4 Submask Enumeration

```
for (int sub = mask; sub; sub = (sub - 1) & mask) {
    // iterate over all non-empty subsets of mask
}
```

This runs in $O(3^N)$ total for all masks.

6. Step-by-Step Algorithm (TSP)

1. Let n = number of cities, `dist[i][j]` = distance from i to j
2. Create `dp[mask][last]` with size $(1<<n) \times n$, initialize to INF
3. `dp[1][0] = 0` (starting at city 0, only city 0 visited)
4. For each mask from 1 to $(1<<n)-1$:
 - For each last city where `dp[mask][last]` is computed:
 - For each next city not in mask:
 - `newMask = mask | (1<<next)`
 - `dp[newMask][next] = min(dp[newMask][next], dp[mask][last] + dist[last][next])`
5. Answer = min over last of `dp[(1<<n)-1][last] + dist[last][0]` (return to start)

7. Dry Run

TSP with 3 cities: 0→1: 10, 0→2: 15, 1→2: 20

mask	last	dp
001 (1)	0	0
011 (3)	1	0+10=10
011 (3)	0	INF (not possible to add 0 to {0,1})
101 (5)	2	0+15=15
111 (7)	1	10+20=30 (from 011→1, add 2)
111 (7)	2	15+20=35 (from 101→2, add 1)

Answer = $\min(30 + \text{dist}[1][0], 35 + \text{dist}[2][0]) = \min(30+10, 35+15) = \min(40, 50) = 40$

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Traveling Salesman Problem (TSP)
int tsp(int n, const vector<vector<int>>& dist) {
    int FULL = (1 << n) - 1;
    vector<vector<int>> dp(1 << n, vector<int>(n, INT_MAX / 2));
    dp[1][0] = 0; // start at city 0

    for (int mask = 1; mask < (1 << n); mask++) {
        for (int last = 0; last < n; last++) {
            if (!(mask & (1 << last))) continue; // last not in set
            if (dp[mask][last] == INT_MAX / 2) continue;

            for (int next = 0; next < n; next++) {
                if (mask & (1 << next)) continue; // already visited
                int newMask = mask | (1 << next);
                dp[newMask][next] = min(dp[newMask][next],
                                         dp[mask][last] + dist[last][next]);
            }
        }
    }

    int ans = INT_MAX / 2;
    for (int last = 0; last < n; last++) {
        ans = min(ans, dp[FULL][last] + dist[last][0]);
    }
    return ans;
}

// Assignment Problem: assign N tasks to N workers
int assignment(int n, const vector<vector<int>>& cost) {
    vector<int> dp(1 << n, INT_MAX / 2);
    dp[0] = 0;
```

```

    for (int mask = 0; mask < (1 << n); mask++) {
        int worker = __builtin_popcount(mask); // next worker to assign
        if (worker >= n) continue;

        for (int task = 0; task < n; task++) {
            if (mask & (1 << task)) continue; // task already assigned
            int newMask = mask | (1 << task);
            dp[newMask] = min(dp[newMask], dp[mask] + cost[worker][task]);
        }
    }
    return dp[(1 << n) - 1];
}

int main() {
    // TSP: 3 cities
    vector<vector<int>> dist = {
        {0, 10, 15},
        {10, 0, 20},
        {15, 20, 0}
    };
    cout << "TSP: " << tsp(3, dist) << endl; // 40

    // Assignment: 3 workers, 3 tasks
    vector<vector<int>> cost = {
        {3, 2, 7},
        {5, 1, 3},
        {2, 6, 4}
    };
    cout << "Assignment: " << assignment(3, cost) << endl; // min cost
    return 0;
}

```

9. Python Implementation

```

def tsp(n: int, dist) -> int:
    FULL = (1 << n) - 1
    INF = 10**9
    dp = [[INF] * n for _ in range(1 << n)]
    dp[1][0] = 0 # start at city 0

    for mask in range(1, 1 << n):
        for last in range(n):
            if not (mask & (1 << last)):
                continue
            if dp[mask][last] == INF:
                continue

            for nxt in range(n):
                if mask & (1 << nxt):
                    continue

```

```

        new_mask = mask | (1 << nxt)
        dp[new_mask][nxt] = min(
            dp[new_mask][nxt],
            dp[mask][last] + dist[last][nxt]
        )

    ans = INF
    for last in range(n):
        ans = min(ans, dp[FULL][last] + dist[last][0])
    return ans

def assignment(n: int, cost) -> int:
    INF = 10**9
    dp = [INF] * (1 << n)
    dp[0] = 0

    for mask in range(1 << n):
        worker = bin(mask).count('1') # next worker
        if worker >= n:
            continue

        for task in range(n):
            if mask & (1 << task):
                continue
            new_mask = mask | (1 << task)
            dp[new_mask] = min(dp[new_mask], dp[mask] + cost[worker][task])

    return dp[(1 << n) - 1]

if __name__ == "__main__":
    dist = [
        [0, 10, 15],
        [10, 0, 20],
        [15, 20, 0]
    ]
    print(f"TSP: {tsp(3, dist)}") # 40

    cost = [
        [3, 2, 7],
        [5, 1, 3],
        [2, 6, 4]
    ]
    print(f"Assignment: {assignment(3, cost)}")

```

10. Code Explanation

- **dp[mask][last]**: Minimum cost to visit all cities in **mask**, ending at **last**.
- **Transition**: From state (**mask**, **last**), try adding a new city **next** not in **mask**. New cost = **dp[mask][last] + dist[last][next]**.

- **Assignment problem:** $dp[mask] = \text{min cost to assign tasks in } mask \text{ to first } \text{popcount}(mask) \text{ workers.}$
Worker index = number of tasks assigned so far.
- **Bit operations:** $mask \& (1 \ll i)$ checks if i is in set. $mask \mid (1 \ll i)$ adds i .

11. Complexity Analysis

Aspect	Complexity
Time (TSP)	$O(n^2 \times 2^n)$
Time (Assignment)	$O(n \times 2^n)$
Time (General)	$O(2^n \times \text{poly}(n))$
Space	$O(n \times 2^n)$
N limit	Typically $N \leq 20$ ($2^{20} \approx 10^6$)

12. Common Patterns

Pattern 1: TSP (Traveling Salesman)

- **How to identify:** "Find shortest route visiting all cities exactly once"
- **Approach:** $dp[mask][last]$, minimize over last + return to start
- **Example:** LeetCode 943 — Find the Shortest Superstring

Pattern 2: Assignment Problem

- **How to identify:** "Assign N items to N positions, minimize cost"
- **Approach:** $dp[mask]$, worker index = $\text{popcount}(mask)$
- **Example:** GFG — Assignment Problem

Pattern 3: Partition Equal Subset

- **How to identify:** "Can we partition set into two subsets with equal sum?"
- **Approach:** $dp[mask] = \text{sum of subset}$, check if any subset sum = total/2
- **Example:** LeetCode 416 — Partition Equal Subset Sum

Pattern 4: Hamiltonian Path

- **How to identify:** "Path that visits every vertex exactly once"
- **Approach:** Similar to TSP but no return to start
- **Example:** LeetCode 847 — Shortest Path Visiting All Nodes

Pattern 5: Subset Cover / Exact Cover

- **How to identify:** "Cover all elements with minimum number of subsets"
- **Approach:** $dp[mask] = \text{min count to cover elements in mask}$
- **Example:** LeetCode — Minimum Number of Taps to Open

13. Common Mistakes

- Forgetting to use `1LL << n` for $n > 31$ (integer overflow on 32-bit)
- Using `INT_MAX` without dividing by 2 (adding to it overflows)
- Not checking if `last` is in the mask before accessing `dp[mask][last]`
- Incorrect bit manipulation order of operations
- Not handling the return-to-start in TSP
- Nested loops in wrong order (mask should be outer loop)
- Memory blowup for $n > 25$

14. Edge Cases

- $n = 1$ (single city/item, TSP = 0)
- $n = 0$ (no cities)
- Complete graph vs sparse graph
- Asymmetric distances (directed graph)
- Negative weights (be careful with INF)
- Multiple edges between same nodes

15. Variations

DP with Submask Enumeration

When you need to iterate over subsets of a mask (e.g., partition into two sets). $O(3^N)$ total.

DP with Bitmask + BFS

For problems like shortest path visiting all nodes (LeetCode 847). Use BFS with mask state.

DP with Bitmask + Sliding Window

When items are arranged in a line and you need to select a subset with adjacency constraints.

16. Related Algorithms/Data Structures

Algorithm	Connection
Hungarian Algorithm	Assignment problem in $O(n^3)$
Held-Karp Algorithm	TSP with bitmask DP
Branch and Bound	Alternative for TSP with larger n
Minimum Cost Flow	Assignment as flow problem
Subset Sum	Foundation of many bitmask DP problems

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Subset Sum	GFG	Subset sum	Easy

Problem	Platform	Pattern	Difficulty
Partition Equal Subset Sum	LeetCode 416	Subset sum DP	Easy

Medium

Problem	Platform	Pattern	Difficulty
Shortest Path Visiting All Nodes	LeetCode 847	BFS + bitmask	Medium
Number of Ways to Wear Different Hats	LeetCode 1434	Bitmask DP	Medium
Minimum Cost to Connect Two Groups	LeetCode 1595	Assignment DP	Medium

Hard

Problem	Platform	Pattern	Difficulty
Find the Shortest Superstring	LeetCode 943	TSP-like	Hard
Minimum Number of Refueling Stops	LeetCode 871	DP with bitmask	Hard
Can I Win	LeetCode 464	Game DP with bitmask	Hard

18. Interview Explanation

"Bitmask DP uses a bitmask integer to represent a subset of items. Each bit indicates whether an item is included. The DP state is `dp[mask][last]` for problems like TSP where order matters, or just `dp[mask]` for problems like assignment where it doesn't. The transition tries adding a new item to the current set. Complexity is $O(n^2 \times 2^n)$ which works for $n \leq 20$. Key operations are checking membership with `mask & (1<<i)` and adding with `mask | (1<<i)`."

19. Revision Notes

- **Key idea:** Bitmask represents subset of items
- **State:** `dp[mask]` or `dp[mask][last]`
- **Transition:** Add new item to mask
- **TSP recurrence:** `dp[mask|(1<<next)][next] = min(dp[mask][last] + cost[last][next])`
- **Complexity:** $O(n^2 \times 2^n)$ time, $O(n \times 2^n)$ space
- **N limit:** $\leq 20-25$ ($2^{20} \approx 1M$, $2^{25} \approx 33M$)
- **Common trap:** Integer overflow, INF overflow, order of loops
- **Submask enumeration:** `for (int sub = mask; sub; sub = (sub-1) & mask)`

20. Final Cheat Sheet

Bitmask DP

When: Small N (≤ 20), subset/permutation/assignment problems

State: `dp[mask]` or `dp[mask][last]`

Mask: bit $i = 1 \rightarrow$ item i included

```
Add:    mask | (1<<i)
Check:  mask & (1<<i)
Time:   O(n2 × 2n)  Space: O(n × 2n)
Edge:   overflow (use 1LL<<n), N=1, INF handling
```

8. Interval DP

1. Overview

Interval DP is a category of DP where the state represents a contiguous interval (subarray) of the input. The DP solves smaller intervals first and combines them to form larger intervals. This is the same pattern as Matrix Chain Multiplication — the classic example of interval DP.

2. Intuition

Think of an interval as a segment $[i, j]$ of an array. The DP value for $[i, j]$ depends on some split point k within the interval, where we combine solutions for $[i, k]$ and $[k+1, j]$ (or $[k, j]$).

Analogy: Imagine you have a line of people and you want to merge adjacent people into groups. To merge a group from i to j , you first merge i to k , then merge $k+1$ to j , then merge the two resulting groups. The cost depends on the split point.

Core insight: The interval DP recurrence follows the pattern:

$$dp[i][j] = \min/\max \text{ over } k \text{ in } [i, j] \text{ of } (dp[i][k] + dp[k+1][j] + \text{cost}(i, j, k))$$

The intervals are filled by increasing length, ensuring subintervals are computed first.

3. When to Use It

- Problems involving merging adjacent elements
- Problems where you process a subarray
- Palindromic substring problems (can also be done with interval DP)
- Burst balloons, minimum cost to merge stones
- Optimal binary search tree
- Polygon triangulation
- Removing boxes / strange printer

Trigger phrases: "subarray", "interval", "merge adjacent", "burst balloons", "merge stones", "remove boxes"

4. When Not to Use It

- When the problem doesn't need the interval structure (e.g., simple prefix/suffix)
- When greedy works (e.g., merging with equal cost)
- For very large N (≥ 1000) — $O(n^3)$ is too slow
- When the problem is about subsequences (not substrings) — uses different DP

5. Core Concepts

5.1 Interval State

$dp[i][j]$ = optimal value for subarray from index i to j (inclusive).

5.2 Length-Based Iteration

Intervals are processed by increasing length (1, 2, 3, ..., n) to ensure that subintervals are computed before the larger intervals.

5.3 Split Point

A split point k divides the interval into two subintervals. The combination of the two subintervals plus the cost of merging gives the value for the larger interval.

5.4 Cost Function

The cost of merging, which depends on the specific problem. Examples:

- MCM: $arr[i-1] \times arr[k] \times arr[j]$
- Burst balloons: $nums[i-1] \times nums[k] \times nums[j+1]$
- Merge stones: sum of stones in interval

6. Step-by-Step Algorithm (Minimum Cost to Merge Stones — basic version)

1. Let $arr[]$ be the array of values
2. Create DP table $dp[n][n]$ initialized to 0
3. For length from 2 to n :
 - For i from 0 to n -length:
 - $j = i + \text{length} - 1$
 - $dp[i][j] = INF$
 - For k from i to $j-1$:
 - $\text{cost} = dp[i][k] + dp[k+1][j] + \text{cost_to_merge}(i, j)$
 - $dp[i][j] = \min(dp[i][j], \text{cost})$
4. Answer = $dp[0][n-1]$

7. Dry Run

Merge stones: $arr = [1, 2, 3]$, cost to merge = sum of all stones in interval

(i,j)	0	1	2
0	0	3	9
1	-	0	5
2	-	-	0

Length 2:

- `dp[0][1]`: $1+2 = 3$ (merge 1 and 2)
- `dp[1][2]`: $2+3 = 5$ (merge 2 and 3)

Length 3:

- `dp[0][2]`:
 - $k=0$: $dp[0][0] + dp[1][2] + (1+2+3) = 0 + 5 + 6 = 11$
 - $k=1$: $dp[0][1] + dp[2][2] + (1+2+3) = 3 + 0 + 6 = 9$
 - $\text{Min} = 9$

Answer: 9

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Minimum cost to merge adjacent elements (cost = sum of interval)
int minCostMerge(const vector<int>& arr) {
    int n = arr.size();
    if (n <= 1) return 0;

    vector<int> prefix(n + 1, 0);
    for (int i = 0; i < n; i++) {
        prefix[i + 1] = prefix[i] + arr[i];
    }

    auto sum = [&](int i, int j) {
        return prefix[j + 1] - prefix[i];
    };

    vector<vector<int>> dp(n, vector<int>(n, 0));

    for (int len = 2; len <= n; len++) {
        for (int i = 0; i + len - 1 < n; i++) {
            int j = i + len - 1;
            dp[i][j] = INT_MAX;
            for (int k = i; k < j; k++) {
                dp[i][j] = min(dp[i][j],
                               dp[i][k] + dp[k + 1][j] + sum(i, j));
            }
        }
    }
    return dp[0][n - 1];
}

// Burst Balloons (LeetCode 312)
int maxCoins(vector<int>& nums) {
    int n = nums.size();
    vector<int> arr(n + 2, 1);
    for (int i = 0; i < n; i++) arr[i + 1] = nums[i];
```

```

vector<vector<int>> dp(n + 2, vector<int>(n + 2, 0));

for (int len = 1; len <= n; len++) {
    for (int i = 1; i + len - 1 <= n; i++) {
        int j = i + len - 1;
        for (int k = i; k <= j; k++) {
            // k is the LAST balloon to burst in [i, j]
            int coins = dp[i][k - 1] + dp[k + 1][j]
                + arr[i - 1] * arr[k] * arr[j + 1];
            dp[i][j] = max(dp[i][j], coins);
        }
    }
}

return dp[1][n];
}

int main() {
    vector<int> arr = {1, 2, 3};
    cout << "Min cost to merge: " << minCostMerge(arr) << endl; // 9

    vector<int> balloons = {3, 1, 5, 8};
    cout << "Max coins: " << maxCoins(balloons) << endl; // 167
    return 0;
}

```

9. Python Implementation

```

def min_cost_merge(arr):
    n = len(arr)
    if n <= 1:
        return 0

    prefix = [0] * (n + 1)
    for i in range(n):
        prefix[i + 1] = prefix[i] + arr[i]

    def range_sum(i, j):
        return prefix[j + 1] - prefix[i]

    dp = [[0] * n for _ in range(n)]

    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            dp[i][j] = float('inf')
            for k in range(i, j):
                dp[i][j] = min(dp[i][j],
                    dp[i][k] + dp[k + 1][j] + range_sum(i, j))

    return dp[0][n - 1]

```

```
def max_coins(nums):
    n = len(nums)
    arr = [1] + nums + [1]
    dp = [[0] * (n + 2) for _ in range(n + 2)]

    for length in range(1, n + 1):
        for i in range(1, n - length + 2):
            j = i + length - 1
            for k in range(i, j + 1):
                coins = dp[i][k - 1] + dp[k + 1][j] + arr[i - 1] * arr[k] * arr[j
+ 1]

                dp[i][j] = max(dp[i][j], coins)

    return dp[1][n]

if __name__ == "__main__":
    print(f"Min cost merge: {min_cost_merge([1, 2, 3])}") # 9
    print(f"Max coins: {max_coins([3, 1, 5, 8])}") # 167
```

10. Code Explanation

- **Length-based iteration:** The outer loop iterates over interval length. This ensures subintervals are computed before larger intervals.
- **Prefix sum:** Used for $O(1)$ range sum queries. `sum(i, j)` gives the total cost of merging the entire interval.
- **Burst Balloons variation:** The key insight is to think backwards — k is the LAST balloon to burst in the interval. When it bursts, the adjacent balloons are `arr[i-1]` and `arr[j+1]` (the boundaries of the interval).
- **`dp[i][j]` initialization:** Set to 0 for single-element intervals (no cost to merge one element).

11. Complexity Analysis

Aspect	Complexity
Time	$O(n^3)$ — three nested loops (length, i , k)
Space	$O(n^2)$ — DP table
With Knuth optimization	$O(n^2)$ (for certain cost functions)

12. Common Patterns

Pattern 1: MCM (Matrix Chain Multiplication)

- **How to identify:** "Minimize cost of multiplying matrices"
- **Approach:** Standard interval DP with split point
- **Example:** GFG — Matrix Chain Multiplication

Pattern 2: Burst Balloons

- **How to identify:** "Burst balloons to maximize coins, $\text{coin} = \text{nums}[i-1] \times \text{nums}[i] \times \text{nums}[i+1]$ "
- **Approach:** Think of last balloon to burst, reverse MCM
- **Example:** LeetCode 312 — Burst Balloons

Pattern 3: Minimum Cost to Merge Stones

- **How to identify:** "Merge adjacent piles with cost = sum of pile"
- **Approach:** Standard interval DP + prefix sum
- **Example:** LeetCode 1000 — Minimum Cost to Merge Stones

Pattern 4: Optimal Binary Search Tree

- **How to identify:** "Minimize search cost in BST with given frequencies"
- **Approach:** Interval DP with weight (sum of frequencies) as cost
- **Example:** GFG — Optimal Binary Search Tree

Pattern 5: Remove Boxes / Strange Printer

- **How to identify:** "Remove boxes to maximize points, or print with minimum operations"
- **Approach:** Interval DP with extra state for color/value
- **Example:** LeetCode 546 — Remove Boxes, LeetCode 664 — Strange Printer

13. Common Mistakes

- Processing intervals in wrong order (not by increasing length)
- Off-by-one: $\text{dp}[i][k] + \text{dp}[k+1][j]$ vs $\text{dp}[i][k-1] + \text{dp}[k][j]$
- Forgetting that $\text{dp}[i][i] = 0$ for single-element intervals
- Not using prefix sums for range queries ($O(n)$ per query makes it $O(n^4)$)
- Using INT_MAX without considering overflow when adding
- Not handling the case where k is the first or last element

14. Edge Cases

- Single element: cost = 0
- Two elements: only one split point
- Array with all equal values
- Negative values (may affect min/max logic)
- Very large values (overflow — use long long)
- Empty array

15. Variations

Knuth Optimization

For cost functions that satisfy the quadrangle inequality (Monge property), the split point k is monotonic, reducing time to $O(n^2)$.

Divide and Conquer DP

For cost functions that are hard to compute but still satisfy optimal substructure.

3D Interval DP

When you need an extra state (e.g., number of merges K , or color of boxes).

16. Related Algorithms/Data Structures

Algorithm	Connection
Matrix Chain Multiplication	Classic interval DP example
Prefix Sum	Used for $O(1)$ range sum queries
Knuth Optimization	Optimizes $O(n^3) \rightarrow O(n^2)$ for certain costs
Divide and Conquer DP	Another optimization technique for interval DP

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Minimum Cost to Merge Stones	LeetCode 1000	Classic interval DP	Easy
Unique Binary Search Trees II	LeetCode 95	Interval DP	Easy

Medium

Problem	Platform	Pattern	Difficulty
Burst Balloons	LeetCode 312	Reverse interval DP	Medium
Minimum Score Triangulation	LeetCode 1039	Polygon triangulation	Medium
Remove Boxes	LeetCode 546	3D interval DP	Medium

Hard

Problem	Platform	Pattern	Difficulty
Strange Printer	LeetCode 664	Interval DP	Hard
Minimum Cost to Cut a Board	LeetCode 1547	Interval DP	Hard
Allocate Mailboxes	LeetCode 1478	Interval DP	Hard

18. Interview Explanation

"Interval DP solves problems on contiguous subarrays where the solution for a larger interval depends on splitting it into smaller intervals. I process intervals by increasing length, and for each interval $[i, j]$, I try all split points k and compute $dp[i][j] = \min \text{ over } k \text{ of } (dp[i][k] + dp[k+1][j] + \text{cost}(i, j, k))$. The cost function depends on the problem — for MCM it's the matrix multiplication cost, for Burst Balloons it's the coin value. Time is $O(n^3)$ and space is $O(n^2)$."

19. Revision Notes

- **Key idea:** Process intervals by increasing length, try split points
- **Recurrence:** $dp[i][j] = \min(dp[i][k] + dp[k+1][j] + cost(i,j,k))$
- **Fill order:** by length (1, 2, 3, ..., n)
- **Base:** $dp[i][i] = 0$
- **Cost function:** Problem-specific (prefix sum for merge, $arr[i-1] \times arr[k] \times arr[j]$ for MCM)
- **Complexity:** $O(n^3)$ time, $O(n^2)$ space
- **Optimization:** Knuth optimization for $O(n^2)$ when cost satisfies quadrangle inequality
- **Common trap:** Wrong loop order, off-by-one in split point

20. Final Cheat Sheet

Interval DP

When: Subarray problems, merging adjacent elements

DP: $dp[i][j]$ = optimal value for interval $[i, j]$

Recurrence: $dp[i][j] = \min/\max(dp[i][k] + dp[k+1][j] + cost)$

Fill: by increasing length

Time: $O(n^3)$ Space: $O(n^2)$

Edge: single element, two elements, negative values

Optimization: Knuth $\rightarrow O(n^2)$

9. Count DP

1. Overview

Count DP (Combinatorial DP) is a category of DP problems where the goal is to count the number of ways to achieve a certain configuration or state. Instead of finding min/max values, we sum over all valid transitions. The DP value at each state represents the number of ways to reach that state.

2. Intuition

If you have a state with multiple ways to reach it, the total number of ways to reach the current state is the sum of ways to reach all predecessor states.

Analogy: Think of a city with roads. If there are 3 ways to reach town A and 2 ways to reach town B, and both A and B have roads to town C, then there are $3+2 = 5$ ways to reach C. This is the fundamental principle of counting DP.

Core insight: Count DP follows the principle:

$$dp[\text{current}] = \text{sum over all valid predecessors of } dp[\text{predecessor}]$$

This is especially important in problems like:

- Number of ways to reach a target sum
- Number of ways to climb stairs
- Number of unique paths in a grid
- Number of ways to make change (coin change 2)
- Number of ways to partition a set

3. When to Use It

- "Number of ways" problems
- Counting paths in a graph/grid
- Counting combinations/partitions
- Problems where order matters (permutations) vs doesn't (combinations)
- Problems with DP recurrence that uses sum instead of min/max

Trigger phrases: "number of ways", "count the number of", "how many ways", "distinct ways"

4. When Not to Use It

- When the answer is a min/max value — use standard DP
- When the number of ways is astronomical and you don't need modulo — use combinatorial formulas
- When the problem has a closed-form formula (e.g., combinations, permutations)
- When the DP state space is too large for counting efficiently

5. Core Concepts

5.1 Summation Recurrence

The fundamental recurrence: $dp[state] = \sum(dp[prev])$ for all valid transitions from prev to state.

5.2 Modulo Arithmetic

Since the number of ways can be huge, answers are typically given modulo $1e9+7$ or similar.

5.3 Order Matters vs Doesn't Matter

- **Order matters (permutations):** Outer loop over target, inner loop over items
- **Order doesn't matter (combinations):** Outer loop over items, inner loop over target

5.4 Base Case

$dp[0] = 1$ or $dp[empty] = 1$ — one way to have nothing.

6. Step-by-Step Algorithm (Coin Change 2 — Number of Ways to Make Change)

1. Let $amount = target$, $coins[] =$ coin denominations
2. Create $dp[amount+1]$, set $dp[0] = 1$
3. For each coin in coins:
 - For target from coin to amount:
 - $dp[target] += dp[target - coin]$

4. Answer = `dp[amount]`

7. Dry Run

coins = [1, 2, 5], amount = 5

target	0	1	2	3	4	5
Initial	1	0	0	0	0	0
After coin 1	1	1	1	1	1	1
After coin 2	1	1	2	2	3	3
After coin 5	1	1	2	2	3	4

Answer: 4 ways (1+1+1+1+1, 1+1+1+2, 1+2+2, 5)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

// Number of ways to make change (coin change 2)
int coinChangeWays(int amount, const vector<int>& coins) {
    vector<int> dp(amount + 1, 0);
    dp[0] = 1;

    for (int coin : coins) {
        for (int target = coin; target <= amount; target++) {
            dp[target] = (dp[target] + dp[target - coin]) % MOD;
        }
    }
    return dp[amount];
}

// Number of ways to climb stairs (order matters: 1 or 2 steps)
int climbStairs(int n) {
    vector<int> dp(n + 1, 0);
    dp[0] = 1;

    for (int i = 1; i <= n; i++) {
        if (i >= 1) dp[i] = (dp[i] + dp[i - 1]) % MOD;
        if (i >= 2) dp[i] = (dp[i] + dp[i - 2]) % MOD;
    }
    return dp[n];
}

// Number of unique paths in a grid
int uniquePaths(int m, int n) {
    vector<vector<int>> dp(m, vector<int>(n, 0));
```

```

// First row and first column: only 1 way
for (int i = 0; i < m; i++) dp[i][0] = 1;
for (int j = 0; j < n; j++) dp[0][j] = 1;

for (int i = 1; i < m; i++) {
    for (int j = 1; j < n; j++) {
        dp[i][j] = (dp[i - 1][j] + dp[i][j - 1]) % MOD;
    }
}
return dp[m - 1][n - 1];
}

// Number of ways to partition a set into k subsets (Stirling numbers)
int stirlingSecond(int n, int k) {
    vector<vector<int>> dp(n + 1, vector<int>(k + 1, 0));
    dp[0][0] = 1;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= min(i, k); j++) {
            dp[i][j] = (dp[i - 1][j - 1] + (long long)j * dp[i - 1][j]) % MOD;
        }
    }
    return dp[n][k];
}

int main() {
    cout << "Coin change ways (5, [1,2,5]): " << coinChangeWays(5, {1, 2, 5}) <<
endl; // 4
    cout << "Climb stairs (5): " << climbStairs(5) << endl; // 8
    cout << "Unique paths (3x7): " << uniquePaths(3, 7) << endl; // 28
    cout << "Stirling 2nd (5,3): " << stirlingSecond(5, 3) << endl; // 25
    return 0;
}

```

9. Python Implementation

```

MOD = 10**9 + 7

def coin_change_ways(amount: int, coins) -> int:
    dp = [0] * (amount + 1)
    dp[0] = 1

    for coin in coins:
        for target in range(coin, amount + 1):
            dp[target] = (dp[target] + dp[target - coin]) % MOD

    return dp[amount]

```

```

def climb_stairs(n: int) -> int:
    dp = [0] * (n + 1)
    dp[0] = 1

    for i in range(1, n + 1):
        if i >= 1:
            dp[i] = (dp[i] + dp[i - 1]) % MOD
        if i >= 2:
            dp[i] = (dp[i] + dp[i - 2]) % MOD

    return dp[n]

def unique_paths(m: int, n: int) -> int:
    dp = [[0] * n for _ in range(m)]

    for i in range(m):
        dp[i][0] = 1
    for j in range(n):
        dp[0][j] = 1

    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = (dp[i - 1][j] + dp[i][j - 1]) % MOD

    return dp[m - 1][n - 1]

def stirling_second(n: int, k: int) -> int:
    dp = [[0] * (k + 1) for _ in range(n + 1)]
    dp[0][0] = 1

    for i in range(1, n + 1):
        for j in range(1, min(i, k) + 1):
            dp[i][j] = (dp[i - 1][j - 1] + j * dp[i - 1][j]) % MOD

    return dp[n][k]

if __name__ == "__main__":
    print(f"Coin change ways: {coin_change_ways(5, [1, 2, 5])}") # 4
    print(f"Climb stairs: {climb_stairs(5)}") # 8
    print(f"Unique paths: {unique_paths(3, 7)}") # 28
    print(f"Stirling 2nd: {stirling_second(5, 3)}") # 25

```

10. Code Explanation

- **Coin change (order doesn't matter):** Outer loop over coins, inner loop over target. This ensures each coin is used at most once per combination (combinations style).
- **Climb stairs (order matters):** Outer loop over step count, inner loop over step sizes. This counts different sequences of steps (1,2 vs 2,1).

- **Unique paths:** $dp[i][j] = dp[i-1][j] + dp[i][j-1]$. Since you can only move right or down, the number of ways to reach (i,j) is the sum of ways from above and left.
- **Stirling numbers:** $dp[i][j] = dp[i-1][j-1] + j * dp[i-1][j]$. Either element i forms its own subset, or it joins one of j existing subsets.

11. Complexity Analysis

Problem	Time	Space
Coin Change 2	$O(N \times \text{amount})$	$O(\text{amount})$
Climb Stairs	$O(N)$	$O(N)$
Unique Paths	$O(M \times N)$	$O(M \times N)$
Stirling 2nd	$O(N \times K)$	$O(N \times K)$

12. Common Patterns

Pattern 1: Coin Change 2 (Combinations)

- **How to identify:** "Number of ways to make sum, order doesn't matter"
- **Approach:** Outer loop over items, inner loop over target
- **Example:** LeetCode 518 — Coin Change 2

Pattern 2: Climb Stairs (Permutations)

- **How to identify:** "Number of ways to reach target, order matters"
- **Approach:** Outer loop over target, inner loop over items
- **Example:** LeetCode 70 — Climbing Stairs

Pattern 3: Grid Paths

- **How to identify:** "Number of ways to reach bottom-right from top-left"
- **Approach:** DP with right/down moves
- **Example:** LeetCode 62 — Unique Paths

Pattern 4: Partition Problems

- **How to identify:** "Number of ways to partition a set/array"
- **Approach:** Stirling numbers, Catalan numbers, or problem-specific DP
- **Example:** LeetCode 698 — Partition to K Equal Sum Subsets

13. Common Mistakes

- Confusing combinations (order doesn't matter) with permutations (order matters)
- Forgetting to use modulo (integer overflow)
- Not initializing $dp[0] = 1$
- Using wrong loop order for the problem
- Not handling the case where items can be used unlimited times vs limited times
- Off-by-one when computing dp array size

14. Edge Cases

- Amount = 0: exactly 1 way (use no coins)
- No coins and amount > 0: 0 ways
- N = 0 or 1 for stairs/unique paths
- Negative numbers (not applicable for counting)
- Large amount (overflow, use modulo)

15. Variations

Combinatorial DP with Constraints

When certain items cannot be used together, or there are limits on how many times an item can be used.

DP with Inclusion-Exclusion

Count total ways, subtract invalid ways using inclusion-exclusion principle.

Count DP on Trees

Count the number of ways to color a tree with constraints.

16. Related Algorithms/Data Structures

Algorithm	Connection
Combinatorics	Closed-form formulas for simple counting problems
Python's <code>math.comb</code>	For direct combination calculations
Modulo Arithmetic	Required for handling large numbers
DP (Min/Max)	Same structure but with sum instead of min/max

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Climbing Stairs	LeetCode 70	Permutations	Easy
Unique Paths	LeetCode 62	Grid DP	Easy

Medium

Problem	Platform	Pattern	Difficulty
Coin Change 2	LeetCode 518	Combinations	Medium
Number of Ways to Stay in Place	LeetCode 1269	Count DP	Medium
Count Square Submatrices	LeetCode 1277	Matrix count	Medium

Hard

Problem	Platform	Pattern	Difficulty
Number of Ways to Form a Target String	LeetCode 1639	Count DP	Hard
Count Vowels Permutation	LeetCode 1220	Count DP	Hard
Ways to Split Array Into Good Subarrays	LeetCode 2750	Count DP	Hard

18. Interview Explanation

"Count DP is used when we need to find the number of ways to achieve a target. The recurrence is $dp[current] = \text{sum}(dp[previous])$ over all valid transitions. The key distinction is whether order matters — for combinations, I iterate over items first, then target; for permutations, I iterate over target first, then items. The base case is always $dp[0] = 1$ (one way to have nothing). I always use modulo to prevent overflow."

19. Revision Notes

- **Key idea:** $dp[current] = \text{sum}(dp[previous])$ over all valid transitions
- **Base case:** $dp[0] = 1$ (one way to have nothing)
- **Combinations (order doesn't matter):** Outer loop over items, inner loop over target
- **Permutations (order matters):** Outer loop over target, inner loop over items
- **Modulo:** Always use $MOD = 1e9+7$
- **Common trap:** Wrong loop order, forgetting modulo
- **Complexity:** $O(N \times \text{target})$ for most problems

20. Final Cheat Sheet

Count DP

When: "Number of ways" problems
 Recurrence: $dp[cur] = \text{sum}(dp[prev])$
 Base: $dp[0] = 1$
 Combinations: items outer, target inner
 Permutations: target outer, items inner
 Modulo: Always use $1e9+7$
 Time: $O(N \times \text{target})$ Space: $O(\text{target})$
 Edge: empty target, no items

10. Probability DP

1. Overview

Probability DP (also called Expected Value DP) is a technique where we compute probabilities or expected values using DP. The recurrence typically involves summing over possible outcomes weighted by their

probabilities. This is crucial for problems involving random processes, games of chance, and stochastic systems.

2. Intuition

The expected value of a random variable is the sum of all possible values, each weighted by its probability: $E[X] = \sum(p_i \times x_i)$. In DP terms, if from a state you can transition to multiple next states with certain probabilities, the expected value of the current state is the sum of (expected value of next state + immediate reward) $\times$ probability.

Analogy: Imagine playing a board game where you roll a die and move forward. To find the expected number of turns to reach the finish, you can compute: for each position, the expected turns = $1 + (1/6) \times$ sum of expected turns from each of the 6 possible next positions.

Core insight: For expected value DP, the recurrence is:

$$E[\text{current}] = \text{sum over all next states of } P(\text{current} \rightarrow \text{next}) \times (\text{cost} + E[\text{next}])$$

This is essentially a system of linear equations that can be solved with DP if the graph is a DAG, or with Gaussian elimination for general graphs.

3. When to Use It

- Expected number of steps/trials to reach a target
- Probability of winning a game
- Expected value of a random process
- Markov chain problems
- Random walk problems
- Problems involving dice rolls, coin flips, card draws
- Problems with "expected value" or "probability" in the description

Trigger phrases: "expected value", "expected number", "probability of", "random", "expected steps", "probability that"

4. When Not to Use It

- When the state space is infinite (use analytical formulas)
- When the problem has a simple closed-form solution (e.g., linearity of expectation)
- When the graph has cycles and you don't want to solve linear equations (use analytical methods)
- When probabilities are complex and require Monte Carlo simulation instead

5. Core Concepts

5.1 Expected Value

$E[X] = \sum(p_i \times x_i)$ — the weighted average of all possible outcomes.

5.2 Law of Total Expectation

$E[X] = \sum(P(\text{event}_i) \times E[X \mid \text{event}_i])$ — the expected value can be decomposed into conditional expectations.

5.3 Linearity of Expectation

$E[X + Y] = E[X] + E[Y]$ — even if X and Y are dependent. This is extremely useful.

5.4 DP Recurrence for Expected Value

$$E[\text{state}] = \text{sum over next of } P(\text{state} \rightarrow \text{next}) \times (\text{reward} + E[\text{next}])$$

If the graph has cycles, this becomes a system of linear equations.

6. Step-by-Step Algorithm (Expected Dice Rolls to Reach Target)

1. Let $dp[i]$ = expected number of rolls to reach $\text{sum} \geq \text{target}$ from current sum i
2. Base: $dp[\text{target}] = 0, dp[\text{target}+1] = 0, \dots, dp[\text{target}+5] = 0$ (already reached or exceeded with last roll)
3. For i from $\text{target}-1$ down to 0:
 - $dp[i] = 1 + (1/6) \times \text{sum}(dp[i+1] + dp[i+2] + \dots + dp[i+6])$
4. Answer = $dp[0]$

7. Dry Run

Expected number of dice rolls to reach $\text{sum} \geq 5$.

i	dp[i]	Calculation
5	0	base
6	0	base
7	0	base
8	0	base
9	0	base
10	0	base
4	1	$1 + (1/6)(0+0+0+0+0+0) = 1$
3	$1 + (1/6)(1+0+0+0+0+0) = 1.167$	
2	$1 + (1/6)(1.167+1+0+0+0+0) = 1.361$	
1	$1 + (1/6)(1.361+1.167+1+0+0+0) = 1.588$	
0	$1 + (1/6)(1.588+1.361+1.167+1+0+0) = 1.853$	

Answer: ≈ 1.853 rolls

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

// Expected number of dice rolls to reach sum >= target
double expectedDiceRolls(int target) {
    vector<double> dp(target + 6, 0.0);
    for (int i = target; i < target + 6; i++) dp[i] = 0.0;

    for (int i = target - 1; i >= 0; i--) {
        double sum = 0.0;
        for (int roll = 1; roll <= 6; roll++) {
            sum += dp[i + roll];
        }
        dp[i] = 1.0 + sum / 6.0;
    }
    return dp[0];
}

// Probability of winning a game (simplified)
// Each round: win with prob p, go to next round with prob q, lose with prob 1-p-q
double probabilityWin(int rounds, double p, double q) {
    vector<double> dp(rounds + 1, 0.0);
    dp[0] = 0.0; // no rounds left = lose

    for (int r = 1; r <= rounds; r++) {
        //  $E[r] = p \cdot 1 + q \cdot E[r+1] + (1-p-q) \cdot 0$ 
        // But this has forward reference, so we need backward equations
        // Actually:  $dp[r] = \text{probability of winning with } r \text{ rounds to play}$ 
        //  $dp[r] = p + q \cdot dp[r-1]$  (win this round OR pass to next round)
        dp[r] = p + q * dp[r - 1];
    }
    return dp[rounds];
}

// Random walk: expected steps to reach 0 or N from position i
// Probability of left = 0.5, probability of right = 0.5
double randomWalkExpectedSteps(int n, int start) {
    // This is a Markov chain with absorbing states at 0 and N
    //  $E[i] = 1 + 0.5 \cdot E[i-1] + 0.5 \cdot E[i+1]$ 
    // This is a system of linear equations with boundary  $E[0] = E[N] = 0$ 
    // Solution:  $E[i] = i \cdot (N - i)$ 
    // This is a special case; for general probabilities, use Gaussian elimination
    return (double)start * (n - start);
}

int main() {
    cout << fixed << setprecision(3);
    cout << "Expected rolls to reach 5: " << expectedDiceRolls(5) << endl; //
~1.853
    cout << "Win prob with 3 rounds (p=0.3, q=0.5): "

```

```

        << probabilityWin(3, 0.3, 0.5) << endl;
    cout << "Expected steps in random walk (N=10, start=5): "
        << randomWalkExpectedSteps(10, 5) << endl; // 25
    return 0;
}

```

9. Python Implementation

```

def expected_dice_rolls(target: int) -> float:
    dp = [0.0] * (target + 6)

    for i in range(target - 1, -1, -1):
        total = sum(dp[i + roll] for roll in range(1, 7))
        dp[i] = 1.0 + total / 6.0

    return dp[0]

def probability_win(rounds: int, p: float, q: float) -> float:
    dp = [0.0] * (rounds + 1)
    dp[0] = 0.0

    for r in range(1, rounds + 1):
        dp[r] = p + q * dp[r - 1]

    return dp[rounds]

def random_walk_expected_steps(n: int, start: int) -> float:
    # E[i] = i * (N - i) for symmetric random walk
    return float(start * (n - start))

if __name__ == "__main__":
    print(f"Expected rolls to reach 5: {expected_dice_rolls(5):.3f}") # ~1.853
    print(f"Win prob: {probability_win(3, 0.3, 0.5):.3f}")
    print(f"Random walk steps: {random_walk_expected_steps(10, 5):.3f}") # 25.0

```

10. Code Explanation

- **Expected dice rolls:** Process from target backwards. For each state, compute expected remaining rolls = 1 (current roll) + average of expected rolls from each possible next state.
- **Probability of winning:** $dp[r] = p + q \times dp[r-1]$. Either win this round (p) with certainty, or pass to next round (q) and continue from there.
- **Random walk:** The recurrence $E[i] = 1 + 0.5 \times E[i-1] + 0.5 \times E[i+1]$ with $E[0] = E[N] = 0$ has a closed form $E[i] = i \times (N-i)$ for symmetric walks.
- **For cycles:** If the DP graph has cycles, we need to solve a system of linear equations using Gaussian elimination or iterative methods.

11. Complexity Analysis

Aspect	Complexity
Expected dice rolls (DAG)	$O(\text{target} \times \text{states})$
Probability win (DAG)	$O(\text{rounds})$
General Markov chain	$O(N^3)$ with Gaussian elimination
Space	$O(N)$ for linear DP, $O(N^2)$ for Gaussian elimination

12. Common Patterns

Pattern 1: Expected Value on DAG

- **How to identify:** "Expected number of steps/rolls to reach target"
- **Approach:** Process states in reverse topological order, $E[i] = 1 + \sum(P[i \rightarrow j] \times E[j])$
- **Example:** Dice roll to reach target

Pattern 2: Gambler's Ruin

- **How to identify:** "Probability of winning before losing all money"
- **Approach:** Markov chain with absorbing boundaries, solve linear equations
- **Example:** LeetCode — New 21 Game

Pattern 3: Random Walk

- **How to identify:** "Random walk on a line/graph"
- **Approach:** Solve recurrence or use symmetry
- **Example:** GFG — Random Walk

Pattern 4: Probability of State

- **How to identify:** "Probability of being in a state after N steps"
- **Approach:** Matrix exponentiation or DP per step
- **Example:** LeetCode 688 — Knight Probability in Chessboard

13. Common Mistakes

- Confusing probability and expected value
- Not handling cycles properly (need Gaussian elimination)
- Using integer arithmetic instead of double
- Forgetting to add 1 (the current step) when computing expected steps
- Not considering that probability DP requires careful floating-point precision
- Wrong base case (e.g., $dp[\text{target}] = 0$ vs $dp[\text{target}] = 1$)

14. Edge Cases

- $p = 0$ or $p = 1$ (deterministic cases)
- Infinite expected value (e.g., unfair random walk with drift away from target)

- Very small probabilities (floating-point underflow)
- Very large state space (cannot use DP)
- Absorbing states (once reached, cannot leave)

15. Variations

Markov Chain Monte Carlo

For very large state spaces, use simulation instead of exact DP.

Matrix Exponentiation

When the transition probabilities are the same for each step, use matrix exponentiation to compute probability after N steps in $O(N^3 \log N)$.

Gaussian Elimination for General Markov Chains

For cycles, set up linear equations and solve with Gaussian elimination.

16. Related Algorithms/Data Structures

Algorithm	Connection
Markov Chains	Foundation of probability DP
Gaussian Elimination	Solving cyclic dependencies
Matrix Exponentiation	Fast computation of N -step probabilities
Monte Carlo Simulation	Approximation for large state spaces
Linearity of Expectation	Simplifies many problems

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Expected Dice Rolls	GFG	Expected value on DAG	Easy
Probability of Winning	Codeforces	Simple probability	Easy

Medium

Problem	Platform	Pattern	Difficulty
New 21 Game	LeetCode 837	Gambler's ruin	Medium
Knight Probability	LeetCode 688	Probability per step	Medium
Soup Servings	LeetCode 808	Expected value DP	Medium

Hard

Problem	Platform	Pattern	Difficulty
Race Car	LeetCode 818	Probability DP	Hard
Probability of a Two Boxes	LeetCode	Markov chain	Hard
Random Pick with Weight	LeetCode 528	Probability distribution	Hard

18. Interview Explanation

"Probability DP computes expected values or probabilities using DP. The core recurrence is $E[\text{state}] = \sum(P(\text{state} \rightarrow \text{next}) \times (\text{reward} + E[\text{next}]))$. For problems without cycles, I process states in reverse topological order. For problems with cycles, I need to set up a system of linear equations and use Gaussian elimination. The key is to correctly identify the states, transitions, and probabilities. I always use double precision and handle edge cases like absorbing states."

19. Revision Notes

- **Key idea:** $E[\text{state}] = \sum(P(\text{transition}) \times (\text{reward} + E[\text{next}]))$
- **DAG:** Process in reverse topological order
- **Cycles:** Use Gaussian elimination to solve linear equations
- **Base:** Absorbing states have $E = 0$ (or reward value)
- **Add 1:** For expected steps, always add 1 for the current step
- **Precision:** Use double, be careful with very small probabilities
- **Complexity:** $O(N)$ for DAG, $O(N^3)$ for general Markov chains

20. Final Cheat Sheet

Probability DP

When: Expected value / probability problems

DAG: $E[i] = 1 + \sum(P[i \rightarrow j] \times E[j])$ (reverse order)

Cycle: Solve linear equations: $E[i] - \sum(P[i \rightarrow j] \times E[j]) = 1$

Base: $E[\text{absorbing}] = 0$

Precision: Use double

Time: $O(N)$ for DAG, $O(N^3)$ for cycles

Edge: $p=0$, $p=1$, infinite expectation

11. SOS DP (Sum Over Subsets)

1. Overview

SOS DP (Sum Over Subsets / Subset Convolution) is a technique for computing DP over subsets of a set, where the state is a bitmask and we need to aggregate information from all subsets (or supersets) of a given mask. It's a powerful optimization that reduces the naive $O(3^N)$ subset iteration to $O(N \times 2^N)$.

2. Intuition

For a bitmask, its subsets are all masks where some bits are turned off. The naive way to iterate over all subsets of all masks is $O(3^N)$ (each element can be in the subset, in the superset, or excluded). SOS DP processes bits one at a time, building contributions incrementally.

Analogy: Imagine you have a list of features for products. For each product, you want to know the total value of all products that have a subset of its features. Instead of checking every pair of products, you process one feature at a time, building up the subset contributions.

Core insight: The SOS DP recurrence:

```
dp[mask][i] = sum over subsets of mask that differ only in the first i bits
```

Which simplifies to:

```
if (mask & (1<<i)): dp[mask] += dp[mask ^ (1<<i)]
```

This processes each bit independently, propagating subset sums in $O(N \times 2^N)$.

6. Step-by-Step Algorithm

1. Initialize `dp[mask] = f[mask]` (base value for each mask)
2. For each bit i from 0 to $N-1$:
 - For each mask from 0 to $(1 \ll N)-1$:
 - If `mask & (1<<i)`: `dp[mask] += dp[mask ^ (1<<i)]`
3. Now `dp[mask] = sum of f[sub]` for all subsets `sub` of `mask`

7. Dry Run

$N = 2$, $f = [a_0, a_1, a_2, a_3]$ for masks 00, 01, 10, 11

Initial `dp` = $[a_0, a_1, a_2, a_3]$

Bit 0 ($i=0$):

- `mask=01 (1)`: `dp[01] += dp[00] =  $a_1 + a_0$`
- `mask=11 (3)`: `dp[11] += dp[10] =  $a_3 + a_2$`
- `dp` = $[a_0, a_1+a_0, a_2, a_3+a_2]$

Bit 1 ($i=1$):

- `mask=10 (2)`: `dp[10] += dp[00] =  $a_2 + a_0$`
- `mask=11 (3)`: `dp[11] += dp[01] =  $(a_3+a_2) + (a_1+a_0)$`
- `dp` = $[a_0, a_1+a_0, a_2+a_0, a_3+a_2+a_1+a_0]$

Now `dp[mask] = sum of f[sub]` over all subsets of `mask`.

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

// SOS DP: sum over subsets
// dp[mask] = sum of f[sub] for all subsets sub of mask
vector<int> sumOverSubsets(const vector<int>& f, int n) {
    int N = 1 << n;
    vector<int> dp = f;

    for (int i = 0; i < n; i++) {
        for (int mask = 0; mask < N; mask++) {
            if (mask & (1 << i)) {
                dp[mask] += dp[mask ^ (1 << i)];
            }
        }
    }
    return dp;
}

// SOS DP: sum over supersets
// dp[mask] = sum of f[super] for all supersets super of mask
vector<int> sumOverSupersets(const vector<int>& f, int n) {
    int N = 1 << n;
    vector<int> dp = f;

    for (int i = 0; i < n; i++) {
        for (int mask = 0; mask < N; mask++) {
            if (!(mask & (1 << i))) {
                dp[mask] += dp[mask | (1 << i)];
            }
        }
    }
    return dp;
}

// Example: Count pairs where (a[i] & a[j]) == 0
long long countPairsWithZeroAnd(const vector<int>& arr, int maxBit) {
    int N = 1 << maxBit;
    vector<int> freq(N, 0);
    for (int x : arr) freq[x]++;

    // SOS DP: freq[mask] = sum of freq[sub] for all subsets sub of mask
    // This tells us how many numbers are subsets of mask
    vector<int> dp = sumOverSubsets(freq, maxBit);

    // For each number, count how many numbers are subsets of its complement
    long long ans = 0;
    for (int x : arr) {
        int comp = (~x) & (N - 1); // complement within N bits
        ans += dp[comp]; // numbers y where y & x == 0
    }
    // Each pair counted twice

```

```

        return (ans - arr.size()) / 2;
    }

    int main() {
        // Example: SOS DP for subset sums
        int n = 3;
        vector<int> f = {1, 2, 3, 4, 5, 6, 7, 8}; // 8 masks

        auto subsetSum = sumOverSubsets(f, n);
        cout << "Sum over subsets:" << endl;
        for (int mask = 0; mask < 8; mask++) {
            cout << "mask " << mask << ": " << subsetSum[mask] << endl;
        }

        // Example: count pairs with (a & b) == 0
        vector<int> arr = {1, 2, 3, 4, 5};
        cout << "Pairs with zero AND: " << countPairsWithZeroAnd(arr, 4) << endl;

        return 0;
    }

```

9. Python Implementation

```

def sum_over_subsets(f, n):
    """SOS DP: dp[mask] = sum of f[sub] for all subsets sub of mask"""
    N = 1 << n
    dp = f[:]

    for i in range(n):
        for mask in range(N):
            if mask & (1 << i):
                dp[mask] += dp[mask ^ (1 << i)]

    return dp

def sum_over_supersets(f, n):
    """SOS DP: dp[mask] = sum of f[super] for all supersets super of mask"""
    N = 1 << n
    dp = f[:]

    for i in range(n):
        for mask in range(N):
            if not (mask & (1 << i)):
                dp[mask] += dp[mask | (1 << i)]

    return dp

def count_pairs_with_zero_and(arr, max_bit):
    """Count pairs where (a[i] & a[j]) == 0"""

```

```

N = 1 << max_bit
freq = [0] * N
for x in arr:
    freq[x] += 1

dp = sum_over_subsets(freq, max_bit)

ans = 0
for x in arr:
    comp = (~x) & (N - 1)
    ans += dp[comp]

return (ans - len(arr)) // 2

if __name__ == "__main__":
    n = 3
    f = [1, 2, 3, 4, 5, 6, 7, 8]
    subset_sum = sum_over_subsets(f, n)
    for mask in range(8):
        print(f"mask {mask}: {subset_sum[mask]}")

    arr = [1, 2, 3, 4, 5]
    print(f"Pairs with zero AND: {count_pairs_with_zero_and(arr, 4)}")

```

10. Code Explanation

- **Sum over subsets:** For each bit i , if the mask has bit i set, we add the value from the mask without that bit. This propagates subset contributions iteratively.
- **Sum over supersets:** Same idea but reversed. If the mask does NOT have bit i set, add the value from the mask with that bit set.
- **Order of loops:** The outer loop is over bits, inner loop over all masks. This is the key — processing one bit at a time.
- **Zero AND pairs:** $\text{freq}[\text{mask}]$ = count of elements with value = mask. SOS DP gives $\text{dp}[\text{mask}]$ = count of elements whose value is a subset of mask. For each element x , we want elements y where $x \& y == 0$, meaning y is a subset of $\sim x$ (complement).

11. Complexity Analysis

Aspect	Complexity
Time	$O(N \times 2^N)$
Space	$O(2^N)$
Naive alternative	$O(3^N)$

12. Common Patterns

Pattern 1: Subset Sum

- **How to identify:** "Sum over all subsets of each mask"
- **Approach:** Standard SOS DP
- **Example:** Codeforces — SOS DP Problems

Pattern 2: Zero AND Pairs

- **How to identify:** "Count pairs where $(a[i] \& a[j]) == 0$ "
- **Approach:** SOS on complement
- **Example:** Codeforces — AND Graph

Pattern 3: Subset GCD

- **How to identify:** "Sum over subsets where GCD or other bitwise property holds"
- **Approach:** SOS DP on frequency array
- **Example:** Codeforces — GCD of Subsets

Pattern 4: Superset Sum

- **How to identify:** "Sum over all supersets of each mask"
- **Approach:** Reverse SOS DP
- **Example:** Codeforces — OR in Matrix

13. Common Mistakes

- Wrong loop order (inner over bit, outer over mask) — this doesn't work
- Off-by-one in bit count ($N = 1 \ll n$, but masks go from 0 to $N-1$)
- Forgetting to use `long long` for large sums
- Confusing subset and superset ($\text{mask} \& (1 \ll i)$ vs $!(\text{mask} \& (1 \ll i))$)
- Using SOS DP when N is too large ($N > 25$)

14. Edge Cases

- $\text{mask} = 0$ (empty set) — subset of everything
- $\text{mask} = N-1$ (full set) — includes all items
- $f[\text{mask}] = 0$ for most masks
- Large values (overflow, use long long)
- Negative values (be careful with sum)

15. Variations

Subset Convolution

For combining two functions f and g where $\text{dp}[\text{mask}] = \text{sum over sub of } f[\text{sub}] \times g[\text{mask} \wedge \text{sub}]$. Uses $O(N^2 \times 2^N)$ with SOS DP.

Fast Zeta Transform

The mathematical name for SOS DP. Used in inclusion-exclusion and Möbius inversion on subsets.

Fast Möbius Transform

Inverse of SOS DP. $dp[mask] = \text{sum over sub of } (-1)^{(|mask|-|sub|)} \times f[sub]$.

16. Related Algorithms/Data Structures

Algorithm	Connection
Bitmask DP	Foundation of SOS DP
Subset Convolution	Extension of SOS DP
Fast Zeta Transform	Mathematical name for SOS DP
Principle of Inclusion-Exclusion	SOS DP is used for efficient PIE
FFT (Fast Fourier Transform)	Similar concept but for polynomials

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Sum Over Subsets	Codeforces	Standard SOS	Easy
AND Graph	Codeforces	Zero AND pairs	Easy

Medium

Problem	Platform	Pattern	Difficulty
Bitwise OR of Subarrays	LeetCode 898	Subset properties	Medium
Subset GCD	Codeforces	GCD over subsets	Medium
Compatible Numbers	Codeforces	Zero AND pairs	Medium

Hard

Problem	Platform	Pattern	Difficulty
Subset Convolution	Codeforces	Advanced SOS	Hard
Varying Kibibits	Codeforces	Complex SOS	Hard
Wizards and Numbers	Codeforces	SOS + game theory	Hard

18. Interview Explanation

"SOS DP efficiently computes sums over all subsets of each mask in $O(N \times 2^N)$ instead of $O(3^N)$. The key idea is to process one bit at a time: for each bit i , for each mask that has that bit set, I add the value from the mask without that bit. This builds up the subset sum incrementally. It's used in problems involving bitwise AND/OR properties and counting pairs with zero AND. The space is $O(2^N)$."

19. Revision Notes

- **Key idea:** Process bits one at a time, propagate subset sums
- **Recurrence:** `if (mask & (1<<i)) dp[mask] += dp[mask ^ (1<<i)]`
- **Subset vs superset:** Subset = add from mask without bit, Superset = add from mask with bit
- **Complexity:** $O(N \times 2^N)$ time, $O(2^N)$ space
- **Naive alternative:** $O(3^N)$
- **Common trap:** Wrong loop order (must be bit outer, mask inner)
- **N limit:** ≤ 25 ($2^{25} \approx 33M$, may be slow)

20. Final Cheat Sheet

SOS DP (Sum Over Subsets)

When: Subset sum queries, zero AND pairs, bitwise properties

DP: `dp[mask] = sum of f[sub] over all subsets of mask`

Recurrence: `if (mask & (1<<i)) dp[mask] += dp[mask ^ (1<<i)]`

Order: outer loop over bits, inner loop over masks

Time: $O(N \times 2^N)$ Space: $O(2^N)$

Edge: `mask=0`, `mask=N-1`, large values overflow

12. Convex Hull Trick (CHT) Optimization

1. Overview

Convex Hull Trick (CHT) is a DP optimization technique used when the DP recurrence is of the form:

$$dp[i] = \min/\max \text{ over } j < i \text{ of } (dp[j] + m[j] \times x[i] + c[j])$$

where `m[j]` and `c[j]` are slopes and intercepts of lines, and `x[i]` is a monotonic query point. Instead of checking all `j`, we maintain a convex hull of lines and query the best line at each `x`.

2. Intuition

Each choice of `j` defines a line: `y = m[j] * x + c[j]`. The value `dp[i]` is the minimum (or maximum) `y`-value among all lines at `x = x[i]`.

Analogy: Imagine you have a set of lines on a graph. For a given `x`-coordinate, you want the line with the lowest `y`-value. As `x` increases, the optimal line changes. CHT maintains only the "useful" lines — those that form the lower (or upper) envelope.

Core insight:

- If slopes are monotonic (increasing or decreasing), we can use a deque-based CHT (Li Chao or convex hull)
- If `x`-values are monotonic, we can pop from the front of the deque
- The intersection point of two adjacent lines determines when the optimal line switches

3. When to Use It

- DP with division: $dp[i] = \min(dp[j] + (x[i] - x[j])^2 + C)$ or similar
- DP with cost = slope $\times$ $x[i]$ + intercept
- Problems where the recurrence has a linear term in $x[i]$
- Usually seen in problems with $N \geq 10^5$ where $O(N^2)$ is too slow

Trigger phrases: " $dp[i] = \min(dp[j] + (a[i] - a[j])^2)$ ", "divide and conquer", "line container", "dp optimization"

4. When Not to Use It

- When slopes are not monotonic (unless using Li Chao segment tree)
- When N is small ($N \leq 10^3$) — $O(N^2)$ is fine
- When the cost function is not linear in the query point
- When the problem has a simpler solution (e.g., prefix sums)
- When x -values are not monotonic and slopes are not monotonic (use Li Chao tree instead)

5. Core Concepts

5.1 Line

A line is $y = m \times x + c$ where m = slope, c = intercept.

5.2 Lower Envelope

The set of lines that are optimal for at least one x . For minimization, it's the lower convex hull.

5.3 Intersection Point

The x -coordinate where line (m_1, c_1) becomes better than (m_2, c_2) :

$$x = (c_2 - c_1) / (m_1 - m_2)$$

5.4 Deque Maintenance

We maintain lines in a deque. For each new line, we check if it makes the last line obsolete by comparing intersection points.

6. Step-by-Step Algorithm (Deque-based, monotonic slopes and queries)

1. Maintain a deque of lines (m, c)
2. For each query x (in increasing order):
 - While the first two lines' intersection $\leq x$, pop the first line
 - Query the first line: $y = m \times x + c$
3. To add a new line (m, c) :
 - While the last two lines' intersection $\geq$ intersection of last line with new line, pop the last line
 - Add the new line to the back

7. Dry Run

Problem: $dp[i] = \min(dp[j] + (a[i] - a[j])^2) = a[i]^2 + \min(dp[j] + a[j]^2 - 2 \times a[i] \times a[j])$

Let $m = -2 \times a[j]$, $c = dp[j] + a[j]^2$, $x = a[i]$

Lines at each step:

- $j=0$: $dp[0]=0$, $a[0]=0 \rightarrow m=0$, $c=0$
- $j=1$: $dp[1]=1$, $a[1]=1 \rightarrow m=-2$, $c=2$
- etc.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Convex Hull Trick for DP optimization
// Recurrence: dp[i] = min(dp[j] + m[j] * x[i] + c[j])
// Assumes: slopes are monotonic (decreasing for min), query x is increasing

struct Line {
    long long m, c; // y = m * x + c
    long long eval(long long x) const { return m * x + c; }
};

// For double intersection points (avoid precision issues with fractions)
struct CHT {
    deque<Line> dq;

    // Check if line l2 is unnecessary
    bool isBad(const Line& l1, const Line& l2, const Line& l3) {
        // (c3 - c1) / (m1 - m3) <= (c2 - c1) / (m1 - m2)
        // Cross-multiply to avoid division
        return (__int128)(l3.c - l1.c) * (l1.m - l2.m)
            <= (__int128)(l2.c - l1.c) * (l1.m - l3.m);
    }

    // Add line: ensure slopes are decreasing (for min queries)
    void addLine(long long m, long long c) {
        Line newLine = {m, c};
        while (dq.size() >= 2 && isBad(dq[dq.size() - 2], dq.back(), newLine)) {
            dq.pop_back();
        }
        dq.push_back(newLine);
    }

    // Query minimum at x (x is increasing)
    long long query(long long x) {
        while (dq.size() >= 2 && dq[0].eval(x) >= dq[1].eval(x)) {
            dq.pop_front();
        }
        return dq[0].eval(x);
    }
};
```

```

    }
};

// Example: Minimizing  $dp[i] = \min(dp[j] + (a[i] - a[j])^2) + C$ 
// This simplifies to:  $dp[i] = a[i]^2 + C + \min(dp[j] + a[j]^2 - 2*a[i]*a[j])$ 
// Line:  $m = -2*a[j]$ ,  $c = dp[j] + a[j]^2$ , query  $x = a[i]$ 
long long solveDP(const vector<long long>& a, long long C) {
    int n = a.size();
    vector<long long> dp(n, 0);
    CHT cht;

    dp[0] = 0; // or some base cost
    // For  $j=0$ :  $m = -2*a[0]$ ,  $c = dp[0] + a[0]^2 = a[0]^2$ 
    cht.addLine(-2 * a[0], dp[0] + a[0] * a[0]);

    for (int i = 1; i < n; i++) {
        long long best = cht.query(a[i]);
        dp[i] = a[i] * a[i] + C + best;
        cht.addLine(-2 * a[i], dp[i] + a[i] * a[i]);
    }
    return dp[n - 1];
}

int main() {
    vector<long long> a = {0, 1, 3, 6};
    long long C = 0;
    cout << "DP result: " << solveDP(a, C) << endl;
    return 0;
}

```

9. Python Implementation

```

from collections import deque

class CHT:
    """Convex Hull Trick for min queries with decreasing slopes and increasing x"""

    def __init__(self):
        self.dq = deque()

    def _is_bad(self, l1, l2, l3):
        # Returns True if l2 is unnecessary
        #  $(c_3 - c_1) * (m_1 - m_2) \leq (c_2 - c_1) * (m_1 - m_3)$ 
        return (l3[1] - l1[1]) * (l1[0] - l2[0]) <= (l2[1] - l1[1]) * (l1[0] - l3[0])

    def add_line(self, m, c):
        """Add line  $y = m*x + c$ . Slopes must be decreasing."""
        while len(self.dq) >= 2 and self._is_bad(self.dq[-2], self.dq[-1], (m,

```

```

c)):
    self.dq.pop()
    self.dq.append((m, c))

def query(self, x):
    """Query minimum at x. x must be increasing."""
    while len(self.dq) >= 2:
        m1, c1 = self.dq[0]
        m2, c2 = self.dq[1]
        if m1 * x + c1 >= m2 * x + c2:
            self.dq.popleft()
        else:
            break
    m, c = self.dq[0]
    return m * x + c

def solve_dp(a, C):
    """dp[i] = min(dp[j] + (a[i] - a[j])^2) + C"""
    n = len(a)
    dp = [0] * n
    cht = CHT()

    dp[0] = 0
    cht.add_line(-2 * a[0], dp[0] + a[0] * a[0])

    for i in range(1, n):
        best = cht.query(a[i])
        dp[i] = a[i] * a[i] + C + best
        cht.add_line(-2 * a[i], dp[i] + a[i] * a[i])

    return dp[n - 1]

if __name__ == "__main__":
    a = [0, 1, 3, 6]
    print(f"DP result: {solve_dp(a, 0)}")

```

10. Code Explanation

- **Line representation:** Each choice j is a line $y = m \cdot x + c$. The DP $dp[i]$ is the minimum y -value at $x = a[i]$.
- **isBad:** Checks if three lines make the middle one unnecessary. Uses cross-multiplication to avoid floating-point division.
- **addLine:** Maintains the lower convex hull. While the last line is unnecessary, pop it.
- **query:** For increasing x , while the first line is worse than the second, pop it. The first line is optimal for the current x .
- **DP example:** $dp[i] = \min(dp[j] + (a[i] - a[j])^2)$. Expand: $dp[i] = a[i]^2 + \min(dp[j] + a[j]^2 - 2 \times a[i] \times a[j])$. Here $m = -2 \times a[j]$, $c = dp[j] + a[j]^2$, $x = a[i]$.

11. Complexity Analysis

Aspect	Complexity
Add line (amortized)	$O(1)$
Query (amortized)	$O(1)$
Total for N lines/queries	$O(N)$
Space	$O(N)$

12. Common Patterns

Pattern 1: DP with Square

- **How to identify:** $dp[i] = \min(dp[j] + (x[i] - x[j])^2 + C)$
- **Approach:** Expand the square, identify m, c, x
- **Example:** LeetCode — Minimum Cost to Cut a Board

Pattern 2: DP with Division

- **How to identify:** $dp[i] = \min(dp[j] + (sum[i] - sum[j]) / something)$
- **Approach:** Rearrange into linear form
- **Example:** Codeforces — Calendar

Pattern 3: Alien's Trick (Lagrange Multiplier)

- **How to identify:** "Minimize cost with exactly K items" — use with CHT
- **Approach:** Binary search on penalty, run CHT, count items used
- **Example:** Codeforces — CF 125E

13. Common Mistakes

- Using CHT when slopes are not monotonic (need Li Chao tree)
- Overflow in cross-multiplication (use `__int128` in C++ or `Decimal` in Python)
- Wrong sign for slopes (decreasing for min, increasing for max)
- Not handling the case where two lines have the same slope
- Forgetting that CHT requires both slope monotonicity and query monotonicity for deque version

14. Edge Cases

- Single line (no choice)
- Parallel lines (same slope, different intercept)
- Floating-point intersection (use cross-multiplication instead)
- Very large values (overflow with `long long`)
- Non-monotonic queries (use Li Chao segment tree)

15. Variations

Li Chao Segment Tree

For non-monotonic slopes or queries. Implement a segment tree over x-coordinates, each node stores the line that is optimal at the midpoint.

Upper Hull (Max Queries)

Flip signs: add lines with increasing slopes, pop when first line $\leq$ second line.

Dynamic CHT (Li Chao Tree)

Allows adding lines and querying at any x, not just monotonic.

16. Related Algorithms/Data Structures

Algorithm	Connection
Li Chao Segment Tree	Generalization for non-monotonic queries
Divide and Conquer DP Optimization	Another DP optimization technique
Knuth Optimization	Yet another DP optimization
Convex Hull (Geometry)	Same concept but in 2D geometry

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Line Container	CSES	Basic CHT	Easy
DP with CHT	AtCoder	Standard	Easy

Medium

Problem	Platform	Pattern	Difficulty
Minimum Cost to Cut	Codeforces	DP with square	Medium
Aliens (IOI)	Codeforces	CHT + Lagrange	Medium
Calendar	Codeforces	DP with division	Medium

Hard

Problem	Platform	Pattern	Difficulty
Partition the Array	Codeforces	Complex CHT	Hard
Mowing the Field	USACO	CHT on 2D	Hard
Guardians of the Galaxy	Codeforces	Advanced CHT	Hard

18. Interview Explanation

"Convex Hull Trick optimizes DP of the form $dp[i] = \min(dp[j] + m[j] \times x[i] + c[j])$. Each j defines a line, and we need the minimum y at $x = x[i]$. If slopes are decreasing and queries are increasing, I maintain a deque of lines. For each query, I pop lines from the front until the first line is optimal. For each new line, I pop from the back while the last line is unnecessary. Each operation is $O(1)$ amortized, making the total $O(N)$."

19. Revision Notes

- **Key idea:** Each j is a line, query minimum at $x[i]$
- **Recurrence form:** $dp[i] = \min(dp[j] + m[j] \times x[i] + c[j])$
- **Deque CHT:** Needs monotonic slopes and monotonic queries
- **Li Chao Tree:** For non-monotonic cases
- **Complexity:** $O(N)$ for deque CHT, $O(N \log X)$ for Li Chao
- **Common trap:** Overflow in cross-multiplication, non-monotonic slopes
- **isBad check:** $(c_3 - c_1)(m_1 - m_2) \leq (c_2 - c_1)(m_1 - m_3)$

20. Final Cheat Sheet

Convex Hull Trick (CHT)

When: $dp[i] = \min(dp[j] + m[j] \times x[i] + c[j])$
 Deque CHT: slopes decreasing, x increasing
 Li Chao: general case, $O(N \log X)$
 Add line: $O(1)$ amortized
 Query: $O(1)$ amortized
 Time: $O(N)$ for deque, $O(N \log X)$ for Li Chao
 Edge: parallel lines, overflow, single line

13. Divide and Conquer DP Optimization

1. Overview

Divide and Conquer DP (D&C DP) is an optimization technique for DP recurrences of the form:

$$dp[i][j] = \min \text{ over } k < i \text{ of } (dp[k][j-1] + \text{cost}(k+1, i))$$

where the optimal split point $\text{opt}[i][j]$ is monotonic in i (i.e., $\text{opt}[i][j] \leq \text{opt}[i+1][j]$). This allows us to divide the range of k and recursively compute the DP, reducing $O(N^2 \times K)$ to $O(N \log N \times K)$ or $O(NK \log N)$.

2. Intuition

The key insight is the **monotonicity of optimal split points**: as i increases, the optimal k also increases (or stays the same). This is called the "quadrangle inequality" or "Monge property."

Analogy: Imagine you're cutting a rope at different positions. If you want to make more cuts in a longer rope, you don't start cutting from the leftmost point — you start from where you left off. The optimal cut position moves rightward as the rope gets longer.

Core insight: When computing $dp[i][j]$ for a range of i values, the optimal k for each i is monotonic. This means we don't need to search all k for each i — we can restrict the search range using the monotonicity property.

3. When to Use It

- DP with 2D states where one dimension is the number of groups/partitions
- $dp[i][j] = \min(dp[k][j-1] + \text{cost}(k+1, i))$ where cost satisfies quadrangle inequality
- Partition problems (divide array into K groups to minimize cost)
- Problems where the optimal split point is monotonic
- Usually for problems with $N \geq 10^4$ and $K \geq 10$ where $O(N^2K)$ is too slow

Trigger phrases: "partition array into K groups", " $dp[i][j] = \min(dp[k][j-1] + \text{cost})$ ", "divide and conquer DP"

4. When Not to Use It

- When the optimal split point is not monotonic
- When cost computation is expensive (each cost call should be $O(1)$ or $O(\log N)$)
- When N is small ($O(N^2K)$ is acceptable)
- When the DP has a simpler solution (e.g., prefix sums)
- When the cost function doesn't satisfy quadrangle inequality

5. Core Concepts

5.1 Monotonic Optimal Split Point

$opt[i][j] \leq opt[i+1][j]$ — the optimal k for i is always $\leq$ the optimal k for $i+1$.

5.2 Quadrangle Inequality

The cost function satisfies: $\text{cost}(a, c) + \text{cost}(b, d) \leq \text{cost}(a, d) + \text{cost}(b, c)$ for $a \leq b \leq c \leq d$.

5.3 Divide and Conquer

For a fixed j , instead of computing $dp[i][j]$ for all i in order, we:

1. Pick the middle i
2. Find $opt[mid][j]$ by checking all k in the allowed range
3. Recursively compute for left half ($k \leq opt[mid]$) and right half ($k \geq opt[mid]$)

6. Step-by-Step Algorithm

1. Define $dp[i][j] = \min$ cost to partition first i elements into j groups
2. Define $\text{compute}(l, r, optL, optR, j)$:
 - If $l > r$, return
 - $mid = (l + r) / 2$

- Find `opt[mid]` by checking all `k` from `optL` to `min(optR, mid-1)`
 - Recursively compute left: `compute(1, mid-1, optL, opt[mid], j)`
 - Recursively compute right: `compute(mid+1, r, opt[mid], optR, j)`
3. For each `j` from 1 to `K`, call `compute(1, N, 0, N-1, j)`

7. Dry Run

$N = 4, K = 2, \text{cost}(i, j) = (j-i+1)^2$ (cost of one group from i to j)

$\text{dp}[i][j]$ = min cost to partition first i elements into j groups.

For $j=1$: $\text{dp}[i][1] = \text{cost}(1, i) = i^2$

For $j=2$: compute i from 1 to 4

- $i=1$: $\text{dp}[1][2] = \text{INF}$ (can't partition 1 element into 2 groups)
- $i=2$: $\text{dp}[2][2] = \min \text{ over } k < 2: \text{dp}[1][1] + \text{cost}(2,2) = 1 + 1 = 2, \text{opt}=1$
- $i=3$: $\text{dp}[3][2] = \min(\text{dp}[1][1] + \text{cost}(2,3), \text{dp}[2][1] + \text{cost}(3,3)) = \min(1+4, 4+1) = 5, \text{opt}=1 \text{ or } 2$
- $i=4$: $\text{dp}[4][2] = \min(\text{dp}[1][1] + \text{cost}(2,4), \text{dp}[2][1] + \text{cost}(3,4), \text{dp}[3][1] + \text{cost}(4,4)) = \min(1+9, 4+4, 9+1) = 8, \text{opt}=2$

Optimal split points: $\text{opt}[2]=1, \text{opt}[3]=1, \text{opt}[4]=2$ — monotonic increasing.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Divide and Conquer DP Optimization
// dp[i][j] = min over k < i of (dp[k][j-1] + cost(k+1, i))

class DnCDP {
    int n, k;
    const long long INF = 1e18;
    vector<vector<long long>> dp;
    vector<long long> prefix;

    // Cost of group from l to r (inclusive)
    // Example: (sum[l..r])^2 - change based on problem
    long long cost(int l, int r) {
        long long sum = prefix[r] - prefix[l - 1];
        return sum * sum;
    }

    // Compute dp[l..r][j] using D&C
    // optL and optR are bounds for optimal k
    void compute(int l, int r, int optL, int optR, int j) {
        if (l > r) return;

        int mid = (l + r) / 2;
        int bestK = -1;
        dp[mid][j] = INF;
```

```

        // Find optimal k for mid
        for (int k = optL; k <= min(optR, mid - 1); k++) {
            long long val = dp[k][j - 1] + cost(k + 1, mid);
            if (val < dp[mid][j]) {
                dp[mid][j] = val;
                bestK = k;
            }
        }

        // Recursively compute left and right halves
        compute(l, mid - 1, optL, bestK, j);
        compute(mid + 1, r, bestK, optR, j);
    }

public:
    DnCDP(int n, int k, const vector<long long>& arr)
        : n(n), k(k), dp(n + 1, vector<long long>(k + 1, INF)) {
        prefix.resize(n + 1, 0);
        for (int i = 1; i <= n; i++) {
            prefix[i] = prefix[i - 1] + arr[i - 1];
        }

        // Initialize j=1 (one group)
        for (int i = 1; i <= n; i++) {
            dp[i][1] = cost(1, i);
        }

        // Compute for j = 2..k
        for (int j = 2; j <= k; j++) {
            compute(1, n, 1, n, j);
        }
    }

    long long getAnswer() {
        return dp[n][k];
    }
};

int main() {
    vector<long long> arr = {1, 2, 3, 4};
    int n = 4, k = 2;
    DnCDP solver(n, k, arr);
    cout << "Min cost: " << solver.getAnswer() << endl;
    return 0;
}

```

9. Python Implementation

```

class DnCDP:
    def __init__(self, arr, k):

```

```

self.n = len(arr)
self.k = k
self.INF = 10**18

# Prefix sum for O(1) cost
self.prefix = [0] * (self.n + 1)
for i in range(1, self.n + 1):
    self.prefix[i] = self.prefix[i - 1] + arr[i - 1]

# dp[i][j] = min cost for first i elements, j groups
self.dp = [[self.INF] * (k + 1) for _ in range(self.n + 1)]

# j = 1
for i in range(1, self.n + 1):
    self.dp[i][1] = self._cost(1, i)

# j = 2..k
for j in range(2, k + 1):
    self._compute(1, self.n, 1, self.n, j)

def _cost(self, l, r):
    s = self.prefix[r] - self.prefix[l - 1]
    return s * s # (sum)^2, change as needed

def _compute(self, l, r, optL, optR, j):
    if l > r:
        return

    mid = (l + r) // 2
    best_k = -1
    self.dp[mid][j] = self.INF

    for k in range(optL, min(optR, mid - 1) + 1):
        val = self.dp[k][j - 1] + self._cost(k + 1, mid)
        if val < self.dp[mid][j]:
            self.dp[mid][j] = val
            best_k = k

    self._compute(l, mid - 1, optL, best_k, j)
    self._compute(mid + 1, r, best_k, optR, j)

def get_answer(self):
    return self.dp[self.n][self.k]

if __name__ == "__main__":
    arr = [1, 2, 3, 4]
    solver = DnCDP(arr, 2)
    print(f"Min cost: {solver.get_answer()}")

```

10. Code Explanation

- **compute(l, r, optL, optR, j):** Computes $dp[i][j]$ for all i in $[l, r]$. The optimal k for i is known to be in $[optL, optR]$.
- **Mid-point divide:** Pick the middle i , find its optimal k by checking all k in $[optL, optR]$. Then recursively solve left half ($k \leq optK$) and right half ($k \geq optK$).
- **Monotonicity:** The left half's optimal k is $\leq optK$, and the right half's is $\geq optK$. This reduces the search range dramatically.
- **Cost function:** Must be $O(1)$ or $O(\log N)$ for the algorithm to be efficient.

11. Complexity Analysis

Aspect	Complexity
Each compute call	$O(N \log N)$ per j
Total	$O(K \times N \log N)$
Naive	$O(K \times N^2)$
Space	$O(N \times K)$ or $O(N)$ with optimization

12. Common Patterns

Pattern 1: Partition into K Groups

- **How to identify:** "Divide array into K subarrays, minimize sum of costs"
- **Approach:** D&C DP with cost function for each subarray
- **Example:** LeetCode 1335 — Minimum Difficulty of Job Schedule

Pattern 2: Optimal Binary Search Tree

- **How to identify:** "Construct BST with minimum search cost"
- **Approach:** D&C DP with frequency sum as cost
- **Example:** GFG — Optimal Binary Search Tree

Pattern 3: DP with Quadrangle Inequality

- **How to identify:** Cost function satisfies Monge property
- **Approach:** D&C DP or Knuth optimization
- **Example:** Codeforces — Yet Another Subarray Problem

13. Common Mistakes

- Cost function not satisfying quadrangle inequality (monotonicity doesn't hold)
- Cost computation not $O(1)$ (makes the algorithm $O(N^2 \log N)$ instead of $O(N \log N)$)
- Off-by-one in k range ($k < i$, so $k \leq \text{mid}-1$)
- Forgetting to handle the case where $i \leq j$ (can't have more groups than elements)
- Not using `long long` for large values

14. Edge Cases

- $k = 1$ (no partition needed)

- $k = n$ (each element is its own group)
- All elements equal
- Single element
- Cost function returns 0 for empty groups
- Very large cost values (use INF appropriately)

15. Variations

Knuth Optimization

Another monotonicity-based optimization for $O(N^2)$ DP. Used when the split point is monotonic and the cost function satisfies quadrangle inequality.

Alien's Trick (Lagrange Multiplier)

Used when K is a parameter. Binary search on the cost per group, then solve for unconstrained K .

1D/1D DP Optimization

A general class of DP where $dp[i] = \min(dp[j] + cost(j, i))$. Can be optimized with CHT, D&C, or Knuth depending on the cost function.

16. Related Algorithms/Data Structures

Algorithm	Connection
Knuth Optimization	Similar but for $O(N^2)$ to $O(N)$
Convex Hull Trick	Different optimization for 1D/1D DP
Quadrangle Inequality	Mathematical foundation
Segment Tree	Can be used when cost is hard to compute

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Array Partition	Codeforces	Basic D&C DP	Easy
Min Difficulty	LeetCode 1335	Job schedule	Easy

Medium

Problem	Platform	Pattern	Difficulty
Optimal BST	GFG	OBST with D&C	Medium
Minimum Cost to Cut	LeetCode 1547	Partition DP	Medium
Yet Another Subarray	Codeforces	D&C DP	Medium

Hard

Problem	Platform	Pattern	Difficulty
Guardians of the Galaxy	Codeforces	Complex D&C DP	Hard
Subarray Cost	Codeforces	Advanced D&C	Hard
Partition Array	Codeforces	D&C + CHT	Hard

18. Interview Explanation

"Divide and Conquer DP optimization is used for recurrences of the form $dp[i][j] = \min(dp[k][j-1] + cost(k+1, i))$ where the optimal split point is monotonic. Instead of checking all k for each i , I use a divide-and-conquer approach: compute the middle i 's optimal k , then recursively process the left and right halves with restricted k ranges. This reduces the complexity from $O(N^2K)$ to $O(NK \log N)$. The cost function must be $O(1)$ to be efficient."

19. Revision Notes

- **Key idea:** Monotonic optimal split point enables D&C approach
- **Recurrence:** $dp[i][j] = \min(dp[k][j-1] + cost(k+1, i))$
- **Algorithm:** For each j , compute dp for all i using D&C with restricted k ranges
- **Complexity:** $O(K \times N \log N)$ time, $O(N \times K)$ space
- **Prerequisite:** Cost function must satisfy quadrangle inequality
- **Common trap:** Cost computation not $O(1)$, non-monotonic opt points

20. Final Cheat Sheet

Divide and Conquer DP Optimization

When: $dp[i][j] = \min(dp[k][j-1] + cost(k+1, i))$, opt monotonic

Algorithm: D&C over i , restrict k range using monotonicity

Cost: Must be $O(1)$ or $O(\log N)$

Time: $O(K \times N \log N)$ Space: $O(N \times K)$

Edge: $k=1, k=n$, all equal elements

14. Knuth Optimization

1. Overview

Knuth optimization is a technique for DP recurrences of the form:

$$dp[i][j] = \min_{k \in [i, j-1]} (dp[i][k] + dp[k+1][j] + cost(i, j))$$

where the optimal split point $\text{opt}[i][j]$ is monotonic: $\text{opt}[i][j-1] \leq \text{opt}[i][j] \leq \text{opt}[i+1][j]$. This reduces the search space from $O(n^3)$ to $O(n^2)$.

2. Intuition

In interval DP (like MCM), we try all k from i to $j-1$. Knuth optimization says: for each interval $[i, j]$, you only need to check k in the range $[\text{opt}[i][j-1], \text{opt}[i+1][j]]$ — a much smaller range.

Analogy: Imagine you're cutting a rope at different positions. If you know the optimal cut for a shorter rope, the optimal cut for a longer rope can't be to the left of the shorter rope's optimal cut. This monotonicity drastically reduces the search space.

Core insight: The monotonicity of $\text{opt}[i][j]$ means:

- $\text{opt}[i][j-1] \leq \text{opt}[i][j]$ (as j increases, opt moves right)
- $\text{opt}[i][j] \leq \text{opt}[i+1][j]$ (as i increases, opt moves right) This is called the "knuth condition" on the DP.

3. When to Use It

- Interval DP problems where $\text{cost}(i, j)$ satisfies quadrangle inequality
- Matrix Chain Multiplication (if cost satisfies Monge property)
- Optimal Binary Search Tree
- Problems where the DP has the form $\text{dp}[i][j] = \min(\text{dp}[i][k] + \text{dp}[k+1][j] + \text{cost}(i, j))$
- When N is large (≥ 1000) and $O(n^3)$ is too slow

Trigger phrases: "interval DP optimization", "Knuth optimization", "optimal BST", "quadrangle inequality"

4. When Not to Use It

- When the cost function doesn't satisfy quadrangle inequality
- When the DP is not of the interval DP form
- When N is small ($O(n^3)$ is acceptable)
- When the cost function is not computable in $O(1)$
- When the monotonicity doesn't hold (check quadrangle inequality first)

5. Core Concepts

5.1 Monotonic Optimal Split Point

$\text{opt}[i][j-1] \leq \text{opt}[i][j] \leq \text{opt}[i+1][j]$ for all $i < j$.

5.2 Quadrangle Inequality

The cost function satisfies: $\text{cost}(a, c) + \text{cost}(b, d) \leq \text{cost}(a, d) + \text{cost}(b, c)$ for $a \leq b \leq c \leq d$.

5.3 Knuth Condition

The DP value and cost function together satisfy the monotonicity of opt .

5.4 Reduced Search Space

Instead of checking all k from i to $j-1$, check only from $\text{opt}[i][j-1]$ to $\text{opt}[i+1][j]$.

6. Step-by-Step Algorithm

1. Initialize $\text{dp}[i][i] = 0$ and $\text{opt}[i][i] = i$
2. For length from 2 to n :
 - For i from 1 to $n - \text{length} + 1$:
 - $j = i + \text{length} - 1$
 - Initialize $\text{dp}[i][j] = \text{INF}$
 - For k from $\text{opt}[i][j-1]$ to $\text{opt}[i+1][j]$:
 - $\text{cost} = \text{dp}[i][k] + \text{dp}[k+1][j] + \text{cost}(i, j)$
 - If $\text{cost} < \text{dp}[i][j]$: update $\text{dp}[i][j]$ and set $\text{opt}[i][j] = k$
3. Answer = $\text{dp}[1][n]$

7. Dry Run

Optimal BST with keys [1, 2, 3], frequencies [0.2, 0.3, 0.5]

Without Knuth: $O(n^3) = 27$ iterations With Knuth: $O(n^2) = \sim 9$ iterations

opt table:

- $\text{opt}[1][1] = 1, \text{opt}[2][2] = 2, \text{opt}[3][3] = 3$
- $\text{opt}[1][2]$: check k in $[\text{opt}[1][1], \text{opt}[2][2]] = [1, 2] \rightarrow 2$ iterations
- $\text{opt}[2][3]$: check k in $[\text{opt}[2][2], \text{opt}[3][3]] = [2, 3] \rightarrow 2$ iterations
- $\text{opt}[1][3]$: check k in $[\text{opt}[1][2], \text{opt}[2][3]] \rightarrow 2$ iterations

Total: 6 iterations instead of 9 (saving grows with n)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Knuth Optimization for Optimal BST
// dp[i][j] = min search cost for keys i..j
// freq[i] = frequency of key i

struct KnuthOptimization {
    int n;
    vector<long long> freq;
    vector<long long> prefix;
    vector<vector<long long>> dp;
    vector<vector<int>> opt;

    KnuthOptimization(const vector<long long>& freq)
        : n(freq.size() - 1), freq(freq) { // freq[1..n]
        prefix.resize(n + 1, 0);
        for (int i = 1; i <= n; i++) {
            prefix[i] = prefix[i - 1] + freq[i];
        }
    }
};
```

```

        dp.assign(n + 2, vector<long long>(n + 2, 0));
        opt.assign(n + 2, vector<int>(n + 2, 0));
        solve();
    }

    long long rangeSum(int i, int j) {
        return prefix[j] - prefix[i - 1];
    }

    void solve() {
        // Base: single key
        for (int i = 1; i <= n; i++) {
            dp[i][i] = freq[i]; // cost = frequency of that key
            opt[i][i] = i;
        }

        // Intervals of increasing length
        for (int len = 2; len <= n; len++) {
            for (int i = 1; i + len - 1 <= n; i++) {
                int j = i + len - 1;
                dp[i][j] = LLONG_MAX;

                // Knuth optimization: restricted k range
                int start = opt[i][j - 1];
                int end = opt[i + 1][j];

                for (int k = start; k <= end; k++) {
                    long long cost = dp[i][k - 1] + dp[k + 1][j] + rangeSum(i, j);
                    if (cost < dp[i][j]) {
                        dp[i][j] = cost;
                        opt[i][j] = k;
                    }
                }
            }
        }
    }

    long long getAnswer() {
        return dp[1][n];
    }
};

// Knuth for MCM (if cost satisfies quadrangle inequality)
long long mcmKnuth(const vector<int>& arr) {
    int n = arr.size() - 1;
    vector<vector<long long>> dp(n + 1, vector<long long>(n + 1, 0));
    vector<vector<int>> opt(n + 1, vector<int>(n + 1, 0));

    for (int i = 1; i < n; i++) {
        dp[i][i + 1] = (long long)arr[i - 1] * arr[i] * arr[i + 1];
        opt[i][i + 1] = i;
    }
}

```

```

    for (int len = 3; len <= n; len++) {
        for (int i = 1; i + len - 1 <= n; i++) {
            int j = i + len - 1;
            dp[i][j] = LLONG_MAX;

            for (int k = opt[i][j - 1]; k <= opt[i + 1][j]; k++) {
                long long cost = dp[i][k] + dp[k + 1][j]
                    + (long long)arr[i - 1] * arr[k] * arr[j];
                if (cost < dp[i][j]) {
                    dp[i][j] = cost;
                    opt[i][j] = k;
                }
            }
        }
    }
    return dp[1][n];
}

int main() {
    // Optimal BST example
    vector<long long> freq = {0, 20, 30, 50}; // 1-indexed, 3 keys
    KnuthOptimization knuth(freq);
    cout << "Optimal BST cost: " << knuth.getAnswer() << endl;

    // MCM with Knuth
    vector<int> arr = {10, 30, 5, 60};
    cout << "MCM with Knuth: " << mcmKnuth(arr) << endl; // 4500
    return 0;
}

```

9. Python Implementation

```

class KnuthOptimization:
    """Knuth optimization for Optimal BST"""

    def __init__(self, freq):
        # freq[1..n], freq[0] is dummy
        self.n = len(freq) - 1
        self.freq = freq

        self.prefix = [0] * (self.n + 1)
        for i in range(1, self.n + 1):
            self.prefix[i] = self.prefix[i - 1] + freq[i]

        self.dp = [[0] * (self.n + 2) for _ in range(self.n + 2)]
        self.opt = [[0] * (self.n + 2) for _ in range(self.n + 2)]
        self._solve()

    def _range_sum(self, i, j):
        return self.prefix[j] - self.prefix[i - 1]

```

```

def _solve(self):
    for i in range(1, self.n + 1):
        self.dp[i][i] = self.freq[i]
        self.opt[i][i] = i

    for length in range(2, self.n + 1):
        for i in range(1, self.n - length + 2):
            j = i + length - 1
            self.dp[i][j] = float('inf')

            start = self.opt[i][j - 1]
            end = self.opt[i + 1][j]

            for k in range(start, end + 1):
                cost = self.dp[i][k - 1] + self.dp[k + 1][j] +
self._range_sum(i, j)
                if cost < self.dp[i][j]:
                    self.dp[i][j] = cost
                    self.opt[i][j] = k

def get_answer(self):
    return self.dp[1][self.n]

def mcm_knuth(arr):
    n = len(arr) - 1
    dp = [[0] * (n + 1) for _ in range(n + 1)]
    opt = [[0] * (n + 1) for _ in range(n + 1)]

    for i in range(1, n):
        dp[i][i + 1] = arr[i - 1] * arr[i] * arr[i + 1]
        opt[i][i + 1] = i

    for length in range(3, n + 1):
        for i in range(1, n - length + 2):
            j = i + length - 1
            dp[i][j] = float('inf')

            for k in range(opt[i][j - 1], opt[i + 1][j] + 1):
                cost = dp[i][k] + dp[k + 1][j] + arr[i - 1] * arr[k] * arr[j]
                if cost < dp[i][j]:
                    dp[i][j] = cost
                    opt[i][j] = k

    return dp[1][n]

if __name__ == "__main__":
    freq = [0, 20, 30, 50]
    knuth = KnuthOptimization(freq)
    print(f"Optimal BST cost: {knuth.get_answer()}")

    print(f"MCM with Knuth: {mcm_knuth([10, 30, 5, 60])}") # 4500

```

10. Code Explanation

- **opt[i][j] tracking:** Stores the optimal split point for interval [i, j]. This is used to restrict the search for larger intervals.
- **Restricted k range:** For interval [i, j], only check k from `opt[i][j-1]` to `opt[i+1][j]`. This is the key optimization.
- **Base case for opt:** `opt[i][i] = i` (for single element, the split point is itself).
- **OBST cost:** The cost of searching a BST is the sum of frequencies of keys in the interval (each key's depth contributes its frequency).

11. Complexity Analysis

Aspect	Complexity
Time (without optimization)	$O(n^3)$
Time (with Knuth)	$O(n^2)$
Space	$O(n^2)$
Per interval	$O(\text{opt}[i+1][j] - \text{opt}[i][j-1] + 1) \rightarrow \text{amortized } O(1)$

12. Common Patterns

Pattern 1: Optimal BST

- **How to identify:** "Construct BST with minimum search cost for given frequencies"
- **Approach:** Knuth optimization on interval DP
- **Example:** GFG — Optimal Binary Search Tree

Pattern 2: Matrix Chain Multiplication (with Knuth condition)

- **How to identify:** MCM-like problems where cost satisfies quadrangle inequality
- **Approach:** Knuth optimization
- **Example:** Special case of MCM

Pattern 3: General Interval DP with Knuth

- **How to identify:** Interval DP where opt is monotonic
- **Approach:** Verify quadrangle inequality, then apply Knuth
- **Example:** Codeforces — Subarray Problems

13. Common Mistakes

- Applying Knuth without verifying the quadrangle inequality
- Off-by-one in opt array initialization (`opt[i][i] = i`)
- Not handling the case where `opt[i+1][j]` might be 0 (uninitialized for $j = i+1$)
- Using the wrong range (should be `opt[i][j-1]` to `opt[i+1][j]`, not `opt[i][j-1]` to `opt[i+1][j]`)
- Forgetting to set `opt[i][i]` for single-length intervals

14. Edge Cases

- $n = 1$ (single element, no optimization needed)
- $n = 2$ (only one split point, trivial)
- All frequencies equal
- Unbalanced frequencies
- Very large n ($O(n^2)$ is still heavy for $n > 5000$)

15. Variations

Knuth for 1D DP

Some 1D DP problems also satisfy the Knuth condition. $dp[i] = \min(dp[k] + cost(k, i))$ with monotonic opt.

Divide and Conquer DP

When the DP is of the form $dp[i][j] = \min(dp[k][j-1] + cost(k+1, i))$, D&C DP is used instead of Knuth.

Monge Array

An array where the quadrangle inequality holds. Many problems can be optimized if the cost matrix is a Monge array.

16. Related Algorithms/Data Structures

Algorithm	Connection
Divide and Conquer DP	Alternative for different DP form
Convex Hull Trick	Another DP optimization
Quadrangle Inequality	Mathematical foundation
Monge Array	Property of cost matrix

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Optimal BST	GFG	Standard Knuth	Easy
MCM with Knuth	GFG	MCM optimization	Easy

Medium

Problem	Platform	Pattern	Difficulty
Subarray Cost	Codeforces	Knuth on interval DP	Medium

Problem	Platform	Pattern	Difficulty
String Cutting	Codeforces	Interval DP + Knuth	Medium

Hard

Problem	Platform	Pattern	Difficulty
Yet Another Subarray	Codeforces	Complex Knuth	Hard
Guards	Codeforces	Advanced Knuth	Hard

18. Interview Explanation

"Knuth optimization reduces interval DP from $O(n^3)$ to $O(n^2)$ by exploiting the monotonicity of optimal split points. For intervals, $\text{opt}[i][j-1] \leq \text{opt}[i][j] \leq \text{opt}[i+1][j]$. This means for interval $[i, j]$, I only need to check k from $\text{opt}[i][j-1]$ to $\text{opt}[i+1][j]$ instead of all k from i to $j-1$. The optimization requires the cost function to satisfy the quadrangle inequality. It's commonly used in Optimal BST and certain MCM problems."

19. Revision Notes

- **Key idea:** $\text{opt}[i][j-1] \leq \text{opt}[i][j] \leq \text{opt}[i+1][j]$
- **Recurrence:** $\text{dp}[i][j] = \min(\text{dp}[i][k] + \text{dp}[k+1][j] + \text{cost}(i, j))$
- **Optimized range:** k from $\text{opt}[i][j-1]$ to $\text{opt}[i+1][j]$
- **Complexity:** $O(n^2)$ time, $O(n^2)$ space
- **Prerequisite:** Cost function satisfies quadrangle inequality
- **Common trap:** Verifying the Knuth condition, off-by-one in opt

20. Final Cheat Sheet

Knuth Optimization

When: Interval DP with monotonic opt : $\text{opt}[i][j-1] \leq \text{opt}[i][j] \leq \text{opt}[i+1][j]$
 Range: k in $[\text{opt}[i][j-1], \text{opt}[i+1][j]]$
 Time: $O(n^2)$ Space: $O(n^2)$
 Cost: Must satisfy quadrangle inequality
 Edge: $n=1$, $n=2$, all equal frequencies

15. Profile DP (DP with Broken Profile)

1. Overview

Profile DP (also called DP with Broken Profile or DP on Grid with State) is a technique used for problems on grids where the state is the configuration of a "frontier" — a line that separates processed cells from unprocessed ones. It's commonly used for tiling problems and counting valid configurations on a grid.

2. Intuition

When processing a grid cell by cell (row by row, column by column), the "profile" is the state of the boundary between processed and unprocessed cells. For a grid with N columns, the profile is typically a bitmask of length N encoding the state of the current column's cells.

Analogy: Imagine you're painting a wall row by row, one cell at a time. The "profile" is the jagged line between painted and unpainted cells. The state of this line determines what you can paint next.

Core insight: When filling a grid cell by cell, the only information needed from the past is the state of the cells on the current "frontier." This state can be represented as a bitmask of size N (number of columns), and there are 2^N possible profiles.

3. When to Use It

- Tiling problems (dominoes, trominoes, arbitrary polyominoes)
- Counting number of ways to fill a grid with certain shapes
- Problems with constraints on adjacent cells
- Grid coloring problems with connectivity constraints
- Problems where the grid dimensions are small ($N \leq 10-15$)

Trigger phrases: "tiling", "domino", "tromino", "fill the grid", "number of ways to tile", "broken profile"

4. When Not to Use It

- When the grid is too large ($N > 20$, 2^N is too large)
- When the problem can be solved with simple DP (e.g., 1D DP for domino tiling)
- When the shapes are very complex (profile state space explodes)
- When the problem has a closed-form formula (e.g., Fibonacci for $2 \times N$ domino tiling)

5. Core Concepts

5.1 Profile

A bitmask of length N (columns) representing which cells in the current column are occupied (1) or empty (0). The profile is the boundary between filled and unfilled cells.

5.2 Cell-by-Cell Processing

Process the grid one cell at a time, typically in row-major order (left to right, top to bottom).

5.3 Transition

From a current profile, try to place a tile covering the current cell and possibly adjacent cells. The new profile is computed based on which cells become occupied.

5.4 State Compression

The profile is stored as an integer bitmask to enable DP with 2^N states.

6. Step-by-Step Algorithm (Tiling with Dominoes)

1. Let `grid[m][n]`, process cell by cell (row-major)
2. `dp[pos][profile]` = number of ways to tile up to position `pos` with current profile
3. For each cell (r, c):
 - If the cell is already occupied (profile has 1 at this position), skip it (move to next)
 - Try placing a horizontal domino: covers (r, c) and (r, c+1)
 - Try placing a vertical domino: covers (r, c) and (r+1, c)
 - Update profile accordingly
4. Answer = `dp[last][0]` (all cells filled, no occupied cells)

7. Dry Run

2×2 grid, tiling with dominoes.

Initial profile: 00 (no occupied cells in the current column)

Processing cell (0,0):

- Horizontal: covers (0,0) and (0,1). Profile becomes 00 (move to next row after column 1)
- Vertical: covers (0,0) and (1,0). Profile becomes 10 (first column occupied in next row)

Processing cell (0,1):

- From profile 00 at (0,0):
 - Cell (0,1) is free, place horizontal: covers (0,1) and next row... wait, (0,1) is the last column. Horizontal doesn't fit.
 - Vertical: covers (0,1) and (1,1). Profile becomes 01.

... (full trace shows 2 ways: two horizontal dominoes or two vertical dominoes)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

// Tiling a grid with dominoes (1x2 or 2x1)
// Profile DP / Broken Profile DP

class DominoTiling {
    int rows, cols;
    vector<vector<int>>> dp; // dp[pos][profile]

public:
    DominoTiling(int rows, int cols) : rows(rows), cols(cols) {
        // Ensure rows <= cols for efficiency (minimize 2^cols)
        if (rows > cols) swap(rows, cols);
    }

    int countWays() {
        int totalCells = rows * cols;
```

```

    int profileSize = 1 << rows;
    dp.assign(totalCells + 1, vector<int>(profileSize, 0));
    dp[0][0] = 1;

    for (int pos = 0; pos < totalCells; pos++) {
        for (int mask = 0; mask < profileSize; mask++) {
            if (dp[pos][mask] == 0) continue;

            int r = pos % rows;
            int c = pos / rows;

            // If current cell is already occupied in profile
            if (mask & (1 << r)) {
                // Move to next cell, clear this bit
                int newMask = mask & ~(1 << r);
                dp[pos + 1][newMask] = (dp[pos + 1][newMask] + dp[pos][mask])
% MOD;

                continue;
            }

            // Try vertical domino: covers (r, c) and (r+1, c)
            if (r + 1 < rows && !(mask & (1 << (r + 1)))) {
                int newMask = mask | (1 << (r + 1)); // mark r+1 as occupied
                dp[pos + 1][newMask] = (dp[pos + 1][newMask] + dp[pos][mask])
% MOD;

            }

            // Try horizontal domino: covers (r, c) and (r, c+1)
            if (c + 1 < cols && !(mask & (1 << r))) {
                // The cell (r, c+1) is in the next column
                // We'll handle it in the next iteration
                // No change to profile for this column
                dp[pos + 1][mask] = (dp[pos + 1][mask] + dp[pos][mask]) % MOD;
            }
        }
    }

    return dp[totalCells][0];
}

};

// Alternative: DP on grid with bitmask for current row
// For problems like "number of ways to place non-attacking rooks"
class GridProfileDP {
public:
    // Count ways to place K items on N×N grid with no two sharing row/col
    int countPlacements(int n, int k) {
        vector<vector<int>> dp(n + 1, vector<int>(1 << n, 0));
        dp[0][0] = 1;

        for (int row = 0; row < n; row++) {
            for (int mask = 0; mask < (1 << n); mask++) {
                if (dp[row][mask] == 0) continue;

```

```

        // Skip this row (don't place anything)
        dp[row + 1][mask] = (dp[row + 1][mask] + dp[row][mask]) % MOD;

        // Place an item in column j of this row
        for (int col = 0; col < n; col++) {
            if (!(mask & (1 << col))) {
                int newMask = mask | (1 << col);
                dp[row + 1][newMask] = (dp[row + 1][newMask] + dp[row]
[mask]) % MOD;
            }
        }
    }
}

// Sum over all masks with exactly k bits set
int ans = 0;
for (int mask = 0; mask < (1 << n); mask++) {
    if (__builtin_popcount(mask) == k) {
        ans = (ans + dp[n][mask]) % MOD;
    }
}
return ans;
}
};

int main() {
    DominoTiling dt(2, 3);
    cout << "Ways to tile 2x3: " << dt.countWays() << endl; // 3

    GridProfileDP gp;
    cout << "Placements on 4x4 with 2 items: " << gp.countPlacements(4, 2) <<
endl;
    return 0;
}

```

9. Python Implementation

```

MOD = 10**9 + 7

class DominoTiling:
    def __init__(self, rows, cols):
        self.rows = min(rows, cols) # minimize state space
        self.cols = max(rows, cols)

    def count_ways(self):
        total_cells = self.rows * self.cols
        profile_size = 1 << self.rows
        dp = [[0] * profile_size for _ in range(total_cells + 1)]
        dp[0][0] = 1

```

```

        for pos in range(total_cells):
            for mask in range(profile_size):
                if dp[pos][mask] == 0:
                    continue

                r = pos % self.rows
                c = pos // self.rows

                # Cell already occupied
                if mask & (1 << r):
                    new_mask = mask & ~(1 << r)
                    dp[pos + 1][new_mask] = (dp[pos + 1][new_mask] + dp[pos]
[mask])) % MOD
                    continue

                # Vertical domino
                if r + 1 < self.rows and not (mask & (1 << (r + 1))):
                    new_mask = mask | (1 << (r + 1))
                    dp[pos + 1][new_mask] = (dp[pos + 1][new_mask] + dp[pos]
[mask])) % MOD

                # Horizontal domino
                if c + 1 < self.cols:
                    dp[pos + 1][mask] = (dp[pos + 1][mask] + dp[pos][mask]) % MOD

            return dp[total_cells][0]

class GridProfileDP:
    def count_placements(self, n, k):
        dp = [[0] * (1 << n) for _ in range(n + 1)]
        dp[0][0] = 1

        for row in range(n):
            for mask in range(1 << n):
                if dp[row][mask] == 0:
                    continue

                # Skip this row
                dp[row + 1][mask] = (dp[row + 1][mask] + dp[row][mask]) % MOD

                # Place in column j
                for col in range(n):
                    if not (mask & (1 << col)):
                        new_mask = mask | (1 << col)
                        dp[row + 1][new_mask] = (dp[row + 1][new_mask] + dp[row]
[mask])) % MOD

        ans = 0
        for mask in range(1 << n):
            if bin(mask).count('1') == k:
                ans = (ans + dp[n][mask]) % MOD
        return ans

```

```

if __name__ == "__main__":
    dt = DominoTiling(2, 3)
    print(f"Ways to tile 2x3: {dt.count_ways()}") # 3

    gp = GridProfileDP()
    print(f"Placements on 4x4 with 2 items: {gp.count_placements(4, 2)}")

```

10. Code Explanation

- **Profile:** A bitmask of length `rows` (we choose $\text{rows} \leq \text{cols}$ to minimize state space). Bit $i = 1$ means cell $(r=i, c)$ will be occupied in the next row.
- **Cell-by-cell:** The position `pos` goes from 0 to $\text{rows} \times \text{cols} - 1$. $r = \text{pos} \% \text{rows}$, $c = \text{pos} / \text{rows}$.
- **If cell occupied:** If the profile says the current cell is occupied (from a vertical domino placed above), we clear the bit and move to the next cell.
- **Vertical domino:** Place a domino covering (r, c) and $(r+1, c)$. Mark $(r+1)$ in the profile.
- **Horizontal domino:** Place a domino covering (r, c) and $(r, c+1)$. Since $(r, c+1)$ is in the next column, we don't need to change the profile for this column.
- **Final answer:** `dp[totalCells][0]` — all cells filled, no pending occupied cells.

11. Complexity Analysis

Aspect	Complexity
Time	$O(\text{totalCells} \times 2^{\text{rows}} \times \text{transitions})$
Space	$O(2^{\text{rows}})$ or $O(\text{totalCells} \times 2^{\text{rows}})$
Transitions per state	$O(1)$ or $O(\text{rows})$

12. Common Patterns

Pattern 1: Domino Tiling

- **How to identify:** "Number of ways to tile a grid with 1×2 or 2×1 dominoes"
- **Approach:** Profile DP with bitmask for current column
- **Example:** LeetCode 1240 — Tiling a Rectangle

Pattern 2: Non-Attacking Placements

- **How to identify:** "Place items such that no two share row/column"
- **Approach:** Row-by-row DP with bitmask of used columns
- **Example:** N-Queens (count), LeetCode 52 — N-Queens II

Pattern 3: Grid Coloring with Adjacent Constraints

- **How to identify:** "Color cells such that no two adjacent cells have same color"
- **Approach:** Profile DP with color state per column
- **Example:** Codeforces — Grid Coloring

Pattern 4: Polyomino Tiling

- **How to identify:** "Tile with L-shaped or T-shaped pieces"
- **Approach:** Profile DP with extended state for multiple rows
- **Example:** USACO — Tiling

13. Common Mistakes

- Not minimizing the profile dimension (should be $\min(\text{rows}, \text{cols})$)
- Off-by-one in bit operations (forgetting that bit 0 = column 0)
- Incorrect transition when cell is already occupied
- Forgetting the modulo operation
- Not handling the case where horizontal domino extends beyond the grid
- Using too much memory (use `dp[pos & 1]` for space optimization)

14. Edge Cases

- $1 \times N$ grid (linear, can be solved with simple DP)
- $N \times M$ grid where N or M is 0
- Grid with obstacles (modify transitions to check if cell is blocked)
- Grid with odd number of cells (impossible to tile with dominoes)
- Very small grids (1×1 , 2×2)

15. Variations

DP with Broken Profile (True Form)

Process cells column by column within a row. The "broken" profile is the state of cells in the current column that are above the current row.

3D Profile DP

For 3D grids or more complex shapes. State space grows exponentially.

Inclusion-Exclusion Profile

When obstacles are present, use profile DP with inclusion-exclusion to handle restrictions.

16. Related Algorithms/Data Structures

Algorithm	Connection
Bitmask DP	Foundation of profile DP
Backtracking + Memoization	Alternative to iterative profile DP
Transfer-matrix Method	Profile DP + matrix exponentiation for large grids
Burnside's Lemma	For counting tilings with symmetry

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Domino Tiling $2 \times N$	GFG	Simple DP	Easy
Tiling with Dominoes	LeetCode	Profile DP	Easy

Medium

Problem	Platform	Pattern	Difficulty
Tiling a Rectangle	LeetCode 1240	Profile DP	Medium
N-Queens II	LeetCode 52	Row-by-row DP	Medium
Number of Ways to Tile	Codeforces	Profile DP	Medium

Hard

Problem	Platform	Pattern	Difficulty
Polyomino Tiling	USACO	Complex profile	Hard
Grid Coloring	Codeforces	Profile DP	Hard
Tiling with Trominoes	GFG	Profile DP	Hard

18. Interview Explanation

"Profile DP is used for grid problems where we process cells one by one and maintain a bitmask of the current frontier. The profile represents which cells in the current column are occupied. For each cell, if it's already occupied, I clear it and move on. Otherwise, I try placing tiles (vertical or horizontal) and update the profile accordingly. The state space is $O(2^{\min(\text{rows}, \text{cols})})$ and we process each cell once. It's commonly used for tiling problems."

19. Revision Notes

- **Key idea:** Bitmask of current frontier (profile) during cell-by-cell processing
- **State:** `dp[pos][profile]` = number of ways up to position pos with given profile
- **Profile dimension:** Choose $\text{rows} \leq \text{cols}$ to minimize 2^{rows}
- **Transitions:** Place vertical/horizontal tile, or skip if cell occupied
- **Complexity:** $O(\text{totalCells} \times 2^{\text{rows}})$ time, $O(2^{\text{rows}})$ space
- **Common trap:** Not minimizing profile dimension, wrong bit operations
- **Answer:** `dp[totalCells][0]` — all cells filled, no pending occupied cells

20. Final Cheat Sheet

Profile DP (Broken Profile)

When: Grid tiling, placement problems with small dimension

```

State: dp[pos][profile] where profile is bitmask of frontier
Profile: min(rows, cols) bits
Transitions: vertical/horizontal tile placement
Time:  $O(\text{rows} \times \text{cols} \times 2^{\min(\text{rows}, \text{cols})})$ 
Space:  $O(2^{\min(\text{rows}, \text{cols})})$ 
Edge: odd total cells, obstacles,  $1 \times N$  grid

```

16. Rerooting DP

1. Overview

Rerooting DP (also called Re-rooting DP or Tree DP with Two Passes) is a technique used to compute a DP value for **every node as the root** of a tree. Instead of running DP separately for each root ($O(N^2)$), we do two DFS passes: one to compute subtree values, and another to propagate parent values to children. Total time is $O(N)$.

2. Intuition

For many tree problems, we need to compute some value for each node as if it were the root. For example, "sum of distances from each node to all other nodes." Naively, we could run DP from each node, but that's $O(N^2)$.

Analogy: Imagine you're a delivery person and you need to know the total distance from your home to every house in a tree-shaped neighborhood. If you move to a neighbor's house, the distance to all houses on your side of the tree increases by 1, and the distance to all houses on the other side decreases by 1. You can compute this efficiently using the subtree sizes.

Core insight: The first pass computes subtree values (downward DP). The second pass uses the parent's value to compute the child's value (upward DP). The key formula relates a node's value to its parent's value:

$$\text{dp}[\text{child}] = \text{dp}[\text{parent}] + (n - 2 * \text{size}[\text{child}]) * \text{edge_weight}$$

This is because moving the root from parent to child increases distance to $(n - \text{size}[\text{child}])$ nodes and decreases distance to $\text{size}[\text{child}]$ nodes.

3. When to Use It

- Sum of distances from each node to all other nodes
- Computing any root-dependent metric efficiently for all nodes
- Tree DP where the answer is needed for every node as root
- Problems where the formula for rerooting is known (depends on subtree sizes)
- CP problems with N up to 2×10^5 where $O(N^2)$ is impossible

Trigger phrases: "for each node", "sum of distances from each node", "reroot", "two-pass DFS", "tree DP all roots"

4. When Not to Use It

- When you only need the answer for one root (just run DFS once)
- When the tree is very small ($N \leq 1000$, $O(N^2)$ is fine)
- When the DP value cannot be expressed in terms of parent's value and subtree sizes
- When the tree is not static (changes between queries)

5. Core Concepts

5.1 First Pass (Downward DP)

Compute for each node:

- `subtreeSize[u]`: size of subtree rooted at u
- `dpDown[u]`: the DP value for the subtree of u (assuming u is root of its subtree)

5.2 Second Pass (Upward DP / Rerooting)

Propagate the full answer from parent to child:

- `dp[u]`: the final answer for u as root of the entire tree
- Compute `dp[child]` from `dp[parent]` using the rerooting formula

5.3 Rerooting Formula

For sum of distances:

$$dp[child] = dp[parent] + (n - 2 * size[child]) * weight$$

For other problems, the formula varies but typically involves the parent's dp value and the child's subtree size.

6. Step-by-Step Algorithm (Sum of Distances)

1. Build adjacency list of tree
2. First DFS (from root, say 0):
 - Compute `subtreeSize[u] = 1 + sum(subtreeSize[child])`
 - Compute `dpDown[u] = sum(dpDown[child] + subtreeSize[child])` (sum of distances within subtree)
3. Set `dp[0] = dpDown[0]` (root's answer is already complete)
4. Second DFS (reroot):
 - For each child v of u:
 - `dp[v] = dp[u] + (n - 2 * subtreeSize[v]) * weight(u, v)`
 - Recursively process v

7. Dry Run

Tree: 0-1-2 (path, $n=3$), all edges weight 1

First DFS (root=0):

- $\text{subtreeSize}[2] = 1, \text{dpDown}[2] = 0$
- $\text{subtreeSize}[1] = 1 + 1 = 2, \text{dpDown}[1] = 0 + (0 + 1) = 1$
- $\text{subtreeSize}[0] = 1 + 2 = 3, \text{dpDown}[0] = 0 + (1 + 2) = 3$

Second DFS:

- $\text{dp}[0] = 3$
- For child 1: $\text{dp}[1] = 3 + (3 - 2 \cdot 2) \cdot 1 = 3 + (-1) = 2$
- For child 2: $\text{dp}[2] = 2 + (3 - 2 \cdot 1) \cdot 1 = 2 + 1 = 3$

Answer: $\text{dp}[0]=3, \text{dp}[1]=2, \text{dp}[2]=3$

Check manually:

- Node 0: distances $0+1+2 = 3 \checkmark$
- Node 1: distances $1+0+1 = 2 \checkmark$
- Node 2: distances $2+1+0 = 3 \checkmark$

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Sum of Distances in Tree (LeetCode 834)
class RerootingDP {
    int n;
    vector<vector<pair<int, int>>> adj; // (neighbor, weight)
    vector<long long> dpDown, dp;
    vector<int> subtreeSize;

    // First DFS: compute subtree sizes and dpDown
    void dfs1(int u, int parent) {
        subtreeSize[u] = 1;
        dpDown[u] = 0;
        for (auto [v, w] : adj[u]) {
            if (v == parent) continue;
            dfs1(v, u);
            subtreeSize[u] += subtreeSize[v];
            dpDown[u] += dpDown[v] + (long long)subtreeSize[v] * w;
        }
    }

    // Second DFS: reroot
    void dfs2(int u, int parent) {
        for (auto [v, w] : adj[u]) {
            if (v == parent) continue;
            // Rerooting formula
            dp[v] = dp[u] + (long long)(n - 2 * subtreeSize[v]) * w;
            dfs2(v, u);
        }
    }
}
```

```

public:
    RerootingDP(int n) : n(n), adj(n), dpDown(n), dp(n), subtreeSize(n) {}

    void addEdge(int u, int v, int w = 1) {
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }

    vector<long long> solve() {
        dfs1(0, -1);
        dp[0] = dpDown[0];
        dfs2(0, -1);
        return dp;
    }
};

// General rerooting template
// For problems where the DP value is a function of children's DP values
// and the rerooting formula is known

template<typename T>
class RerootingTemplate {
    int n;
    vector<vector<int>>> adj;
    vector<T> dpDown, dp;

    // First pass: compute subtree DP
    T dfs1(int u, int parent) {
        T res = identity(); // base value for leaf
        for (int v : adj[u]) {
            if (v == parent) continue;
            T childVal = dfs1(v, u);
            res = merge(res, childVal, u, v);
        }
        return dpDown[u] = res;
    }

    // Second pass: reroot
    void dfs2(int u, int parent, T parentVal) {
        // Compute prefix and suffix for efficient merging
        int k = adj[u].size();
        vector<T> prefix(k), suffix(k);

        for (int i = 0; i < k; i++) {
            int v = adj[u][i];
            if (v == parent) {
                prefix[i] = parentVal;
            } else {
                prefix[i] = dpDown[v];
            }
            if (i > 0) prefix[i] = merge(prefix[i-1], prefix[i], u, -1);
        }

        for (int i = k - 1; i >= 0; i--) {

```

```

        int v = adj[u][i];
        if (v == parent) {
            suffix[i] = parentVal;
        } else {
            suffix[i] = dpDown[v];
        }
        if (i + 1 < k) suffix[i] = merge(suffix[i], suffix[i+1], u, -1);
    }

    for (int i = 0; i < k; i++) {
        int v = adj[u][i];
        if (v == parent) continue;

        // Merge all children except v, plus parent's contribution
        T withoutV = identity();
        if (i > 0) withoutV = merge(withoutV, prefix[i-1], u, -1);
        if (i + 1 < k) withoutV = merge(withoutV, suffix[i+1], u, -1);

        // Complete the DP for v
        dp[v] = merge(withoutV, identity(), v, -1); // finalize
        dfs2(v, u, withoutV);
    }
}

public:
    virtual T identity() = 0;
    virtual T merge(const T& a, const T& b, int node, int child) = 0;

    vector<T> solve() {
        dfs1(0, -1);
        dp[0] = dpDown[0];
        dfs2(0, -1, identity());
        return dp;
    }
};

int main() {
    // Sum of Distances example
    RerootingDP solver(3);
    solver.addEdge(0, 1);
    solver.addEdge(1, 2);

    auto result = solver.solve();
    for (int i = 0; i < 3; i++) {
        cout << "Sum of distances from " << i << ": " << result[i] << endl;
    }
    // 0: 3, 1: 2, 2: 3

    return 0;
}

```

9. Python Implementation

```

class RerootingDP:
    """Sum of Distances in Tree"""

    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]
        self.dp_down = [0] * n
        self.dp = [0] * n
        self.subtree_size = [0] * n

    def add_edge(self, u, v, w=1):
        self.adj[u].append((v, w))
        self.adj[v].append((u, w))

    def _dfs1(self, u, parent):
        self.subtree_size[u] = 1
        self.dp_down[u] = 0
        for v, w in self.adj[u]:
            if v == parent:
                continue
            self._dfs1(v, u)
            self.subtree_size[u] += self.subtree_size[v]
            self.dp_down[u] += self.dp_down[v] + self.subtree_size[v] * w

    def _dfs2(self, u, parent):
        for v, w in self.adj[u]:
            if v == parent:
                continue
            self.dp[v] = self.dp[u] + (self.n - 2 * self.subtree_size[v]) * w
            self._dfs2(v, u)

    def solve(self):
        self._dfs1(0, -1)
        self.dp[0] = self.dp_down[0]
        self._dfs2(0, -1)
        return self.dp

# Example for a different rerooting problem:
# For each node, find the maximum distance to any other node
class RerootingMaxDistance:
    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]
        self.dp_down = [0] * n # max distance to a leaf in subtree
        self.dp_down2 = [0] * n # second max
        self.dp = [0] * n # final answer

    def add_edge(self, u, v, w=1):
        self.adj[u].append((v, w))
        self.adj[v].append((u, w))

```

```

def _dfs1(self, u, parent):
    for v, w in self.adj[u]:
        if v == parent:
            continue
        self._dfs1(v, u)
        dist = self.dp_down[v] + w
        if dist > self.dp_down[u]:
            self.dp_down2[u] = self.dp_down[u]
            self.dp_down[u] = dist
        elif dist > self.dp_down2[u]:
            self.dp_down2[u] = dist

def _dfs2(self, u, parent, parent_dist):
    # max distance to any node for u
    self.dp[u] = max(self.dp_down[u], parent_dist)

    for v, w in self.adj[u]:
        if v == parent:
            continue

        # Best distance from u that doesn't go through v
        if self.dp_down[v] + w == self.dp_down[u]:
            best_from_u = max(self.dp_down2[u], parent_dist)
        else:
            best_from_u = max(self.dp_down[u], parent_dist)

        self._dfs2(v, u, best_from_u + w)

def solve(self):
    self._dfs1(0, -1)
    self._dfs2(0, -1, 0)
    return self.dp

if __name__ == "__main__":
    # Sum of distances
    solver = RerootingDP(3)
    solver.add_edge(0, 1)
    solver.add_edge(1, 2)
    result = solver.solve()
    for i in range(3):
        print(f"Sum of distances from {i}: {result[i]}")
    # 0: 3, 1: 2, 2: 3

    # Max distance
    solver2 = RerootingMaxDistance(4)
    solver2.add_edge(0, 1)
    solver2.add_edge(1, 2)
    solver2.add_edge(1, 3)
    result2 = solver2.solve()
    for i in range(4):
        print(f"Max distance from {i}: {result2[i]}")

```

10. Code Explanation

- **First DFS:** Computes subtree sizes and the DP value for each node's subtree. $dpDown[u]$ = sum of distances from u to all nodes in its subtree.
- **Rerooting formula:** $dp[v] = dp[u] + (n - 2 \times size[v]) \times w$. When moving root from u to v, distances to nodes in v's subtree decrease by w, and distances to all other nodes increase by w.
- **General template:** For more complex DP where the rerooting formula isn't just a simple addition, use prefix/suffix arrays to efficiently compute the contribution of all children except one.
- **Max distance example:** Track the top two longest distances to leaves. When rerooting, use the best alternative (either the second best in subtree or the parent's best) that doesn't go through the current child.

11. Complexity Analysis

Aspect	Complexity
First DFS	$O(N)$
Second DFS	$O(N)$
Total	$O(N)$
Space	$O(N)$

12. Common Patterns

Pattern 1: Sum of Distances

- **How to identify:** "Sum of distances from each node to all others"
- **Approach:** $dp[v] = dp[u] + (n - 2 \times size[v]) \times w$
- **Example:** LeetCode 834 — Sum of Distances in Tree

Pattern 2: Tree Diameter (all nodes)

- **How to identify:** "For each node, find the farthest node's distance"
- **Approach:** Track top two depths, reroot with max of parent's best
- **Example:** CSES — Tree Distances

Pattern 3: Max/Min of Some Value

- **How to identify:** "For each node, compute some function that depends on the whole tree"
- **Approach:** General template with merge function
- **Example:** Codeforces — Tree DP reroot

Pattern 4: Subtree XOR / Sum

- **How to identify:** "For each node, compute XOR/sum of all nodes in its tree"
- **Approach:** Simple reroot with aggregate values
- **Example:** Codeforces — XOR Tree

13. Common Mistakes

- Off-by-one in subtree size (forgetting to count the node itself)
- Using the wrong rerooting formula (each problem has its own formula)
- Not handling the case where a node has no children (leaf)
- Stack overflow in recursion for deep trees (use iterative DFS or increase stack)
- Forgetting to consider edge weights (assuming weight = 1)

14. Edge Cases

- Single node ($n = 1$): all distances are 0
- Path tree (linear): distances increase to edges, decrease in center
- Star tree (one center, many leaves): center has minimum sum of distances
- Complete binary tree
- Tree with weighted edges (not just unit weight)

15. Variations

Rerooting with Complex DP

When the DP value is a complex struct (not just a single number), use the general template with prefix/suffix arrays.

Rerooting for DP with Constraints

When the DP has multiple states (e.g., $dp[node][0/1]$), reroot for each state separately.

Iterative Rerooting

Use an explicit stack to avoid recursion limits for deep trees.

16. Related Algorithms/Data Structures

Algorithm	Connection
Tree DP (Single Root)	Foundation of rerooting
Tree Diameter	Can be computed with rerooting
Centroid Decomposition	Alternative for some tree problems
Heavy-Light Decomposition	For path queries, not rerooting

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Tree Distances	CSES	Max distance from each node	Easy
Sum of Distances	LeetCode 834	Sum of distances	Easy

Medium

Problem	Platform	Pattern	Difficulty
Tree Distances II	CSES	Sum of distances (weighted)	Medium
Minimum Height Trees	LeetCode 310	Tree centroids	Medium
Collect Coins in a Tree	LeetCode 2603	Rerooting DP	Medium

Hard

Problem	Platform	Pattern	Difficulty
Tree Xor	Codeforces	Rerooting with XOR	Hard
Maximum Subtree	Codeforces	Complex rerooting	Hard
Tree DP reroot	AtCoder	Advanced rerooting	Hard

18. Interview Explanation

"Rerooting DP computes values for every node as root in $O(N)$ using two DFS passes. The first pass computes subtree sizes and DP values bottom-up. The second pass propagates the full answer from parent to child using a rerooting formula. For sum of distances, the formula is $dp[child] = dp[parent] + (n - 2 \times size[child]) \times w$. This works because moving the root across an edge changes distances to nodes in the child's subtree by $-w$ and to all other nodes by $+w$."

19. Revision Notes

- **Key idea:** Two-pass DFS: first subtree, then reroot
- **First pass:** Compute subtree sizes and dpDown
- **Second pass:** Propagate dp from parent to child using rerooting formula
- **Sum of distances:** $dp[v] = dp[u] + (n - 2 \times size[v]) \times w$
- **General template:** Use prefix/suffix arrays for complex merges
- **Complexity:** $O(N)$ time, $O(N)$ space
- **Common trap:** Wrong formula, forgetting edge weights, recursion depth

20. Final Cheat Sheet

Rerooting DP

When: Need DP value for every node as root

Pass 1: DFS compute subtree sizes and dpDown

Pass 2: DFS propagate dp from parent to child

Formula: $dp[v] = dp[u] + (n - 2 \times size[v]) \times w$ (for sum of distances)

For general: use prefix/suffix arrays

Time: $O(N)$ Space: $O(N)$

Edge: single node, path tree, star tree, weighted edges

Master these patterns through practice. Each pattern has its own trigger phrases, recurrence structure, and optimization techniques. The key is to identify which pattern matches the problem and apply the correct DP formulation.